CHENNAI INSTITUTE OF TECHNOLOGY

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

PROJECT-BASED LEARNING (PBL) REPORT

MOBILE APPLICATION DEVELOPMENT (CS4504)

GROUPDEAL

LOCALITY-BASED GROUP BUYING WITH DYNAMIC PRICING & SUPPLIER BIDDING

“BUY TOGETHER, SAVE MORE”

Submitted in partial fulfilment of the requirements for the
Project-Based Learning component of Mobile Application Development

Submitted by

ARUL MURUGAN M (24CS0084)
RITVIK DODDAPANENI (24CS0755)

Under the guidance of

Mrs. M. SINDHU
[Assistant Professor, Department of CSE]

OCTOBER 2026

Academic Year 2026–2027

Vision of the Institute:

To be an eminent centre for Academia, Industry and Research by imparting knowledge, relevant practices and inculcating human values to address global challenges through novelty and sustainability.

Mission of the Institute:

IM1. To creates next generation leaders by effective teaching learning methodologies and instill scientific spark in them to meet the global challenges.

IM2. To transform lives through deployment of emerging technology, novelty and sustainability.

IM3. To inculcate human values and ethical principles to cater the societal needs.

IM4. To contributes towards the research ecosystem by providing a suitable, effective platform for interaction between industry, academia and R & D establishments.

IM5. To nurture incubation centers enabling structured entrepreneurship and start-ups.

Vision of the Department:

To Excel in the emerging areas of Computer Science and Engineering by imparting knowledge, relevant practices and inculcating human values to transform the students as potential resources to contribute innovatively through advanced computing in real-time situations.

Mission of the Department:

DM1: To provide strong fundamentals and technical skills for Computer Science applications through effective teaching learning methodologies.

DM2: To transform lives of the students by nurturing ethical values, creativity, and novelty to become Entrepreneurs and establish start-ups.

DM3: To habituate the students to focus on sustainable solutions to improve the quality of life and the welfare of the society.

DM4: To enhance the fabric of research in computing through collaborative linkages with Industry and academia.

DM5: To inculcate learning of the emerging technologies to pursue higher studies leading to lifelong learning.

BONAFIDE CERTIFICATE

This is to certify that the Project–Based Learning report titled **"GroupDeal: Locality-Based Group Buying with Dynamic Pricing & Supplier Bidding"** is a Bonafide record of work carried out by **ARUL MURUGAN M, 24CS0084** and **RITVIK DODDAPANENI, 24CS0755** of the **Department of Computer Science and Engineering, Chennai Institute of Technology**, as part of the continuous, mentor–guided Project-Based Learning **(PBL)** component of the Mobile Application Development (CS4504) course during the academic year **2026–2027**. This report reflects the team's work across the review cycles listed in the PBL Course and Team Details, not a single end-of-term submission.

Faculty Mentor
Mrs. M. Sindhu
Assistant Professor, Dept. of CSE

Head of Department
Dr. S. Pavithra
Head, Dept. of CSE

Date:

SIRAGU

Submitted for the PBL evaluation held on _____.

Faculty Mentor: _____ PBL In–charge: _____

DECLARATION

We declare that this Project-Based Learning report titled **"GroupDeal: Locality-Based Group Buying with Dynamic Pricing & Supplier Bidding"** reflects our own work carried out under the mentorship of **Mrs. M. Sindhu, Assistant Professor, Department of Computer Science and Engineering**, across the PBL review cycle. The report distinguishes the features implemented in the current Android prototype from those proposed for the complete system. All sources of information used have been duly acknowledged and cited.

Team Members:

ARUL MURUGAN M, 24CS0084 Signature: _____

RITVIK DODDAPANENI, 24CS0755 Signature: _____

Place: Chennai

Date:

SIRAGU

PBL COURSE AND TEAM DETAILS

Field	Details
Course	Mobile Application Development (CS4504) — Project-Based Learning (PBL)
Project Title	GroupDeal — Locality-Based Group Buying with Dynamic Pricing & Supplier Bidding
Faculty Mentor	**Mrs. M. Sindhu, Assistant Professor, Department of CSE**
Academic Year	2026–2027
Section / Team No.	_____
PBL Duration	10 weeks
Review Cycles Completed	Review 0 (Zeroth Review): 23-07-2026 Review 1: _____ Review 2: _____
Project Track / Domain	Mobile Application Development / Local Commerce (Group Buying)

Team Roles and Responsibilities

Name	Register No.	Role	Primary Responsibility
ARUL MURUGAN M	24CS0084	Application Developer & UI/UX Designer	Designed the Material 3 interface and navigation; developed the Login, Onboarding, Locality Picker, Discover, Group Detail, My Groups, Orders and Profile screens in Jetpack Compose; prepared UI validation, screenshots and the poster.
RITVIK DODDAPANENI	24CS0755	Logic, Data & Integration Developer	Designed the data models and GroupDealRepository state layer; implemented PricingEngine, LocalityService, group membership, demand requests and order logic; wrote unit tests and designed the proposed Firebase, Razorpay and FCM integration.

ACKNOWLEDGEMENT

We would like to thank **Mrs. M. Sindhu, Assistant Professor, Department of Computer Science and Engineering**, for mentoring this project across every review cycle of the PBL — from shaping the idea of locality-based group buying at the zeroth review to pushing us to separate what our prototype actually does from what the complete system should do. Her feedback after each review changed the direction of our work more than once, and the project is better for it.

We are also grateful to **Dr. S. Pavithra, M.E., Ph.D., Head of the Department**, for supporting the PBL structure itself, which gave us room to build GroupDeal iteratively instead of rushing a single final version, and to the faculty and staff of the Department of Computer Science and Engineering, Chennai Institute of Technology, for the facilities and guidance provided.

SIRAGU

ABSTRACT

Individual households usually buy groceries and daily essentials at retail prices, while bulk discounts are available mainly to businesses and high-volume buyers. Neighbours who need the same product place separate orders and cannot combine their demand. **GroupDeal** is an Android application that applies the idea of locality-based group buying to this problem. Residents of the same area (identified by pincode) join a shared buying group for a product. Each new member increases the group quantity, and a rule-based, tier-based dynamic pricing engine moves every member of the group to the lowest unit price that the current member count qualifies for. The application displays the current group price, the saving relative to the market price (MRP) and the number of additional members needed to unlock the next price tier. Residents can also request products that are not yet offered and upvote their neighbours' requests, making local demand visible before a deal exists.

The current prototype is a native Android application developed in Kotlin with Jetpack Compose and Material 3. It implements locality selection with pincode lookup, deal discovery with search and category filters, joining and leaving groups with automatic tier and price recalculation, demand requests with one-vote-per-user upvoting, an order record with a tracking timeline, and a member trust score mapped to reputation tiers. Application state is held in a single repository exposed through Kotlin StateFlow and SharedFlow, and the prototype operates on in-memory seed data. The report also presents the proposed complete system — a Firebase backend (Authentication, Cloud Firestore, Cloud Functions, Cloud Messaging), supplier reverse-auction bidding, Razorpay payments and map-based locality services — and clearly distinguishes implemented, partial and proposed features. The pricing logic was verified against the project's unit-test cases and the four seeded deals, for which the engine computes group prices between 31% and 46% below MRP.

Keywords: GroupDeal, Group Buying, Locality-Based Commerce, Tier-Based Dynamic Pricing, Reverse Auction, Android, Kotlin, Jetpack Compose

TABLE OF CONTENTS

TITLE	PAGE NO.
TEAM AND ROLE RESPONSIBILITIES	vi
ABSTRACT AND KEYWORDS	viii
LIST OF FIGURES	xiv
LIST OF TABLES	xv
LIST OF ABBREVIATIONS	xvi
CHAPTER 1 – INTRODUCTION	1
1.1 Project Overview	1
1.2 Background	1
1.3 Motivation	2
1.4 Problem Statement	2
1.5 Need for Locality-Based Group Buying	3
1.6 Objectives	3
1.7 Scope of the Project	4
1.8 Target Users	4
1.9 Project Domain	4
1.10 Key Features	4
1.11 Development Platform and Technologies	5
CHAPTER 2 – LITERATURE SURVEY	6
2.1 Group Buying and Price Discovery	6
2.2 Quantity Discounts and Dynamic Pricing	6
2.3 Reverse Auctions and Supplier Selection	7
2.4 Demand Aggregation and Social Commerce	7
2.5 Trust and Reputation in Online Purchasing	7
2.6 Review of Mobile Technologies	8
2.7 Comparison of Existing Approaches	8
2.8 Research Gap and Identified Limitations	8
CHAPTER 3 – EXISTING SYSTEM	10
3.1 Description of Existing System	10
3.2 Existing Purchasing Workflow	10
3.3 Technologies Used in Existing Systems	11
3.4 Drawbacks and Limitations	11
CHAPTER 4 – PROPOSED SYSTEM	13
4.1 Proposed Solution	13
4.2 Major Features	14

4.2.1 Locality Selection (Implemented) 14
4.2.2 Deal Discovery (Implemented) 14
4.2.3 Group Membership with Dynamic Pricing (Implemented) 14
4.2.4 Pricing Roadmap and Savings (Implemented) 14
4.2.5 Demand Requests and Upvotes (Implemented) 14
4.2.6 Checkout and Order Tracking (Partial) 14
4.2.7 Trust Score (Partial) 14
4.2.8 Supplier Reverse Auction (Proposed) 15
4.2.9 Price Comparison (Proposed) 15
4.3 User Roles 15
4.4 Functional Requirements 15
4.5 Non-Functional Requirements 16
4.6 System Scope 17
4.7 Advantages of the Proposed Solution 17

CHAPTER 5 – SYSTEM DESIGN 18
5.1 System Architecture 18
5.1.1 Physical Architecture 18
5.1.2 Logical Architecture 19
5.1.3 Application Architecture 19
5.1.4 Client-Side Architecture 20
5.1.5 Backend / Cloud Architecture 21
5.1.6 Data Architecture 21
5.1.7 External Service Integration 21
5.2 Use Case Diagram 22
5.3 Activity Diagram 24
5.4 Application Flowchart 25
5.5 Class Diagram 26
5.6 Sequence Diagram 27
5.7 Data Model / ER Representation 29
5.8 Data Flow Diagram 29
5.9 Pricing Engine Design 30
5.9.1 Inputs and Outputs 30
5.9.2 Algorithm 30
5.9.3 Worked Example 32
5.9.4 Edge Cases 33
5.10 Supplier Auction Design 33
5.11 Payment Flow Design 35
5.12 Notification Flow Design 35

CHAPTER 6 – DEVELOPMENT TOOLS AND TECHNOLOGIES 37
6.1 Development Environment 37
6.2 Android Studio 38
6.3 Programming Language – Kotlin 38
6.4 Jetpack Compose and Material 3 39
6.5 State Management – Coroutines, StateFlow and SharedFlow 39
6.6 Image Loading – Coil 39
6.7 Build System – Gradle and Version Catalog 39
6.8 Firebase / Backend Technology 40
6.9 Database Technology 40
6.10 APIs and Web Services 40
6.11 Architecture Libraries – MVVM and Hilt 40
6.12 Testing Tools 41
6.13 Version Control – Git 41
6.14 Technology Stack Summary 41

CHAPTER 7 – SOFTWARE IMPLEMENTATION 42
7.1 Project Setup 42
7.2 Project Structure 42
7.3 Application Architecture 43
7.4 UI Implementation 44
7.5 State Management 44
7.6 Repository Layer 44
7.7 Data Models 45
7.8 Locality Service 45
7.9 PricingEngine 46
7.10 Group Management 46
7.11 Demand Requests 47
7.12 Supplier Bidding 47
7.13 Orders 48
7.14 Payments 48
7.15 Notifications 48
7.16 Validation 48
7.17 Error Handling 49
7.18 Security Considerations 49

CHAPTER 8 – MOBILE APPLICATION IMPLEMENTATION 50
8.1 Application Setup and Configuration 50
8.2 Navigation Design 50
8.3 Authentication and Onboarding Screens 51

8.4 Locality Selection 52
8.5 Home / Discovery Screen 52
8.6 Group Detail and Pricing Screen 53
8.7 Demand Requests and Upvotes 53
8.8 My Groups Screen 53
8.9 Orders and Tracking Screen 54
8.10 Profile and Trust Screen 54
8.11 Input Forms and Validation 54
8.12 Module Integration 54

CHAPTER 9 – TESTING AND RESULTS 56
9.1 Testing Strategy 56
9.2 Unit Testing 56
9.3 Functional Testing 57
9.4 Integration Testing 58
9.5 UI Testing 58
9.6 Pricing Logic Testing 59
9.7 Group Membership Testing 59
9.8 Demand Request Testing 60
9.9 Supplier Bid Testing 60
9.10 Authentication Testing 61
9.11 Payment Testing 61
9.12 Notification Testing 61
9.13 Compatibility Testing 61
9.14 Results 61
9.15 Output Screenshots and Evidence 62

CHAPTER 10 – APPLICATIONS 64
10.1 Application Areas 64
10.2 Real-World Applications 64
10.3 Target User Groups 65
10.4 Potential Industry Applications 65
10.5 Adapting GroupDeal to New Domains 65

CHAPTER 11 – ADVANTAGES AND LIMITATIONS 66
11.1 Advantages 66
11.2 Limitations 66
11.3 Security Considerations 67
11.4 Performance Constraints 67

CHAPTER 12 – CONCLUSION AND FUTURE SCOPE 69
12.1 Conclusion 69
12.2 Project Outcomes 69
12.3 Technical Contribution 70
12.4 Future Enhancements 70
12.5 AI/ML Opportunities 70
12.6 Cloud Scalability 71
12.7 Supplier/Admin Web Dashboard 71
12.8 Logistics Integration 71
12.9 Advanced Analytics 71

CHAPTER 13 – REFERENCES 72
13.1 Books 72
13.2 Research Papers / Journals 72
13.3 Official Technical Documentation 73

CHAPTER 14 – APPENDIX 74
A.1 Important Source Code 74
A.2 PricingEngine Code 75
A.3 Data Models 76
A.4 Repository Structure 82
A.5 Database / Data Structure 82
A.6 Seed Data and Localities 83
A.7 Detailed Test Cases 84
A.8 Weekly PBL Progress Log 86
A.9 Team Responsibilities and Self / Peer Assessment 86
A.10 Additional Diagram — Proposed Firestore Security Rules 86
A.11 Project Repository Information 87

SIRAGU

LIST OF FIGURES

FIGURE NO.	FIGURE NAME	PAGE NO.
Figure 3.1	Existing Individual Purchasing Workflow	10
Figure 4.1	Locality-Based Group Buying Concept	13
Figure 5.1	Target System Architecture of GroupDeal (Proposed Complete System)	18
Figure 5.2	Physical Architecture / Deployment Design	19
Figure 5.3	Logical Architecture of the Application	19
Figure 5.4	Application Architecture of the Current Prototype	20
Figure 5.5	As-Built Prototype Architecture with Planned Integrations	21
Figure 5.6	Use Case Diagram of GroupDeal	23
Figure 5.7	Activity Diagram — Joining a Group Deal	25
Figure 5.8	Application Flowchart	26
Figure 5.9	Class Diagram of the Core Data Model	27
Figure 5.10	Sequence Diagram — Join Group (Implemented Prototype)	28
Figure 5.11	Sequence Diagram — Join and Pay (Proposed Cloud System)	28
Figure 5.12	Proposed Data Model (ER Representation of Firestore Collections)	29
Figure 5.13	Data Flow Diagram (Level 1)	30
Figure 5.14	Flowchart of the Tier-Based Pricing Engine	32
Figure 5.15	Tier Progression of the Seeded Ghee Deal (Price versus Group Size)	33
Figure 5.16	Reverse Auction Workflow (Proposed)	34
Figure 5.17	Group Deal State Machine (GroupState Enumeration)	35
Figure 5.18	Payment Flow (Proposed Razorpay Integration)	35
Figure 5.19	Notification Flow: Implemented In-App Events and Proposed FCM Push	36
Figure 6.1	Technology Stack of GroupDeal by Layer and Status	37
Figure 7.1	Project Structure / Application Package Structure	42
Figure 7.2	Unidirectional Data Flow in the Prototype	43
Figure 7.3	Demand Request Lifecycle	47
Figure 8.1	Navigation Map of the GroupDeal Application	50
Figure 8.2	Layout Schematics of Login, Onboarding, Discover and Group Detail Screens (reconstructed from source, not device screenshots)	52
Figure 8.3	Layout Schematics of Demands, My Groups, Orders and Profile Screens (reconstructed from source, not device screenshots)	53
Figure 9.1	Application Testing Workflow	56
Figure 9.2	Market Price versus Engine-Computed Group Price for the Seeded Deals	62

LIST OF TABLES

TABLE NO.	TABLE NAME	PAGE NO.
Table 1.1	Key Features of GroupDeal and Implementation Status	4
Table 1.2	Development Platform of the GroupDeal Prototype	5
Table 2.1	Comparison of Existing Purchasing Approaches with GroupDeal	8
Table 3.1	Technologies Used in Existing Purchasing Systems	11
Table 3.2	Existing System versus Proposed GroupDeal System	12
Table 4.1	Implementation Status Labels Used in this Report	13
Table 4.2	User Roles and Permissions	15
Table 4.3	Functional Requirements of the Proposed Application	15
Table 4.4	Non-Functional Requirements	16
Table 5.1	External Services and Integration Status	21
Table 5.2	Pricing Engine Inputs and Outputs	30
Table 5.3	Pricing Engine Edge Cases	33
Table 6.1	Hardware Requirements for Development and Use	37
Table 6.2	Software Requirements and Versions	38
Table 6.3	Software Development Tools and Technologies with Verified Status	41
Table 7.1	Source Files and Responsibilities	43
Table 8.1	Application Screens and Their Functionalities	51
Table 8.2	Input Forms and Validation Rules	54
Table 9.1	Unit Test Cases in the Project Source	56
Table 9.2	Functional Test Cases	57
Table 9.3	Integration Test Cases	58
Table 9.4	Planned Compose UI Test Cases	58
Table 9.5	Pricing Engine Boundary Tests (Ghee Deal)	59
Table 9.6	Tier Validation Tests (validateTiers)	59
Table 9.7	Group Membership and State Tests (Ghee Deal, Target 30)	59
Table 9.8	Demand Request Test Cases	60
Table 9.9	Supplier Bid Test Cases	60
Table 9.10	Pricing Engine Output for the Four Seeded Deals	62
Table 9.11	Summary of Verified Prototype Capabilities	62
Table 10.1	Target User Groups and Benefits	65
Table 12.1	Future Enhancements and Priority	70
Table 14.1	Seeded Group Deals	83
Table 14.2	Seeded Demand Requests	83
Table 14.3	Seeded Localities in LocalityService	83
Table 14.4	Weekly PBL Progress Log	86
Table 14.5	Self and Peer Assessment	86
Table 14.6	Project Build and Run Information	87

LIST OF ABBREVIATIONS

Abbreviation	Expansion
API	Application Programming Interface
APK	Android Package Kit
BOM	Bill of Materials
CRUD	Create, Read, Update, Delete
DFD	Data Flow Diagram
DI	Dependency Injection
ER	Entity Relationship
FCM	Firebase Cloud Messaging
GPS	Global Positioning System
HMAC	Hash-based Message Authentication Code
HTTPS	Hypertext Transfer Protocol Secure
IDE	Integrated Development Environment
JVM	Java Virtual Machine
KSP	Kotlin Symbol Processing
MAD	Mobile Application Development
MRP	Maximum Retail Price
MVVM	Model–View–ViewModel
OTP	One-Time Password
PBL	Project-Based Learning
RBAC	Role-Based Access Control
REST	Representational State Transfer
SDK	Software Development Kit
UDF	Unidirectional Data Flow
UI	User Interface
UML	Unified Modeling Language
UPI	Unified Payments Interface
UX	User Experience

CHAPTER 1
INTRODUCTION

1.1 Project Overview

GroupDeal is an Android mobile application for **locality-based group buying**. It allows residents of the same neighbourhood, identified by pincode or area name, to combine their individual purchases of a product into one shared buying group. The central idea is simple: when more members join a group, the group's total quantity increases, a higher quantity-discount tier is reached, and the unit price falls for **every** member of the group — not only for the member who joined last.

The project is summarised by its tagline, **"Buy Together, Save More"**. A resident opens the application, selects a locality, browses the group deals available there and joins a deal. The deal screen shows the current group price, the saving compared with the market price (MRP), the complete pricing roadmap and the number of additional neighbours required to unlock the next, cheaper tier. Residents who want a product that is not yet offered can publish a demand request, and neighbours can upvote it, so that local demand becomes visible before a supplier deal exists.

The work in this report is organised around two views of the same system. The **current prototype** is a working native Android application built in Kotlin with Jetpack Compose; its behaviour is documented directly from the project source code. The **proposed complete system** adds a Firebase cloud backend, supplier reverse-auction bidding, online payment through Razorpay, push notifications and map-based locality services. Throughout the report every feature is labelled as **Implemented**, **Partial**, **Proposed** or **Future** so that the reader can see exactly what the prototype does today and what the architecture is designed to support next.

1.2 Background

Group buying is a purchasing model in which a seller offers a lower price in return for a guaranteed minimum volume, and the price depends on how many buyers commit to the offer. Online group-buying mechanisms have been studied as a form of price discovery since the early 2000s: Anand and Aron compared group-buying schemes with fixed posted prices and showed how volume-dependent pricing can serve buyers and sellers under demand uncertainty [5], while Kauffman and Wang analysed how buyers join group-buying offers over time as the price approaches the next discount level [6]. Quantity discounts — lower unit prices for larger orders — are an established pricing tool in marketing and operations [7].

In India, most households still buy groceries and essentials individually, either from neighbourhood shops or from e-commerce platforms that price each order independently. At the same time, residents of apartment communities and localities often need the same staples — ghee, cooking oil, rice, fruits and spices — at the same time. This combination of repeated, similar local demand and fragmented purchasing creates an opportunity for a mobile application that aggregates demand at the locality level and passes bulk pricing back to individual buyers.

1.3 Motivation

The motivation for GroupDeal came from three observations made during the requirement-analysis phase of the PBL:

- **Bulk prices are invisible to individuals.** A single buyer rarely knows what price a supplier would offer for 15 or 30 units of the same product, and has no way to reach that volume alone.
- **Neighbours cannot easily coordinate.** Messaging groups can collect interest, but they do not calculate prices, track members, or tell participants how close the group is to a better price.
- **Local demand is unstructured.** When many residents want an item that is not available at a good price, there is no simple mechanism to record and measure that demand so that a supplier can respond to it.

A mobile application is a natural fit for this problem because group membership, prices and progress change continuously and must be visible to every member at the same time. The project also provided an opportunity to apply core Mobile Application Development concepts — declarative user interfaces, state management, navigation, input validation, data modelling and testing — to a problem with a clear real-world benefit.

1.4 Problem Statement

Individual buyers pay retail prices for everyday products because bulk discounts are accessible only to high-volume purchasers. Neighbours with the same requirements order separately and cannot pool their demand, and existing platforms price each order independently of local demand. Buyers also cannot see how many more participants would unlock a lower price, and there is no simple way to signal demand for a product that is not yet offered in their locality.

The problem addressed by this project is therefore: **to design and develop a mobile application that lets residents of a locality form buying groups, automatically computes a tier-based group price from the current member count, shows each member the savings and the progress towards the next tier, and captures unmet local demand**

through requests and upvotes — with a system architecture that can be extended to supplier bidding, online payment and real-time notifications.

1.5 Need for Locality-Based Group Buying

Group buying can be organised city-wide or nationally, but the locality is the most practical unit for everyday products for several reasons:

- **Shared delivery point.** Members of one locality can be served by a single batch delivery to a common distribution point, which reduces the logistics cost that makes small individual orders expensive.
- **Similar demand.** Households in the same neighbourhood tend to buy similar staples in similar seasons, so reaching a volume threshold is realistic.
- **Social trust.** Neighbours are more willing to commit to a shared purchase with people they may already know, and a locality-scoped reputation score is more meaningful than an anonymous one [15].
- **Supplier relevance.** Regional suppliers, farm cooperatives and wholesalers serve specific delivery areas; a locality-scoped deal matches the geography of supply.

1.6 Objectives

The objectives of the GroupDeal project are listed below. Each objective is traced to its implementation status in Chapter 4 and verified in Chapter 9.

1. To enable locality-based group buying in which users select their delivery locality by pincode or area name.
2. To implement a rule-based, tier-based dynamic pricing engine that lowers the unit price for all members as the member count crosses defined thresholds.
3. To show every member the current group price, the saving relative to MRP and the number of members needed to unlock the next tier.
4. To capture unmet local demand through product requests and neighbour upvotes.
5. To manage group membership, order placement and order tracking in a single mobile interface.
6. To build member trust through a reputation score mapped to trust tiers.
7. To design a scalable architecture for supplier reverse-auction bidding, secure online payments, real-time synchronisation and push notifications.
8. To demonstrate the practical application of modern Android development practices — Kotlin, Jetpack Compose, Material 3 and reactive state with StateFlow.

1.7 Scope of the Project

The scope of the **current prototype** covers the consumer-side experience of group buying on a single Android device: sign-in and onboarding flows, locality selection, deal discovery, group membership with dynamic pricing, demand requests and upvotes, order records with a tracking timeline, and a profile with a trust score. All data is supplied by seed data held in memory, which makes the prototype fully demonstrable without network services.

The scope of the **proposed system** extends this to a multi-user, cloud-backed service: authenticated users, shared real-time deal state in Cloud Firestore, server-side price computation, supplier and administrator roles, reverse-auction bidding, Razorpay payments and push notifications. The following are outside the scope of this PBL: physical logistics, supplier onboarding contracts, taxation and invoicing, and production deployment on the Google Play Store.

1.8 Target Users

- **Residents and households** in apartment complexes and residential localities who buy groceries and essentials regularly.
- **Resident welfare associations and community groups** that already coordinate purchases informally.
- **Students and working professionals** sharing accommodation, who benefit from pooled buying of staples.
- **Suppliers** — farm cooperatives, wholesalers and regional brands — who can serve aggregated locality demand (proposed role).
- **Platform administrators** who verify suppliers and moderate deals (proposed role).

1.9 Project Domain

GroupDeal belongs to the domain of **mobile commerce (m-commerce)** and, more specifically, **social and local commerce**, in which purchasing decisions are influenced by a community of users [13]. Technically, the project lies in native Android application development with a reactive state model; its pricing component draws on the operations and marketing literature on quantity discounts and dynamic pricing [7], [8], and its proposed supplier-selection mechanism draws on reverse auctions [9].

1.10 Key Features

Table 1.1: Key Features of GroupDeal and Implementation Status

Feature	Description	Status
Locality selection	Pincode lookup and area search across 15 seeded localities in 8 cities; fallback for any valid 6-digit pincode	Implemented

Feature	Description	Status
Deal discovery	Deal feed with text search and category chips; product images loaded from URLs	Implemented
Join / leave group	Membership with duplicate prevention; member count, tier and price recalculated on every change	Implemented
Tier-based dynamic pricing	PricingEngine selects the highest threshold reached and returns its unit price	Implemented
Savings & next-tier display	Saving % vs MRP, locality savings and members needed for the next tier	Implemented
Demand requests & upvotes	Request form and one-vote-per-user upvoting with progress to threshold	Implemented
Checkout & order tracking	Order record with savings and a four-step tracking timeline	Partial
Trust score & tiers	Score mapped to Bronze, Silver, Gold and Platinum tiers	Partial
Supplier reverse auction	Bid data model and seeded winning bids; bid submission and selection	Proposed
Online payment	Razorpay checkout with server-side signature verification	Proposed
Cloud backend & push alerts	Firebase Authentication, Firestore, Cloud Functions and FCM	Proposed
Price comparison	Comparison of group price with marketplace prices	Proposed

1.11 Development Platform and Technologies

Table Table 1.2 summarises the development platform. Chapter 6 describes each technology in detail, including technologies that are part of the proposed architecture but not yet active in the prototype.

Table 1.2: Development Platform of the GroupDeal Prototype

Component	Technology (verified from project source)
Platform	Native Android (minimum SDK 24, target and compile SDK 36)
Programming language	Kotlin 2.2.10
User interface	Jetpack Compose (Compose BOM 2024.09.00) with Material 3 and Material Icons
State management	Kotlin Coroutines 1.10.2, StateFlow and SharedFlow
Image loading	Coil 2.7 (AsyncImage for product image URLs)
Data layer	GroupDealRepository singleton with in-memory seed data
Build system	Gradle with Kotlin DSL, version catalog and Android Gradle Plugin 9.1.1
Testing	JUnit 4 and Robolectric (JVM unit tests)
Proposed backend	Firebase Authentication, Cloud Firestore, Cloud Functions, Cloud Storage, FCM
Proposed services	Razorpay payment gateway, Google Maps / Places

CHAPTER 2
LITERATURE SURVEY

This chapter reviews the research and practice that inform GroupDeal: group buying as a pricing mechanism, quantity discounts and dynamic pricing, reverse auctions for supplier selection, demand aggregation and social commerce, trust and reputation systems, and the mobile technologies used to build such applications. The chapter ends with a comparison of existing approaches and the gap that GroupDeal addresses.

2.1 Group Buying and Price Discovery

Anand and Aron studied group-buying schemes on the web as a price-discovery mechanism and compared them with conventional posted pricing [5]. In a group-buying scheme the seller announces a price schedule in which the unit price falls as the number of committed buyers rises. Their analysis showed that such schedules are particularly useful when demand is uncertain, because the seller effectively lets the market reveal the volume before committing to a price. This is the same structure that GroupDeal uses: each deal carries a schedule of member-count thresholds and prices, and the final price is determined by how many residents join.

Kauffman and Wang examined empirical data from group-buying websites and found that the number of new buyers joining an offer increases as the offer approaches the next price drop — a “price-drop effect” — and that buyers are influenced by the number of people who have already joined [6]. This result directly motivates two design choices in GroupDeal: displaying the number of members still needed for the next tier on every deal, and displaying the live member count (for example, 18/30 joined).

Chen, Chen and Song compared group-buying auctions with fixed pricing and identified the conditions under which the group-buying mechanism benefits sellers, notably when economies of scale in supply are significant [10]. Locality-based batch delivery of staples, as proposed in GroupDeal, is one such case. Jing and Xie further showed that group buying can be viewed as a mechanism that uses social interaction between consumers — better-informed buyers bringing in less-informed ones — to expand sales [12], which supports GroupDeal’s emphasis on neighbours inviting neighbours.

2.2 Quantity Discounts and Dynamic Pricing

Dolan’s review of quantity discounts describes how sellers use volume-based price schedules to share cost savings with buyers and to encourage larger orders [7]. The tier structure in GroupDeal — for example, a base price of ₹1,250 for fewer than 5 members, ₹1,099 from 5 members, ₹949 from 15 members and ₹799 from 30 members — is an **all-units quantity**

discount applied to the group total, in which every unit is charged the price of the tier reached by the group.

Dynamic pricing more broadly refers to prices that change over time in response to demand, inventory or other conditions. Elmaghraby and Keskinocak surveyed dynamic pricing research and practice [8], and Gallego and van Ryzin analysed optimal dynamic pricing under stochastic demand [11]; Talluri and van Ryzin provide a comprehensive treatment of revenue management [4]. These optimisation-based approaches differ from GroupDeal's approach in an important way: GroupDeal does **not** predict demand or optimise prices with a statistical model. Its prices change dynamically, but only according to a **deterministic, rule-based tier schedule** fixed in advance for each deal. This makes the price transparent and explainable to every member, which is essential for trust in a community purchase.

2.3 Reverse Auctions and Supplier Selection

In a reverse auction, several suppliers compete by offering progressively lower prices to a single buyer. Jap studied the introduction of online reverse auctions in industrial procurement and their effect on buyer–supplier relationships [9]. Her work shows that reverse auctions can lower prices but may affect supplier trust if the process is perceived as unfair — an important consideration for GroupDeal's proposed design, in which the aggregated locality demand acts as the "buyer" and verified suppliers bid to fulfil it. The proposed design therefore ranks bids not only on price but also on supplier score and delivery time (Section 5.10).

2.4 Demand Aggregation and Social Commerce

Social commerce refers to commercial activity mediated by social interaction and user contributions. Liang and Turban proposed a research framework for social commerce covering the people, management, technology and information involved [13]. GroupDeal's demand requests are a simple form of social commerce: a resident proposes a product with a target price, neighbours signal interest by upvoting, and the community's collective interest becomes a measurable demand signal. Unlike generic social feeds, the signal is scoped to one locality and is tied to a numeric threshold (for example, 17 of 20 neighbours).

2.5 Trust and Reputation in Online Purchasing

Kim, Ferrin and Rao showed that consumer trust and perceived risk strongly influence online purchase decisions [14]. Resnick and colleagues described reputation systems as a mechanism for building trust among strangers in online interactions by recording and displaying past behaviour [15]. Group buying depends on trust in two directions: members must trust that others will pay, and suppliers must trust that the group will complete the purchase. GroupDeal models this with a trust profile containing a score, completed

purchases, groups initiated, referrals and prompt-payment rate, and maps the score to trust tiers.

2.6 Review of Mobile Technologies

Modern Android applications are built with Kotlin and Jetpack Compose, Google's declarative UI toolkit, following the recommended app architecture of a UI layer driven by observable state and a data layer that owns the application data [16], [17]. Kotlin coroutines and flows provide asynchronous, lifecycle-aware data streams, and StateFlow in particular represents a state value that UI components can observe [18], [19]. For multi-user applications, Firebase provides managed authentication, a real-time document database (Cloud Firestore) with transactional updates, server-side Cloud Functions and push messaging (FCM) [20]–[22], [29]. Payment gateways such as Razorpay provide checkout SDKs with server-side order creation and signature verification [23], and Google Maps Platform provides place and location services [24].

2.7 Comparison of Existing Approaches

Table 2.1: Comparison of Existing Purchasing Approaches with GroupDeal

Feature	Local retail shop	E-commerce app	Messaging-group pooling	GroupDeal
Price depends on group volume	No	No (per order)	Manual negotiation	Yes — automatic tiers
Locality-scoped buying	Yes (single shop)	No	Yes	Yes — pincode / area
Live member count and progress	No	No	Manual counting	Yes
Members needed for next price	No	No	No	Yes
Capture of unmet demand	Informal	Wishlist (individual)	Informal	Requests + upvotes
Supplier competition	No	Marketplace sellers	No	Reverse auction (proposed)
Transparent pricing rule	Varies	Varies	Varies	Published tier roadmap
Order tracking	No	Yes	No	Timeline (partial)
Member trust signal	Personal familiarity	Seller ratings	Personal familiarity	Trust score and tiers

2.8 Research Gap and Identified Limitations

The literature establishes that volume-dependent pricing, visible progress towards the next price, supplier competition and reputation all improve group-buying outcomes [5], [6], [9],

[15]. However, the approaches available to an ordinary household today do not combine these elements at the locality level. Retail and e-commerce channels price each order independently; informal pooling through messaging groups has no price computation, no progress tracking and no record of demand. The gap addressed by GroupDeal is therefore a **single mobile application that brings together locality-scoped group formation, transparent tier-based pricing, visible progress to the next tier and structured local demand capture**, designed so that supplier bidding, payments and real-time synchronisation can be added on the same architecture.

SIRAGU

CHAPTER 3

EXISTING SYSTEM

3.1 Description of Existing System

In the existing system, households buy everyday products individually. A buyer who needs ghee, cooking oil or rice either visits a local shop or orders from an e-commerce platform. Each purchase is priced independently of what the buyer's neighbours are buying, and any discount is limited to a coupon or a promotional offer that is the same for every buyer regardless of local demand.

Where residents do try to buy together, the process is informal. An organiser collects interest through a messaging group, negotiates with a shop or wholesaler by phone, keeps a manual list of participants, collects payments individually and coordinates delivery. The final price depends on the organiser's negotiation rather than on a transparent rule, and participants cannot see how close the group is to a better price.

3.2 Existing Purchasing Workflow

The general workflow of individual purchasing is shown in Figure 3.1. The same sequence is repeated independently by every household that needs the same product.

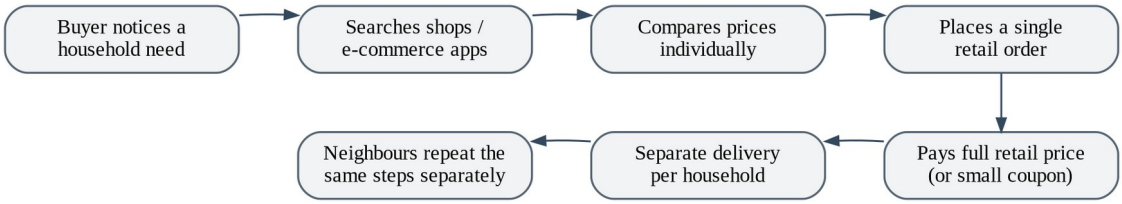


Figure 3.1: Existing Individual Purchasing Workflow

Buyer notices a household need
↓
Searches shops or e-commerce applications
↓
Compares prices individually
↓
Places a single retail order
↓
Pays the retail price
↓
Receives a separate delivery

The workflow shows that no step aggregates demand across households. Even when several neighbours buy the same product in the same week, each pays the single-unit price and each order is delivered separately.

3.3 Technologies Used in Existing Systems

Existing purchasing channels rely on the technologies listed in Table 3.1. These technologies support individual ordering well, but none of them is organised around a locality-level shared purchase.

Table 3.1: Technologies Used in Existing Purchasing Systems

Technology / Component	Usage in Existing Systems
E-commerce web and mobile applications	Product listing, individual cart and checkout, delivery tracking per order.
Cloud servers and databases	Store product catalogues, customer accounts and individual orders.
Payment gateways	Process card, UPI and netbanking payments for single orders.
Messaging applications	Informal coordination of pooled purchases through group chats.
Spreadsheets and manual lists	Tracking of participants and payments in informal group purchases.
Location services	Delivery address capture and serviceability checks by pincode.
Push notifications	Order status updates and promotional messages to individual customers.

3.4 Drawbacks and Limitations

The existing system has the following drawbacks:

1. **Fragmented demand.** Identical local demand is split into many small orders, so no single buyer reaches a bulk-price volume.
2. **Individual retail pricing.** Buyers pay the single-unit price; bulk discounts are available mainly to businesses and high-volume purchasers.
3. **No locality-level pooling.** Platforms do not group buyers by locality, so neighbours cannot combine orders for a shared delivery.
4. **Opaque price–demand relationship.** Buyers cannot see how many more participants would unlock a lower price.
5. **Limited supplier competition.** In informal pooling, the organiser negotiates with one supplier; there is no structured way for several suppliers to compete for aggregated demand.
6. **Manual coordination.** Participant lists, payments and delivery are tracked by hand, which does not scale and is error-prone.
7. **Unrecorded unmet demand.** Requests for products that are not available are expressed informally and are not measured.
8. **Low transparency and trust.** Participants depend on the organiser’s word; there is no shared record of membership, price or payment behaviour.

Table 3.2 compares the existing approach with the proposed GroupDeal system along these dimensions.

Table 3.2: Existing System versus Proposed GroupDeal System

Aspect	Existing System	Proposed GroupDeal System
Demand	Fragmented individual orders	Aggregated into locality buying groups
Pricing	Single-unit retail price	Tier price for all members, based on member count
Transparency	Negotiated, often opaque	Published tier roadmap and live progress
Coordination	Manual lists and chats	Automatic membership tracking in the app
Unmet demand	Informal requests	Requests with upvote count and threshold
Supplier selection	Single supplier, negotiated	Reverse auction among verified suppliers (proposed)
Trust	Personal familiarity only	Trust score and tiers for members
Delivery	Separate delivery per household	Batch delivery to a locality point (proposed)

SIRAGU

CHAPTER 4

PROPOSED SYSTEM

4.1 Proposed Solution

The proposed system, **GroupDeal**, is a mobile platform that converts fragmented household demand into locality-level group purchases. Instead of buying alone, a resident joins a **group deal** published for the resident's locality. Each deal has a market reference price (MRP), a base price and a schedule of **pricing tiers**, each defined by a minimum number of members and the unit price that applies once that number is reached. As residents join, the member count rises; when it crosses a tier threshold, the unit price for the whole group drops to that tier's price. The concept is illustrated in Figure 4.1.

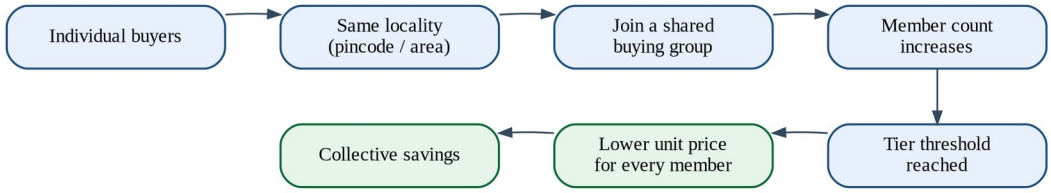


Figure 4.1: Locality-Based Group Buying Concept

When a group reaches its target member count, the proposed system opens a **reverse auction** in which verified suppliers bid to fulfil the aggregated order; the lowest valid bid wins, members pay through a secure payment gateway, and the order is delivered as a batch to a locality distribution point. For products that are not yet available, residents publish **demand requests** that neighbours upvote; a request that reaches its threshold becomes a candidate for a new group deal.

Implementation Status Legend

Because GroupDeal is presented as both a working prototype and a complete system design, the following status labels are used consistently in this report:

Table 4.1: Implementation Status Labels Used in this Report

Label	Meaning
Implemented	Present and working in the current Android prototype; verified from the project source code.
Partial	Present in the prototype in a simplified or simulated form (for example, a local record instead of a server transaction).
Proposed	Part of the GroupDeal system architecture and design, but not yet implemented in the prototype.
Future	Enhancement planned beyond the scope of the complete system (Chapter 12).

4.2 Major Features

4.2.1 Locality Selection (Implemented)

Users select a delivery locality from a searchable list or by entering a six-digit pincode. The prototype includes 15 seeded localities across 8 cities (Bengaluru, New Delhi, Gurugram, Noida, Mumbai, Hyderabad, Pune and Chennai). Any other valid six-digit pincode is accepted through a fallback that infers the city from the first two digits.

4.2.2 Deal Discovery (Implemented)

The Discover screen lists group deals as cards showing the product image, category and brand, current group price, MRP, member progress (for example, 18/30 joined) and the total amount saved by the locality. Users can search by text and filter by category.

4.2.3 Group Membership with Dynamic Pricing (Implemented)

Joining a group increments the member count; leaving decrements it. On every change the PricingEngine recomputes the active tier and unit price, and the group state is updated. Duplicate joins by the same user are prevented.

4.2.4 Pricing Roadmap and Savings (Implemented)

The Group Detail screen presents every tier as a milestone (for example, “15+ Buyers Milestone — 32% Discount — Save ₹451”), the current group price and the members needed to reach the next tier.

4.2.5 Demand Requests and Upvotes (Implemented)

Users publish a request with a product name, category, target group price and retail reference price. Neighbours upvote with an “I Want This (+1)” action; each user can vote once per request, and a progress bar shows the progress towards the vote threshold.

4.2.6 Checkout and Order Tracking (Partial)

The prototype records an order with quantity, unit price, total, amount saved and a four-step tracking timeline. Payment is not processed; the proposed system integrates Razorpay (Section 5.11).

4.2.7 Trust Score (Partial)

Each user has a trust profile with a score out of 1000, mapped to Bronze, Silver, Gold or Platinum tiers. In the prototype the score is set at sign-in or onboarding; the proposed system updates it from payment and participation history.

4.2.8 Supplier Reverse Auction (Proposed)

The data model already contains supplier bids and the group states BIDDING and SUPPLIER_SELECTED, and two seeded deals display a winning supplier. Bid submission, validation and automatic selection are part of the proposed design (Section 5.10).

4.2.9 Price Comparison (Proposed)

The prototype compares the group price against a stored MRP. Comparison with live marketplace prices is proposed; it is a separate feature from group dynamic pricing and depends on the availability of affiliate or partner price APIs.

4.3 User Roles

Table 4.2: User Roles and Permissions

Role	Permissions in the Proposed System	Prototype Status
Customer / Buyer	Select locality; browse, search and filter deals; join and leave groups; view tiers and savings; request and upvote products; check out; track orders; view trust score	Implemented (payment partial)
Supplier	List deals for serviceable localities; submit bids in reverse auctions; view won orders; update fulfilment status	Proposed (role selectable at onboarding; supplier functions not built)
Administrator	Verify suppliers; moderate deals and requests; manage localities; resolve disputes	Proposed (role switch available as a demonstration only)

4.4 Functional Requirements

The functional requirements of GroupDeal are listed in Table 4.3, following the requirements-engineering practice of separating functional and non-functional requirements [1], [2]. Each requirement is labelled with its status in the current prototype.

Table 4.3: Functional Requirements of the Proposed Application

ID	Functional Requirement	Description	Status
FR1	Sign in / onboarding	The system shall allow a user to sign in and, for a new user, capture name, phone, locality and account type.	Partial
FR2	Locality selection	The system shall allow the user to select a locality by search or six-digit pincode.	Implemented
FR3	Deal listing	The system shall list group deals with price, MRP, progress and savings.	Implemented
FR4	Search and filter	The system shall filter deals by text and by category.	Implemented
FR5	Join group	The system shall add the user to a group once and update the member count.	Implemented

ID	Functional Requirement	Description	Status
FR6	Leave group	The system shall remove the user from a group and recompute the price.	Implemented
FR7	Tier pricing	The system shall compute the unit price from the highest tier whose threshold is reached, otherwise the base price.	Implemented
FR8	Next-tier progress	The system shall show the members needed to reach the next tier.	Implemented
FR9	Demand request	The system shall let a user publish a product request for the current locality.	Implemented
FR10	Upvote	The system shall accept at most one upvote per user per request.	Implemented
FR11	Order placement	The system shall create an order with amount, savings and tracking steps.	Partial
FR12	Trust score	The system shall display the user’s trust score and tier.	Partial
FR13	Supplier bidding	The system shall accept, validate and rank supplier bids and select the winner.	Proposed
FR14	Online payment	The system shall collect payment through a gateway and verify it server-side.	Proposed
FR15	Push notifications	The system shall notify members of price drops, auction results and order status.	Proposed
FR16	Request-to-deal conversion	The system shall convert a request that reaches its threshold into a group deal.	Proposed

4.5 Non-Functional Requirements

Table 4.4: Non-Functional Requirements

ID	Requirement	Description
NFR1	Usability	Core actions (join, upvote, checkout) shall be reachable within two taps from the main tabs; prices and progress shall be visible without scrolling on the deal card.
NFR2	Consistency	All screens shall show the same member count and price for a deal at any moment (single source of truth).
NFR3	Correctness	Price computation shall be deterministic and reproducible from the tier schedule and member count.
NFR4	Responsiveness	The UI shall update immediately after a state change without manual refresh.
NFR5	Security	Authentication, price computation and payment verification shall be performed server-side in the complete system; secrets shall never be stored in the app.
NFR6	Reliability	Concurrent joins by many members shall not corrupt the member count (transactional updates in the complete system).
NFR7	Scalability	The backend shall scale with the number of localities, deals and members without redesign.

ID	Requirement	Description
NFR8	Compatibility	The app shall run on Android 7.0 (API 24) and above.
NFR9	Maintainability	UI, state, domain logic and data shall be separated into layers.

4.6 System Scope

The complete GroupDeal system consists of three parts: the **Android application** used by customers (and, in the proposed system, by suppliers and administrators); a **cloud backend** that stores shared state, computes prices authoritatively, runs auctions and sends notifications; and **external services** for payment, location and product images. The prototype implements the Android application with an in-process repository standing in for the backend, which allows the complete user flow to be demonstrated and tested on a single device.

4.7 Advantages of the Proposed Solution

1. **Access to bulk pricing for individuals.** Every member receives the tier price reached by the group.
2. **Transparent pricing rule.** The full tier roadmap is shown before joining, and the price follows a published rule.
3. **Visible progress.** Members see exactly how many neighbours are needed for the next price drop, which encourages sharing.
4. **Structured local demand.** Requests and upvotes turn informal wishes into measurable demand.
5. **Supplier competition (proposed).** Reverse auctions let suppliers compete for proven demand.
6. **Community trust.** Trust tiers reward reliable members and help others decide whom to buy with.
7. **Efficient delivery (proposed).** Batch delivery to a locality point reduces per-order logistics cost.

CHAPTER 5

SYSTEM DESIGN

5.1 System Architecture

The system architecture of GroupDeal describes how users, the Android application, the cloud backend and external services interact. Figure 5.1 shows the **target architecture** of the complete system. The Android application is organised into a UI layer built with Jetpack Compose, a state layer based on StateFlow, a domain layer containing the pricing and locality logic, and a data layer. In the complete system the data layer synchronises with a Firebase backend, where Cloud Functions compute prices authoritatively, run auctions and call external services such as Razorpay.

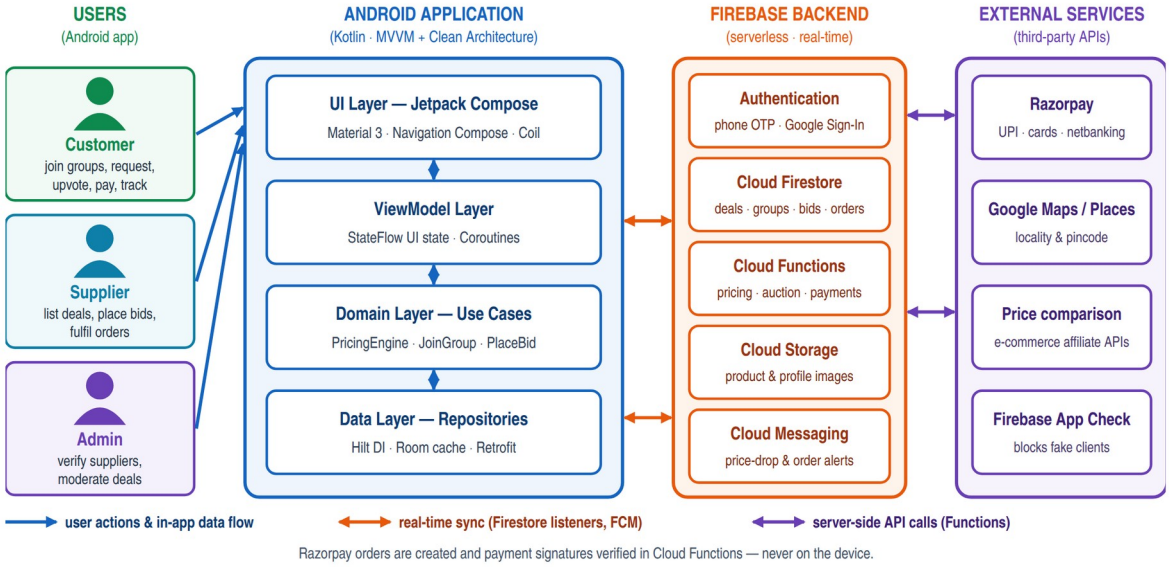


Figure 5.1: Target System Architecture of GroupDeal (Proposed Complete System)

5.1.1 Physical Architecture

The physical architecture (Figure 5.2) identifies the devices and networks involved. Users run GroupDeal on Android smartphones with API level 24 or above. The prototype needs network access only to load product images over HTTPS. In the complete system the device communicates over the internet with a Firebase project hosted on Google Cloud, and Cloud Functions communicate with Razorpay and Google Maps Platform.

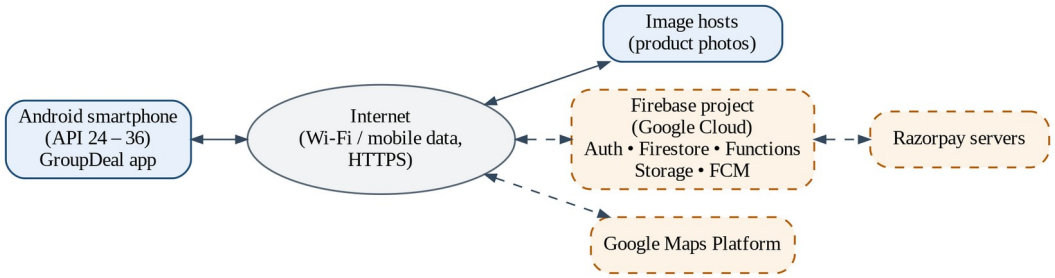


Figure 5.2: Physical Architecture / Deployment Design

5.1.2 Logical Architecture

The logical architecture (Figure 5.3) follows the unidirectional data flow recommended for Android applications [16]: user events flow down from the screens to the state holder, the domain rules compute the new values, and the new immutable state flows back up to the screens, which recompose automatically.

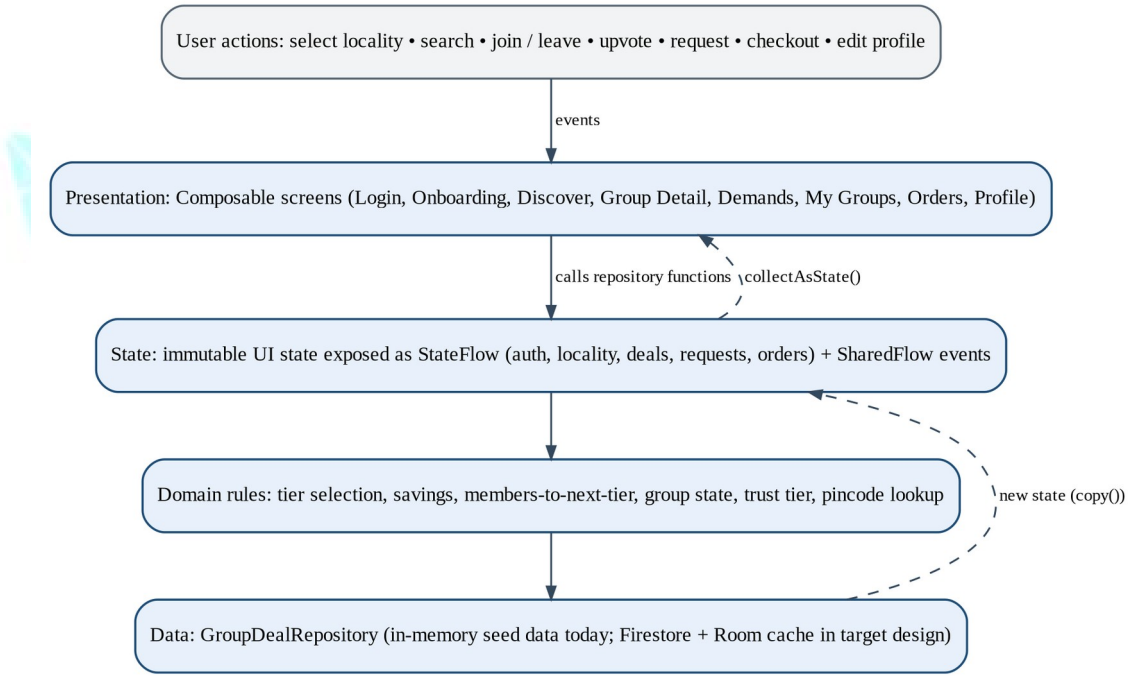


Figure 5.3: Logical Architecture of the Application

5.1.3 Application Architecture

Figure 5.4 shows the application architecture of the current prototype as implemented in the source code. `MainActivity` sets the Compose content, and `AppNavigation` selects the top-level screen from the authentication state (Loading, Unauthenticated, OnboardingRequired or Authenticated). The authenticated shell provides five tabs — Deals, Demands, My Groups, Orders and Profile — with the Group Detail screen shown as an overlay. All screens observe the single `GroupDealRepository` object.

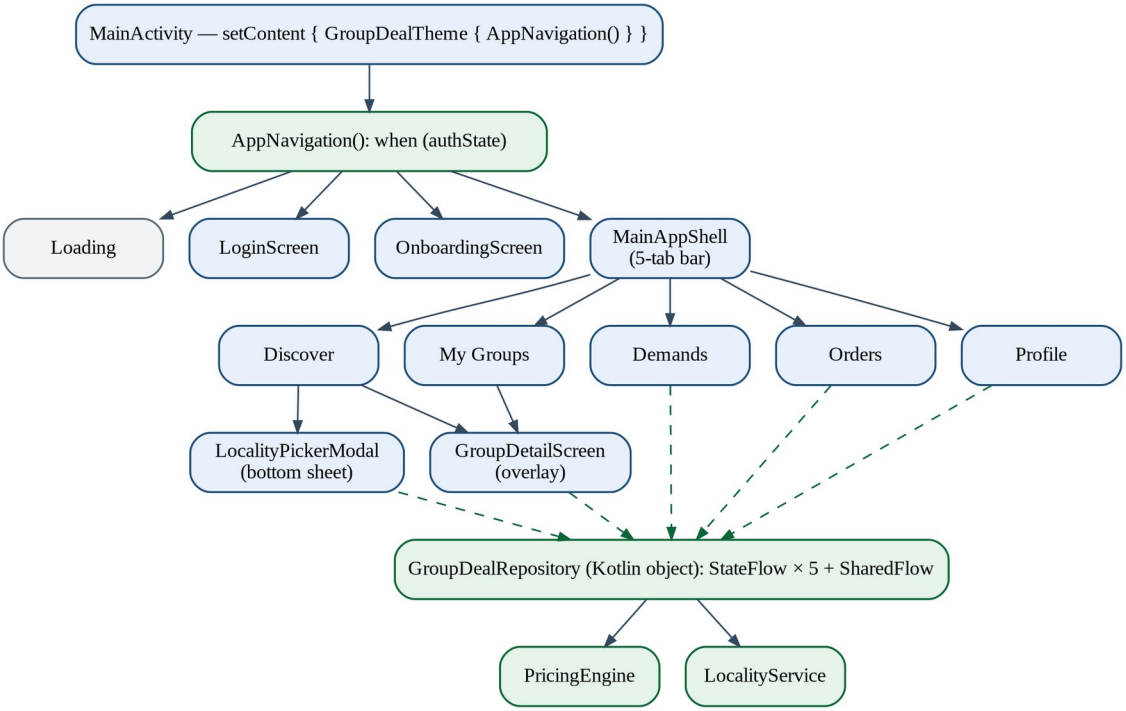


Figure 5.4: Application Architecture of the Current Prototype

5.1.4 Client-Side Architecture

On the client, the prototype uses a **single-module** structure with two top-level packages: `core` (data, engine and model) and `ui` (screens, components, navigation and theme). Figure 5.5 places the prototype in the context of the proposed services: blue solid elements are implemented; orange dashed elements are planned for the next phase. The prototype does not use ViewModel classes; composables collect the repository's StateFlows directly. In the complete system a ViewModel layer with Hilt dependency injection is introduced between the screens and the repositories (Section 7.3).

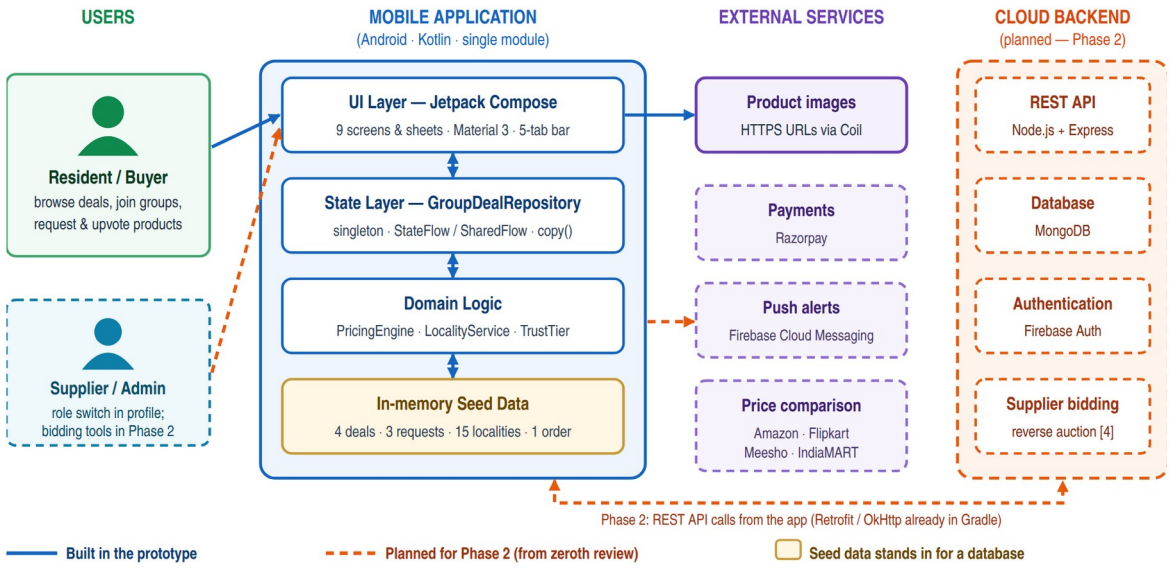


Figure 5.5: As-Built Prototype Architecture with Planned Integrations

5.1.5 Backend / Cloud Architecture

The proposed backend is serverless and built on Firebase. **Firebase Authentication** verifies users by phone OTP or Google sign-in [20]. **Cloud Firestore** stores localities, deals, memberships, requests, bids, orders and payments, and pushes changes to every listening device in real time. **Cloud Functions** hold the logic that must not run on the device: recomputing the tier price inside a transaction when members join, closing auctions at their deadline, creating Razorpay orders and verifying payment signatures [21], [29]. **Cloud Messaging** delivers price-drop, auction and order notifications [22], and **App Check** restricts backend access to genuine app instances [30].

5.1.6 Data Architecture

In the prototype, all data lives in five StateFlows inside `GroupDealRepository`: authentication state, current locality, deals, requests and orders, plus a SharedFlow of locality events. Seed data provides 4 deals, 3 requests, 1 order and 15 localities, and resets when the app restarts. In the complete system, Firestore is the source of truth, a Room database caches deals for offline viewing, and the repositories expose the same StateFlow interface to the UI, so the screens do not need to change when the backend is added. The proposed data model is shown in Section 5.7.

5.1.7 External Service Integration

Table 5.1: External Services and Integration Status

Service	Purpose in GroupDeal	Status
Product image hosts (HTTPS)	Product photographs loaded by Coil’s AsyncImage from image URLs	Implemented

Service	Purpose in GroupDeal	Status
Razorpay	Checkout (UPI, cards, netbanking) with server-side order creation and signature verification [23]	Proposed
Google Maps / Places	Locality search, pincode validation and nearby-area discovery [24]	Proposed
Firebase Cloud Messaging	Push notifications for price drops, auctions and order status [22]	Proposed
Marketplace price sources	Reference prices for comparison, where affiliate or partner APIs are available	Proposed

5.2 Use Case Diagram

The use case diagram (Figure 5.6) shows the interactions of the three actors with the system. Solid use cases are available in the prototype; dashed use cases belong to the proposed system. The customer is the primary actor and participates in most use cases. The diagrams in this chapter use standard UML notation [3].

SIRAGU

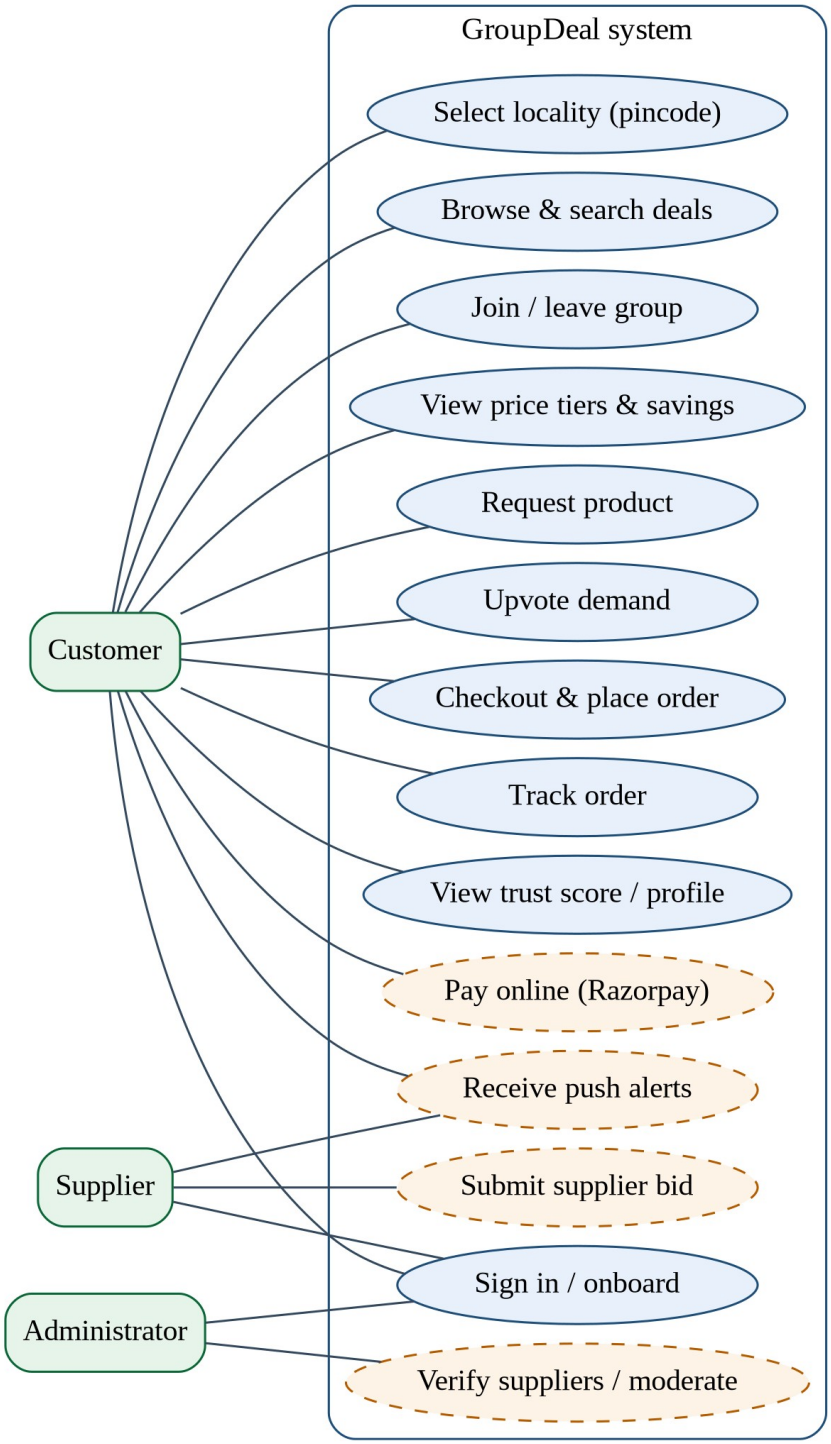


Figure 5.6: Use Case Diagram of GroupDeal

- **Sign in / onboard** — access the app and create a profile with locality and account type.
- **Select locality** — choose the delivery locality by search or pincode.
- **Browse & search deals; view tiers & savings** — discover group deals and inspect the pricing roadmap.

- **Join / leave group** — change membership; triggers price recomputation.
- **Request product; upvote demand** — publish and support local demand.
- **Checkout, track order** — place an order and follow its timeline.
- **Submit bid; pay online; verify suppliers; receive push alerts** — proposed use cases for suppliers, administrators and customers.

5.3 Activity Diagram

Figure 5.7 shows the activity of a customer joining a group deal. The decision nodes capture the two branches present in the source code: a user who is already a member can only leave, and after joining the user may check out immediately or later from My Groups.

SIRAGU

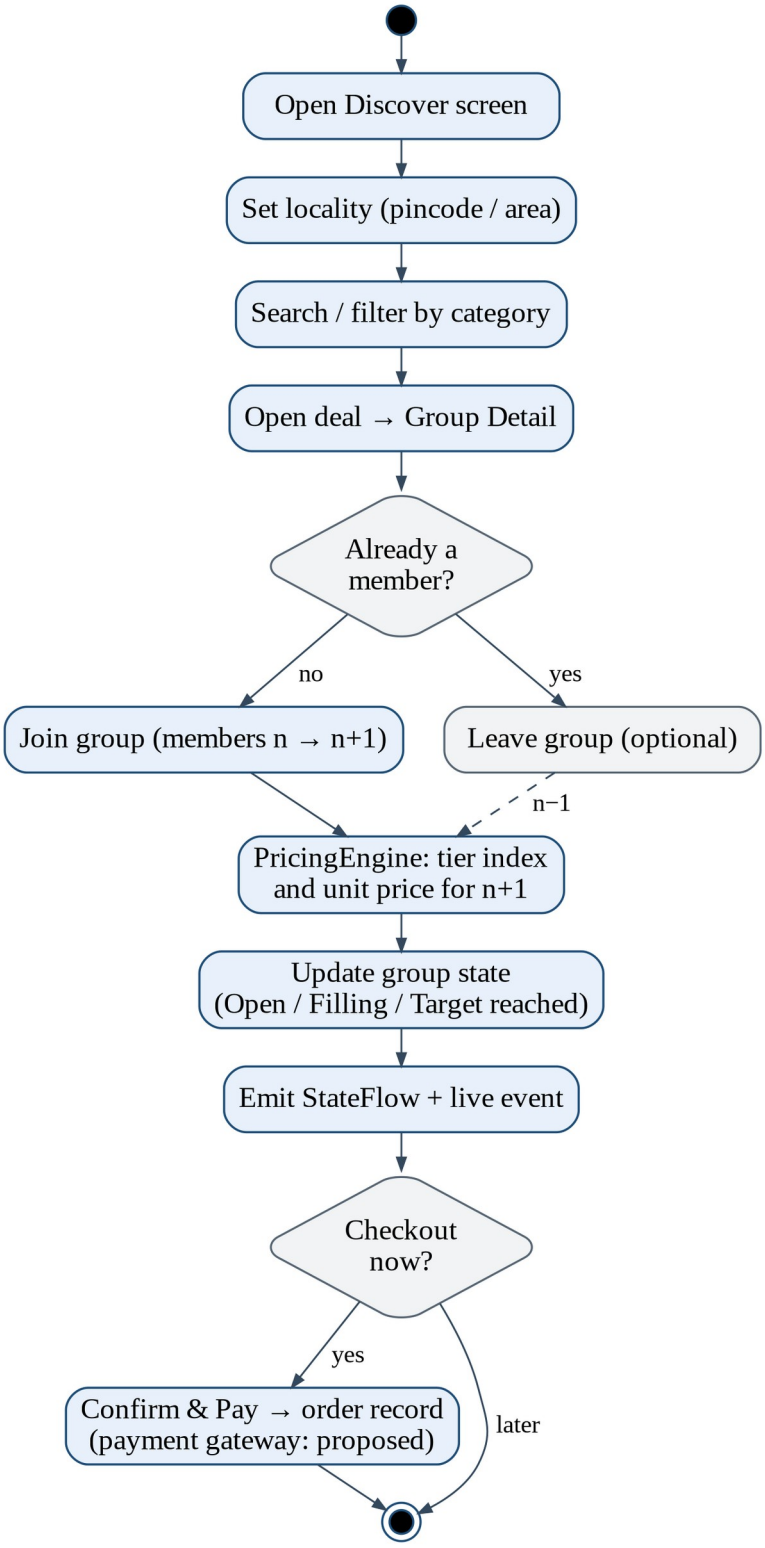


Figure 5.7: Activity Diagram — Joining a Group Deal

5.4 Application Flowchart

The application flowchart (Figure 5.8) follows `AppNavigation`. On launch the authentication state is Loading; it becomes Unauthenticated (Login), OnboardingRequired

(for a new user) or Authenticated (main shell). In the prototype a sign-in with an email address containing “new” simulates a first-time user and opens onboarding.

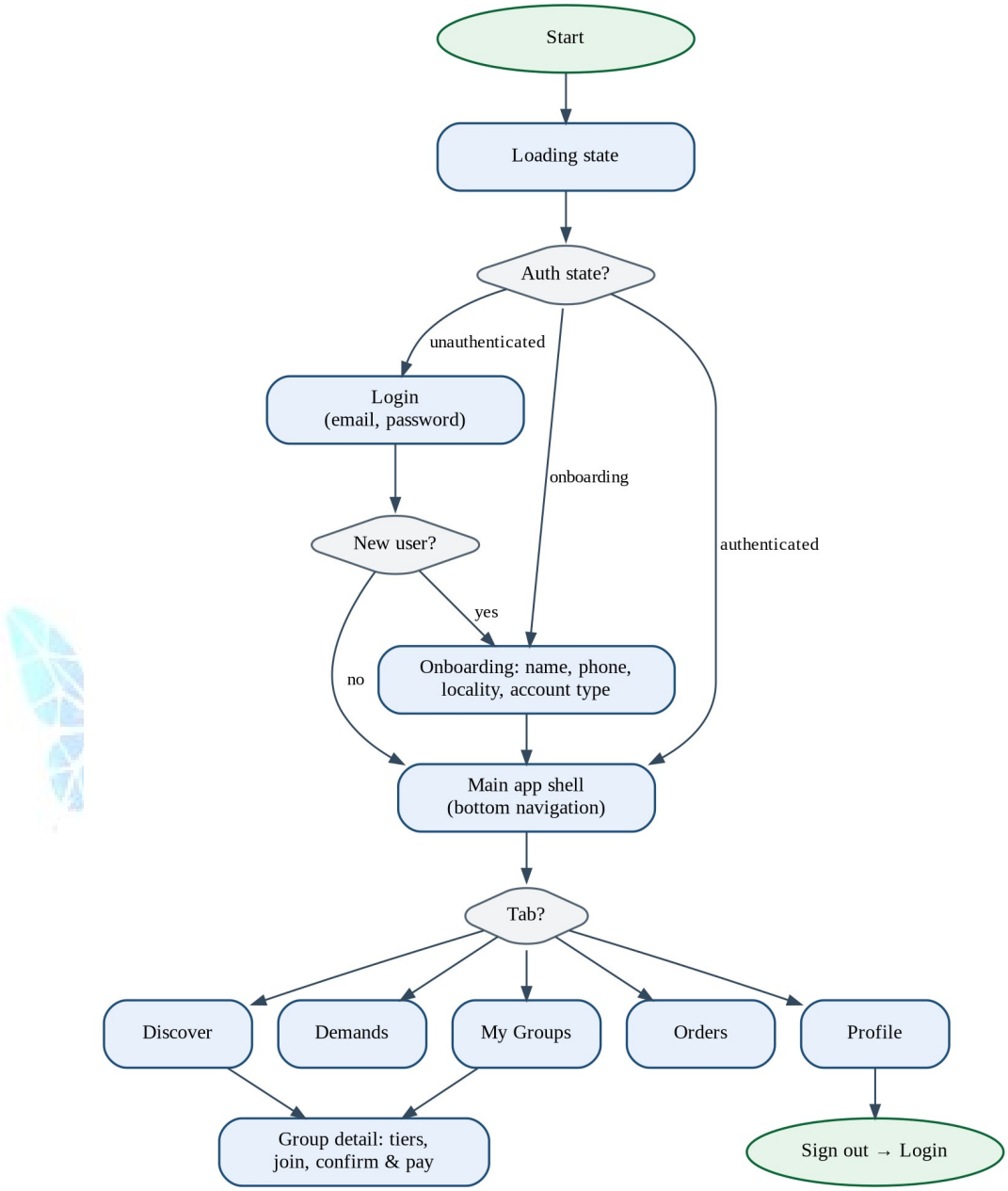


Figure 5.8: Application Flowchart

5.5 Class Diagram

The class diagram (Figure 5.9) is derived from the Kotlin data classes and objects in the `core` package. `GroupDeal` aggregates a list of `PricingTier` objects and may reference a winning `SupplierBid`; its derived properties compute the active tier, next tier, members to next tier, saving percentage and total community savings. `GroupDealRepository` manages deals, requests and orders and delegates price computation to the stateless `PricingEngine` object.

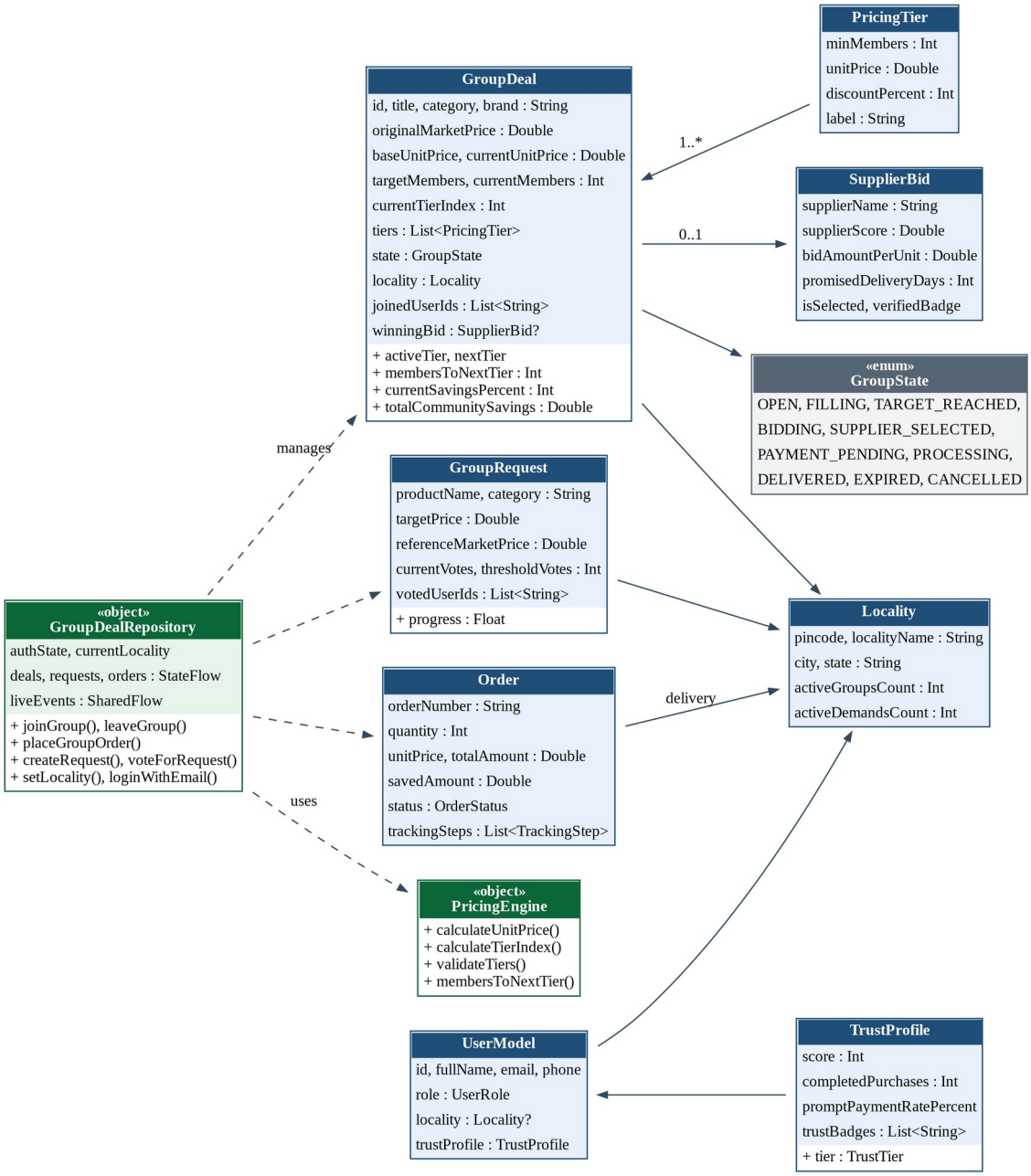


Figure 5.9: Class Diagram of the Core Data Model

5.6 Sequence Diagram

Figure 5.10 traces the join operation exactly as implemented. For the seeded ghee deal with 18 members, joining raises the count to 19; the engine returns tier index 1 and a unit price of ₹949; the repository derives the state FILLING (because 19 ≥ 0.6 × 30), replaces the deal with an updated copy and publishes the new list, and every screen collecting the deals flow recomposes. A live event is also emitted for the banner.

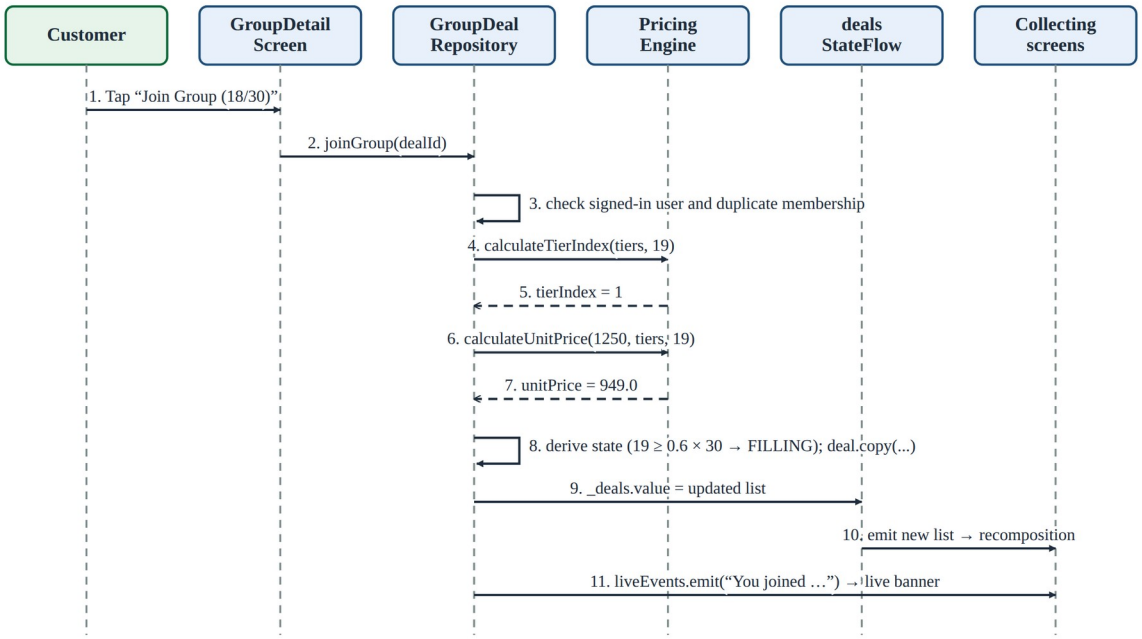


Figure 5.10: Sequence Diagram — Join Group (Implemented Prototype)

Figure 5.11 shows the proposed cloud sequence for the same operation, followed by payment. The key difference is that the price is computed by a Cloud Function after a Firestore transaction, so all members’ devices receive the same authoritative price through snapshot listeners, and payment is verified on the server.

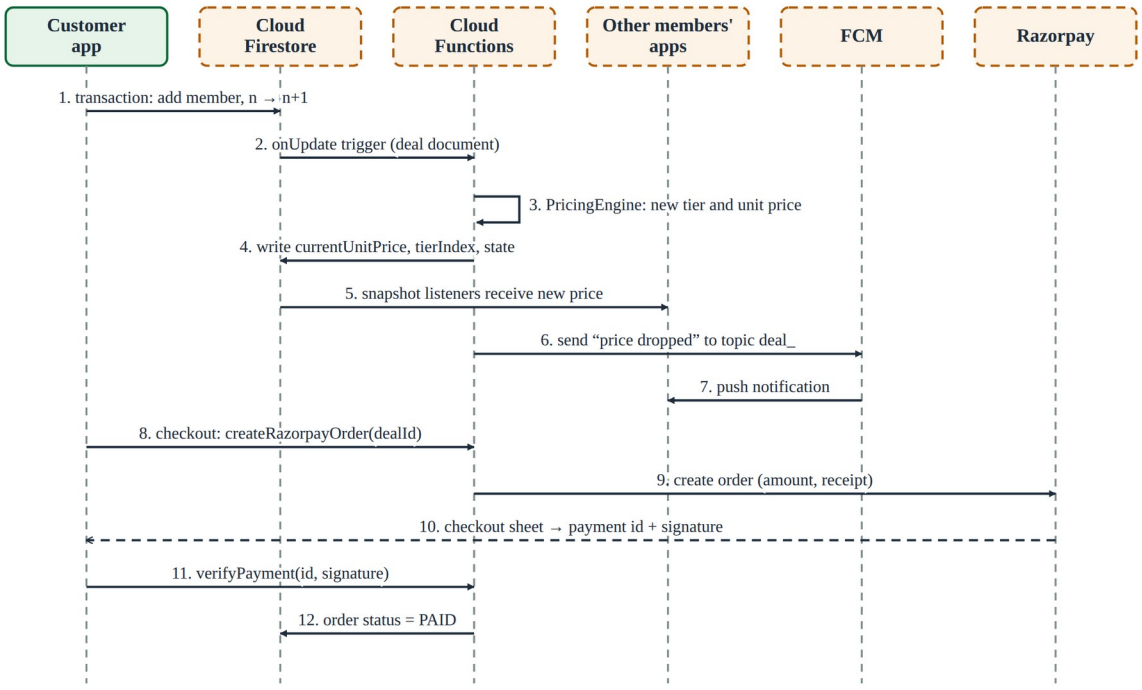


Figure 5.11: Sequence Diagram — Join and Pay (Proposed Cloud System)

5.7 Data Model / ER Representation

Figure 5.12 presents the proposed Firestore data model as an ER-style diagram. The prototype's Kotlin classes map directly to these collections: `GroupDeal` to *deals* (with members stored in a sub-collection to allow transactional joins), `GroupRequest` to *requests*, `SupplierBid` to *bids*, `Order` to *orders* and `UserModel` with `TrustProfile` to *users*. A separate *payments* collection records Razorpay order and verification data.

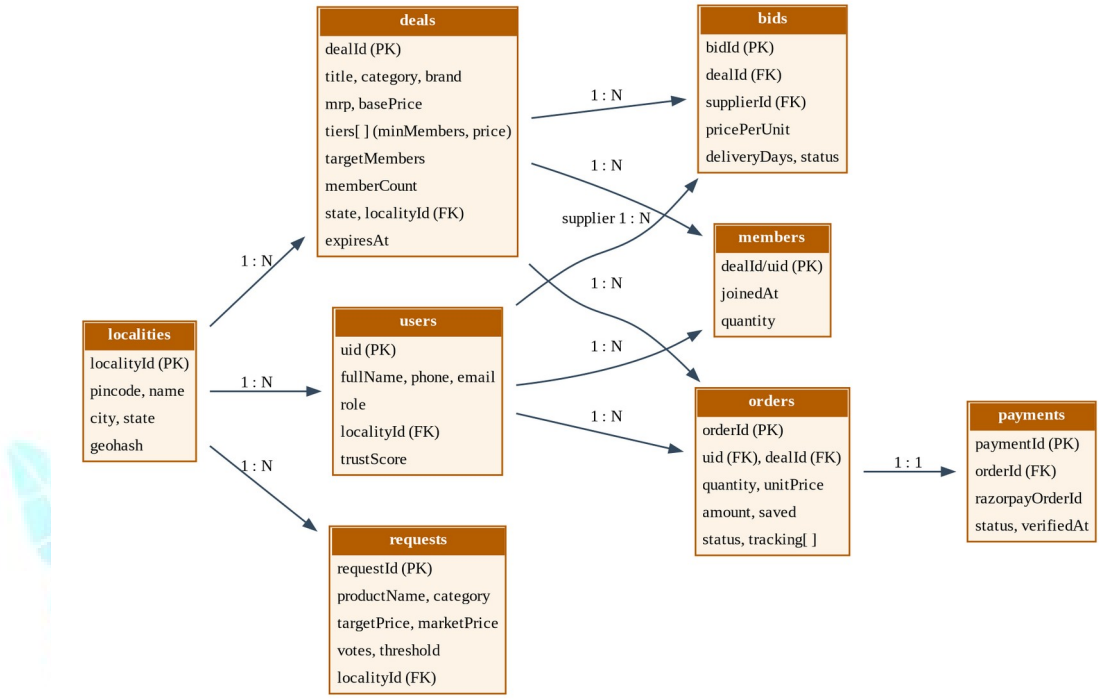


Figure 5.12: Proposed Data Model (ER Representation of Firestore Collections)

5.8 Data Flow Diagram

The level-1 data flow diagram (Figure 5.13) shows six processes. Processes 1.0 to 5.0 are implemented in the prototype; process 6.0 (supplier bidding) is proposed. Data store D2 (deals and tiers) is central: it is read to display prices, updated by the pricing process after each membership change, and read by the order process to fix the unit price at checkout.

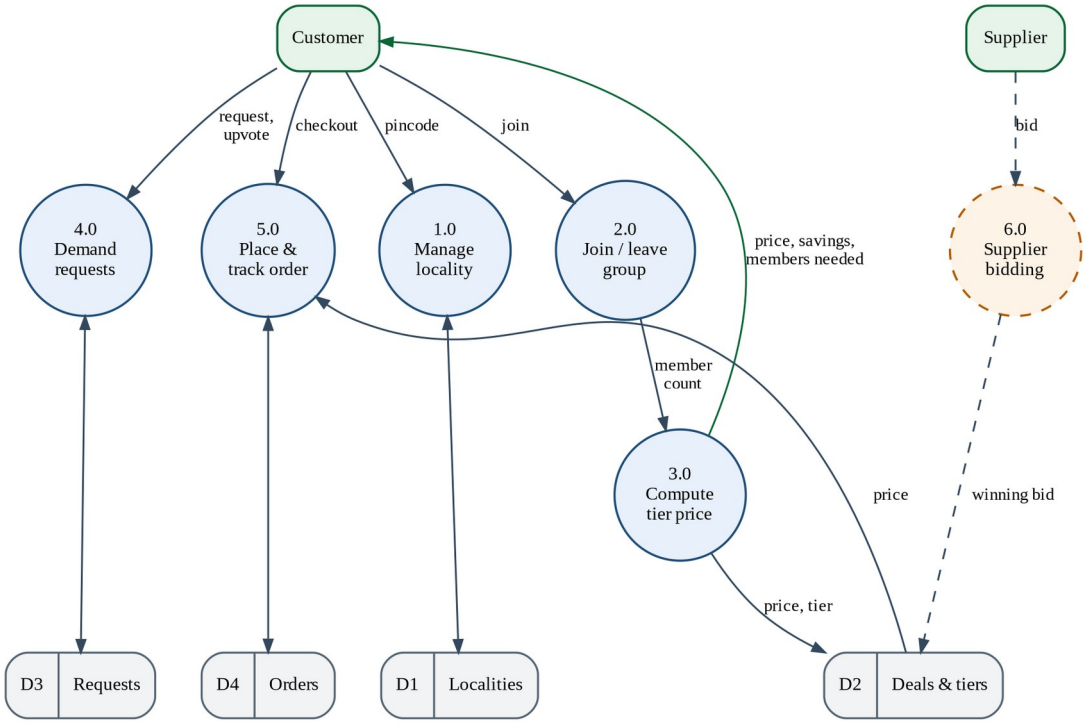


Figure 5.13: Data Flow Diagram (Level 1)

5.9 Pricing Engine Design

The pricing engine is the core algorithm of GroupDeal. It implements **tier-based, rule-based dynamic pricing**: the price is a deterministic step function of the member count. No machine-learning or statistical prediction is involved.

5.9.1 Inputs and Outputs

Table 5.2: Pricing Engine Inputs and Outputs

Type	Name	Meaning
Input	baseUnitPrice	Price per unit before any tier is reached
Input	originalMarketPrice (MRP)	Market reference price used to compute savings
Input	tiers: List<PricingTier>	Each tier has minMembers (threshold) and unitPrice
Input	currentMembers (n)	Number of members currently in the group
Output	unitPrice	Price of the highest tier reached, otherwise the base price
Output	tierIndex	Index of the active tier in ascending order (−1 if none)
Output	savings %	⌊(MRP − unitPrice) / MRP × 100⌋
Output	membersToNextTier	next.minMembers − n, or none at the highest tier

5.9.2 Algorithm

The algorithm is shown as a flowchart in Figure 5.14 and as pseudocode below.

```
ALGORITHM CalculateGroupPrice(baseUnitPrice, tiers, n)
  IF tiers is empty OR n <= 0 THEN RETURN baseUnitPrice
  sortedDesc <- tiers sorted by minMembers in descending order
  FOR EACH tier IN sortedDesc DO
      IF n >= tier.minMembers THEN RETURN tier.unitPrice   // highest threshold
reached
  END FOR
  RETURN baseUnitPrice                                     // below the first
threshold

ALGORITHM MembersToNextTier(tiers, n)
  next <- first tier in ascending order with minMembers > n
  IF next = NONE THEN RETURN NONE ELSE RETURN next.minMembers - n

ALGORITHM ValidateTiers(tiers)       // intended to run when a deal is created
  IF tiers is empty THEN RETURN FALSE
  FOR i <- 0 TO size - 2 DO
      IF tiers[i].minMembers >= tiers[i+1].minMembers THEN RETURN FALSE
      IF tiers[i].unitPrice  <= tiers[i+1].unitPrice  THEN RETURN FALSE
  RETURN TRUE
```

Listing 5.1: Pseudocode of the Pricing Engine

SIRAGU

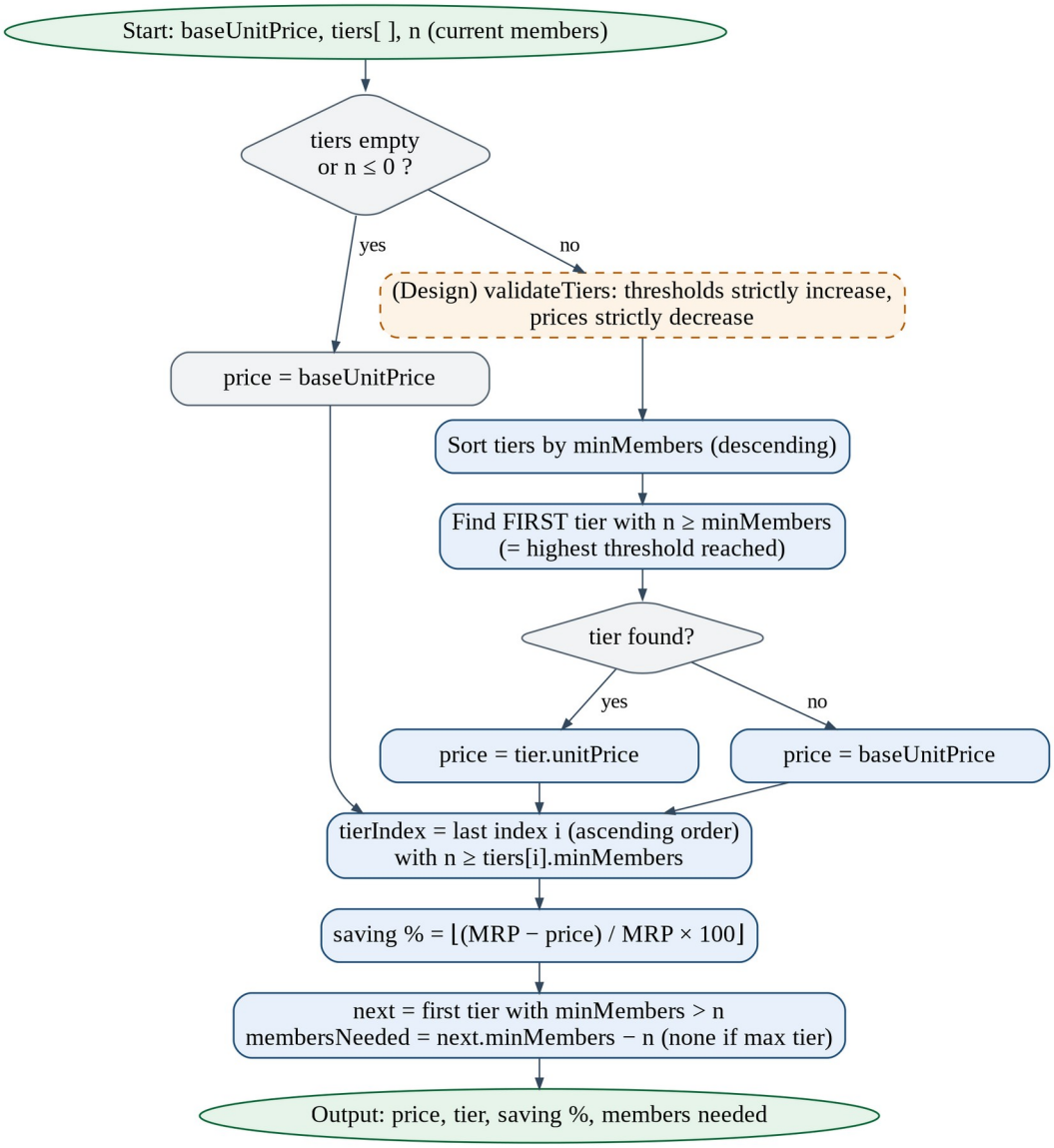


Figure 5.14: Flowchart of the Tier-Based Pricing Engine

Sorting in descending order and taking the **first** tier whose threshold is satisfied guarantees that the highest reached tier is chosen even if the tiers are stored out of order. The time complexity is O(k log k) for k tiers, which is negligible because deals have three tiers.

5.9.3 Worked Example

For the seeded deal "A2 Gir Cow Ghee (1 L)", the MRP is ₹1,400, the base price is ₹1,250 and the tiers are 5 members → ₹1,099, 15 members → ₹949 and 30 members → ₹799. With 18 members, the tiers are examined in the order 30, 15, 5; the first satisfied threshold is 15, so the unit price is ₹949 (tier 2 of 3). The saving is ⌊(1400 − 949) / 1400 × 100⌋ = 32%, the next tier is 30 members and 30 − 18 = **12 more members** are needed to reach ₹799. The total saving across the locality is (1400 − 949) × 18 = ₹8,118. Figure 5.15 plots the complete price schedule.

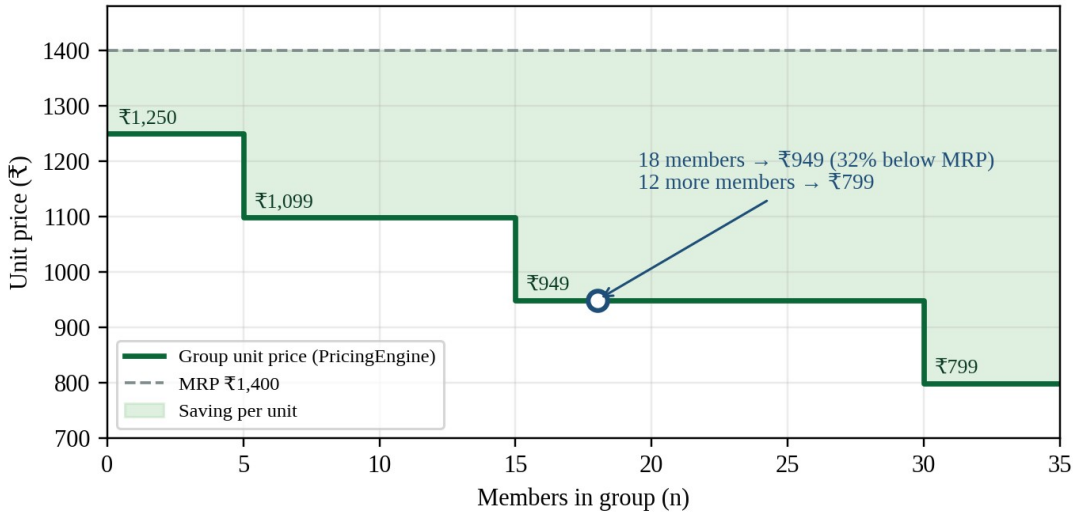


Figure 5.15: Tier Progression of the Seeded Ghee Deal (Price versus Group Size)

5.9.4 Edge Cases

Table 5.3: Pricing Engine Edge Cases

Case	Behaviour
No tiers defined	Base price is returned.
n = 0 or negative	Base price is returned (guard at the start of the function).
n below the first threshold (e.g. 4 of 5)	No tier matches; base price is returned; next tier needs 1 member.
n exactly on a threshold (e.g. 15)	The tier applies (comparison is ≥).
n above the highest threshold (e.g. 35 of 30)	Highest tier price applies; no next tier.
Tiers stored out of order	Correct tier selected because tiers are sorted before matching.
Non-monotonic tiers (price rises with members)	Rejected by validateTiers(); in the prototype the function exists but is not yet called when deals are created.

5.10 Supplier Auction Design

The supplier auction is a **proposed** reverse auction triggered when a group reaches its target. The prototype already models its outcome — the `SupplierBid` class, the BIDDING and SUPPLIER_SELECTED states and seeded winning bids for two deals (Vedic Farms Co-op at ₹949 for the ghee deal and PurePress Natural Oils at ₹319 for the groundnut oil deal) — but bid submission and selection are not yet implemented. Figure 5.16 shows the proposed workflow.

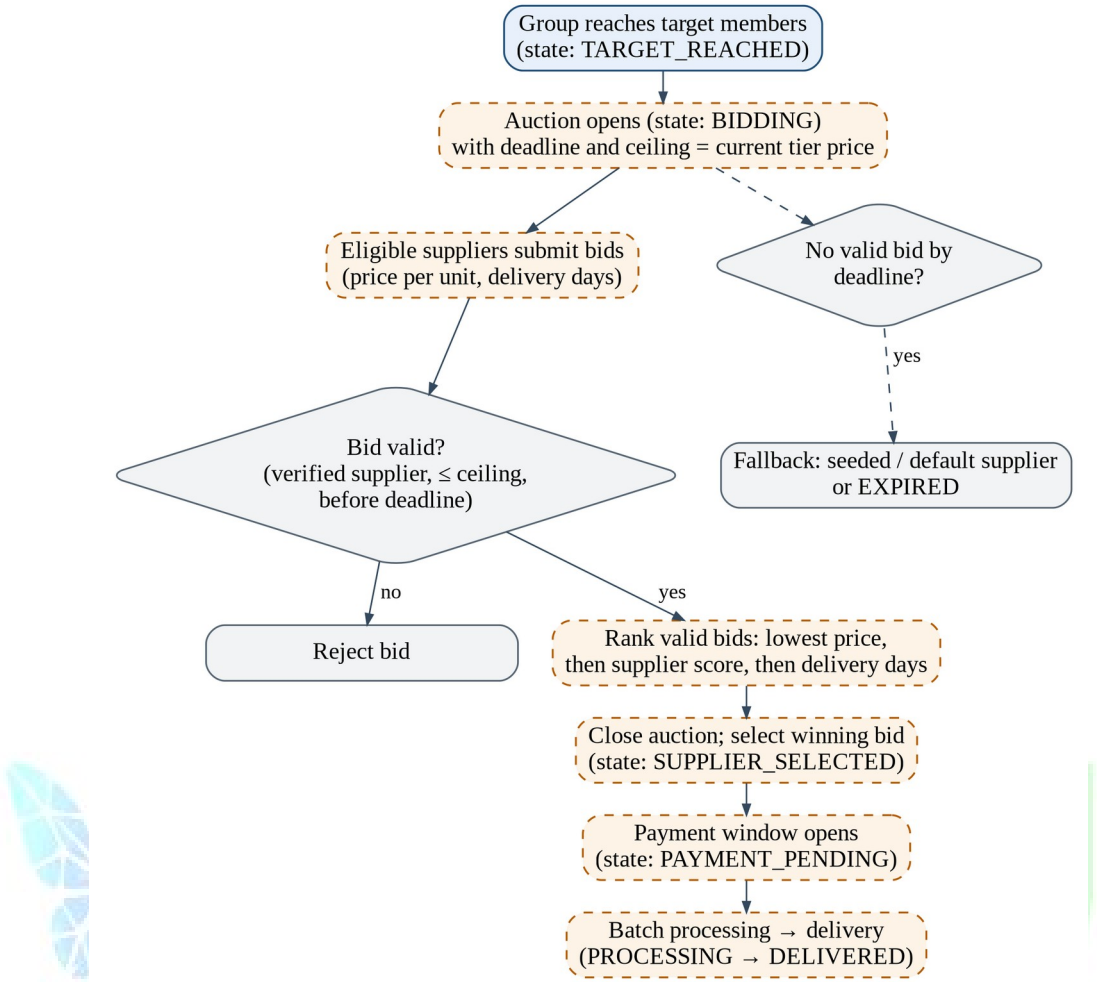


Figure 5.16: Reverse Auction Workflow (Proposed)

The auction opens with a **ceiling** equal to the current tier price, so members never pay more than the price they joined at. A bid is valid if it comes from a verified supplier, does not exceed the ceiling and arrives before the deadline. Valid bids are ranked by price, then supplier score, then promised delivery days; a Cloud Function closes the auction at the deadline and records the winning bid. If no valid bid arrives, the group falls back to a default supplier or expires. The complete lifecycle of a group, as defined by the `GroupState` enumeration, is shown in Figure 5.17; in the prototype only the transitions OPEN → FILLING → TARGET_REACHED are computed automatically.

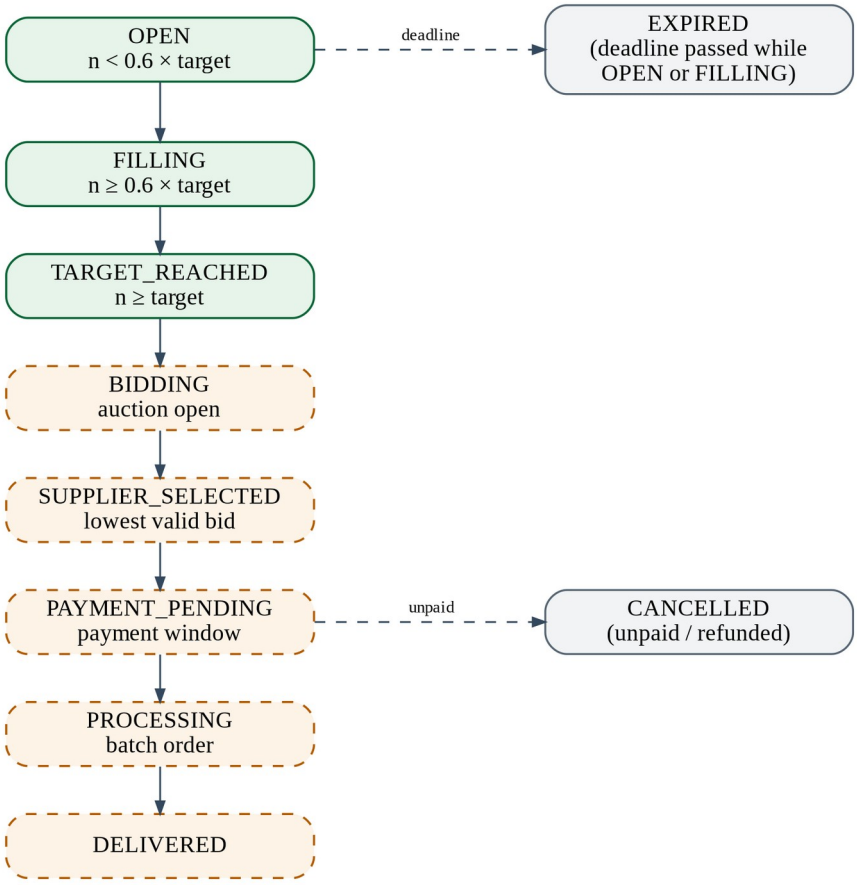


Figure 5.17: Group Deal State Machine (GroupState Enumeration)

5.11 Payment Flow Design

The proposed payment flow (Figure 5.18) follows the standard server-verified pattern for Razorpay [23]. The amount is computed on the server from the final group price; the app opens Razorpay Checkout; the returned payment identifier and signature are sent to a Cloud Function that verifies the signature with the secret key held only on the server; only then is the order marked as paid. In the prototype, “Confirm & Pay” creates a local order record and no payment is processed.

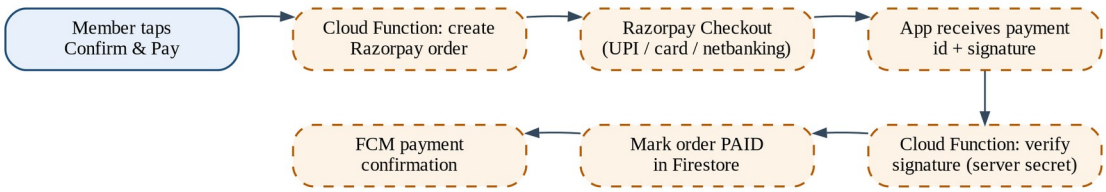


Figure 5.18: Payment Flow (Proposed Razorpay Integration)

5.12 Notification Flow Design

Figure 5.19 contrasts the implemented in-app notification mechanism with the proposed push notifications. In the prototype, repository actions emit `LocalityEvent` messages on a

SharedFlow, which the main shell displays as a live banner. In the complete system, Firestore triggers invoke Cloud Functions that send FCM messages to a topic per deal, so members are notified even when the app is closed.

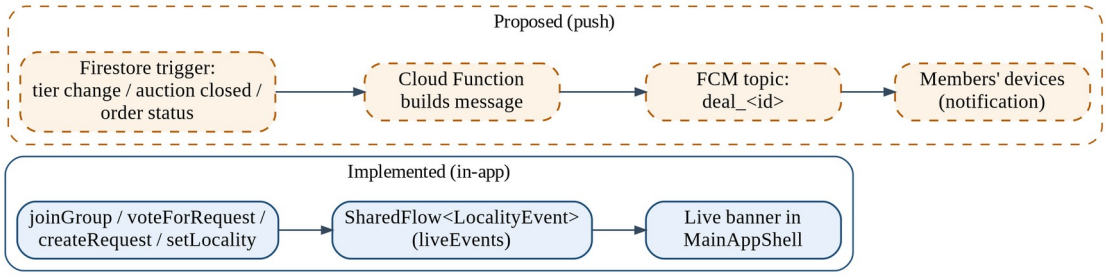


Figure 5.19: Notification Flow: Implemented In-App Events and Proposed FCM Push

SIRAGU

CHAPTER 6

DEVELOPMENT TOOLS AND TECHNOLOGIES

This chapter documents the tools and technologies of GroupDeal. The technology set presented on the project poster represents the complete system; each technology was checked against the project's Gradle configuration and Kotlin source code to determine whether it is **used** in the prototype, **declared** (present as a dependency but not yet used in code) or **proposed** (part of the design only). Figure 6.1 summarises the result.

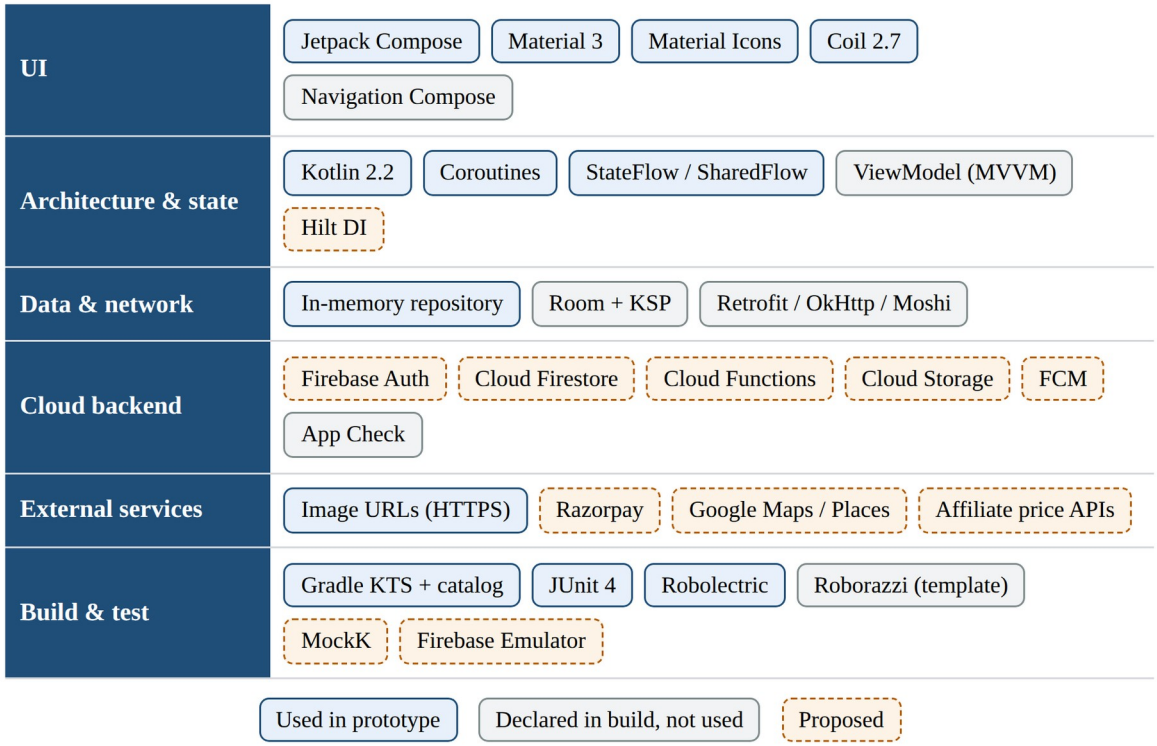


Figure 6.1: Technology Stack of GroupDeal by Layer and Status

6.1 Development Environment

GroupDeal is a Gradle-based Android project written in Kotlin. The project is built with the Android Gradle Plugin 9.1.1 and Gradle Kotlin DSL build scripts, with library versions centralised in a version catalog (`libs.versions.toml`). It is opened, built, run and debugged in **Android Studio**, and executed on the Android Emulator or a physical Android device running Android 7.0 (API 24) or later. Unit tests run on the JVM through Gradle.

Table 6.1: Hardware Requirements for Development and Use

Component	Minimum Requirement	Recommended
Development processor	64-bit x86 or ARM CPU with virtualization support	Quad-core or better
Development memory	8 GB RAM	16 GB RAM (emulator + IDE)

Component	Minimum Requirement	Recommended
Development storage	10 GB free (SDK, emulator images, Gradle cache)	SSD with 20 GB free
Test device	Android smartphone with Android 7.0 (API 24)	Android 12 or later
Network	Internet for dependency download and product images	Broadband / Wi-Fi

Table 6.2: Software Requirements and Versions

Software	Version / Detail	Purpose
Android Studio	Current stable release	IDE, emulator, profiler, Compose preview
Android SDK	compileSdk 36, targetSdk 36, minSdk 24	Platform APIs and build tools
Kotlin	2.2.10 (with Compose compiler plugin)	Programming language
Android Gradle Plugin	9.1.1, Gradle Kotlin DSL	Build automation
Jetpack Compose	Compose BOM 2024.09.00; Material 3	Declarative UI toolkit
Kotlin Coroutines	1.10.2 (core and android)	Asynchronous tasks, StateFlow, SharedFlow
Coil	2.7 (coil-compose)	Asynchronous image loading
JUnit 4, Robolectric	Robolectric configured for SDK 36	JVM unit tests
Git	Any recent version	Version control
Operating system	Windows, macOS or Linux	Development host

6.2 Android Studio

Android Studio is the official integrated development environment for Android. It provides the Kotlin editor, Gradle integration, Compose previews, the layout inspector, the Android Emulator and on-device debugging. For GroupDeal it is used to edit the Kotlin sources, preview Compose screens, run the application on emulators with different screen sizes and API levels, and execute the unit tests.

6.3 Programming Language – Kotlin

Kotlin is the programming language of the entire prototype (22 Kotlin source files, 5,434 lines in the main source set). GroupDeal relies on several Kotlin features: **data classes** with `copy()` for immutable state updates (every membership change produces a new `GroupDeal` object rather than mutating the old one); **enum classes** with properties for `GroupState`, `OrderStatus`, `UserRole` and `TrustTier`; **sealed interfaces** for the authentication state; **object declarations** for the stateless `PricingEngine` and `LocalityService`; **null safety**

for optional values such as the winning bid; and collection operations such as `sortedByDescending` and `firstOrNull` in the pricing algorithm.

6.4 Jetpack Compose and Material 3

The user interface is written entirely in **Jetpack Compose**, Android's declarative UI toolkit [17]; the project does not use XML layouts. Each screen is a composable function that renders the current state, and Compose automatically recomposes the affected parts of the screen when that state changes. **Material 3** components — Scaffold, NavigationBar, Cards, OutlinedTextField, FilterChip, LinearProgressIndicator, ModalBottomSheet and Buttons — provide consistent design [25]. The app defines its own theme with an emerald primary colour (#0F9D58), a dark-emerald secondary colour and an orange tertiary colour for deal badges, with separate light and dark colour schemes; dynamic (wallpaper-based) colour is disabled by default so that the GroupDeal brand colours stay consistent. The extended Material Icons set is used throughout.

6.5 State Management – Coroutines, StateFlow and SharedFlow

Kotlin coroutines provide lightweight asynchronous execution [18]. GroupDeal uses a coroutine scope inside the repository for the simulated sign-in delay and for emitting live events. **StateFlow** is a hot flow that always holds a current value [19]; the repository exposes five StateFlows (authentication state, current locality, deals, requests and orders) that the screens collect with `collectAsState()`. **SharedFlow** carries one-off events: the repository emits a `LocalityEvent` when a user joins a group, votes, creates a request or changes locality, and the main shell shows it as a live banner.

6.6 Image Loading – Coil

Coil is a Kotlin-first image-loading library [26]. GroupDeal uses Coil's `AsyncImage` composable to download and cache product photographs from their HTTPS URLs on deal cards and on the Group Detail screen. This is the prototype's only network interaction.

6.7 Build System – Gradle and Version Catalog

The build uses Gradle Kotlin DSL scripts and a version catalog, so every library version is declared once. The app module applies the Android application plugin, the Compose compiler plugin, KSP (Kotlin Symbol Processing), Roborazzi (screenshot testing), the Secrets Gradle plugin and the Google Services plugin. The last two prepare the project for API keys and Firebase configuration without storing secrets in source code.

6.8 Firebase / Backend Technology

Firebase is the backend platform of the proposed system. Its role in GroupDeal is summarised below; none of these services is active in the prototype, where the repository simulates their behaviour.

- **Firebase Authentication** — phone OTP and Google sign-in for customers, suppliers and administrators [20]. The prototype's login button is labelled "Sign In with Firebase" but performs a simulated 400 ms sign-in.
- **Cloud Firestore** — document database for deals, members, requests, bids, orders and payments, with real-time listeners and transactions [21].
- **Cloud Functions** — server-side price recomputation, auction closing, Razorpay order creation and signature verification [29].
- **Cloud Storage** — product and profile images uploaded by suppliers and users.
- **Firebase Cloud Messaging** — push notifications to members of a deal [22].
- **Firebase App Check** — attests that requests come from the genuine app [30]; the App Check library is already declared in the build.

6.9 Database Technology

The prototype does not use a database: seed data is created in memory when the repository initialises and is lost when the app process ends. The proposed design uses **Cloud Firestore** as the shared, authoritative database and **Room**, Android's SQLite abstraction, as a local cache so that deals and orders remain viewable offline [31]. Room is already declared as a dependency, and the KSP plugin required for Room's annotation processing is applied.

6.10 APIs and Web Services

The proposed system communicates with REST-style services through **Retrofit** with **OkHttp** and the **Moshi** JSON converter [32], all of which are declared in the build but not yet called from code. The external APIs are Razorpay for payments [23], Google Places for locality search [24] and, where access is available, e-commerce affiliate APIs for reference prices. Marketplace price APIs are not openly available for every platform, so the design treats the supplier-provided MRP as the primary reference and marketplace prices as an optional enrichment.

6.11 Architecture Libraries – MVVM and Hilt

The complete system introduces the **MVVM** pattern: each screen obtains its state from a ViewModel that survives configuration changes, and repositories are injected with **Hilt**, Android's dependency-injection library [28]. In the prototype, the lifecycle ViewModel

Compose artifact and Navigation Compose are declared, but no ViewModel classes or NavHost are used; screen selection is driven by state in `AppNavigation`.

6.12 Testing Tools

Unit tests use **JUnit 4**. Tests that need Android resources run on the JVM with **Robolectric** [27], configured for SDK 36. The project also contains an instrumented test template and a Roborazzi screenshot-test template. **MockK** is proposed for mocking repositories and Firebase services in ViewModel tests, and the **Firebase Local Emulator Suite** is proposed for integration tests of Cloud Functions and Firestore rules.

6.13 Version Control – Git

Git is used for version control of the source code, Gradle scripts and documentation, allowing both team members to work on UI and logic modules in parallel and to review changes before merging.

6.14 Technology Stack Summary

Table 6.3: Software Development Tools and Technologies with Verified Status

Technology	Role in GroupDeal	Status
Kotlin 2.2	Application language	Used
Jetpack Compose + Material 3	All screens and components	Used
Material Icons (extended)	Icons across the UI	Used
Coroutines, StateFlow, SharedFlow	State holder and live events	Used
Coil 2.7	Product image loading	Used
Gradle KTS + version catalog	Build and dependency management	Used
JUnit 4 + Robolectric	Unit tests	Used
Navigation Compose	Route-based navigation	Declared, not used
ViewModel (lifecycle-viewmodel-compose)	MVVM state holders	Declared, not used
Room + KSP	Offline cache	Declared, not used
Retrofit, OkHttp, Moshi	REST client and JSON	Declared, not used
Firebase App Check, Firebase AI	App attestation; AI features	Declared, not used
Hilt	Dependency injection	Proposed
Firebase Auth, Firestore, Functions, Storage, FCM	Cloud backend	Proposed
Razorpay	Payment gateway	Proposed
Google Maps / Places	Locality services	Proposed
MockK, Firebase Emulator Suite	Advanced testing	Proposed

CHAPTER 7

SOFTWARE IMPLEMENTATION

7.1 Project Setup

The project is configured as a single Android application module. The module's Gradle script sets the compile and target SDK to 36 and the minimum SDK to 24, enables Jetpack Compose and declares the dependencies through the version catalog. `MainActivity` is the only activity; it calls `setContent` with the application theme and hosts the `AppNavigation` composable. The manifest declares this single launcher activity, and the prototype requests no runtime (dangerous) permissions.

```
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContent {
            GroupDealTheme {
                AppNavigation()      // auth-state driven root composable
            }
        }
    }
}
```

Listing 7.1: Application Entry Point (MainActivity, abridged)

7.2 Project Structure

The source code is organised by responsibility. The `core` package contains the data layer (`GroupDealRepository`, `LocalityService`), the pricing engine and the data models; the `ui` package contains one sub-package per feature plus shared components, navigation and theme. Figure 7.1 shows the structure with the line count of each core file.

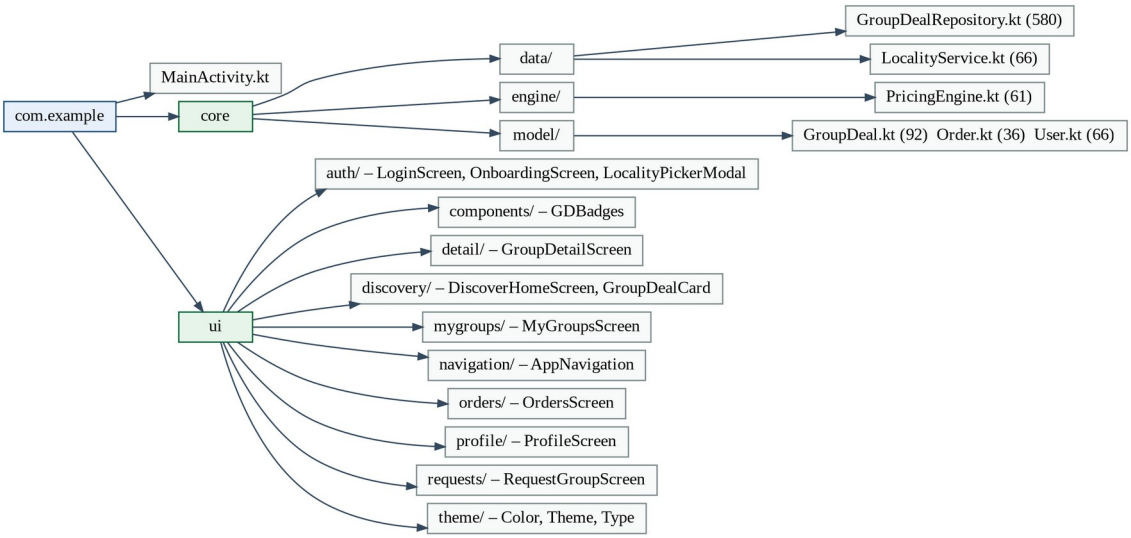


Figure 7.1: Project Structure / Application Package Structure

Table 7.1: Source Files and Responsibilities

File	Lines	Responsibility
core/data/GroupDealRepository.kt	580	Single source of truth: StateFlows, auth, locality, deals, requests, orders, seed data
core/data/LocalityService.kt	66	15 seeded localities, pincode lookup, locality search
core/engine/PricingEngine.kt	61	Tier price, tier index, tier validation, members to next tier
core/model/GroupDeal.kt	92	GroupState, PricingTier, SupplierBid, GroupDeal, GroupRequest
core/model/Order.kt	36	OrderStatus, TrackingStep, Order
core/model/User.kt	66	UserRole, TrustTier, Locality, TrustProfile, UserModel
ui/navigation/AppNavigation.kt	179	Auth-state routing and five-tab main shell
ui/auth/*	1,056	Login, onboarding and locality picker
ui/discovery/*	674	Discover screen and deal card
ui/detail/GroupDetailScreen.kt	581	Pricing roadmap, join, checkout
ui/requests/RequestGroupScreen.kt	441	Demand requests, upvotes, request form
ui/mygroups, ui/orders, ui/profile	1,097	Joined groups, order tracking, profile and trust
ui/components, ui/theme	466	Badges and reusable components; colours, typography, theme

7.3 Application Architecture

The prototype follows a **unidirectional data flow** (Figure 7.2). A screen raises an event by calling a repository function; the repository computes the new state (delegating price computation to `PricingEngine`), replaces the value of a `MutableStateFlow` with a new immutable list, and every screen collecting that flow recomposes. Because all screens observe the same flow, the Discover card, the Group Detail screen and My Groups always show the same member count and price, which satisfies requirement NFR2.

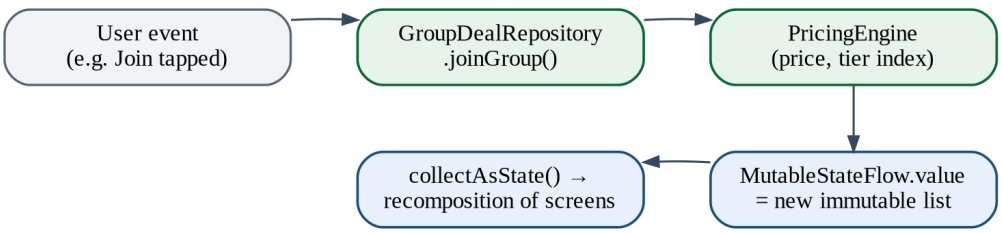


Figure 7.2: Unidirectional Data Flow in the Prototype

In the complete system, a ViewModel per screen will expose screen-specific `UiState` objects built from repository flows, and repositories will be injected with Hilt. This keeps the

existing screens largely unchanged while moving business logic out of the composables and making it testable with MockK.

7.4 UI Implementation

The UI consists of nine screens and sheets written as composable functions: Login, Onboarding, Locality Picker (modal bottom sheet), Discover, Group Detail, Demands, My Groups, Orders and Profile. Reusable components such as status badges are collected in `GDBadges.kt`. `AppNavigation` switches between the top-level screens with a `when` expression over the authentication state, and `MainAppShell` uses a Material 3 `Scaffold` with a `NavigationBar` of five items. The Group Detail screen is displayed as an overlay above the shell when a deal is selected. Chapter 8 describes each screen.

7.5 State Management

```
object GroupDealRepository {
    private val scope = CoroutineScope(Dispatchers.Default)

    private val _authState = MutableStateFlow<AuthUiState>(AuthUiState.Loading)
    val authState: StateFlow<AuthUiState> = _authState.asStateFlow()

    private val _currentLocality =
MutableStateFlow(LocalityService.POPULAR_LOCALITIES[0])
    val currentLocality: StateFlow<Locality> = _currentLocality.asStateFlow()

    private val _deals = MutableStateFlow<List<GroupDeal>>(emptyList())
    val deals: StateFlow<List<GroupDeal>> = _deals.asStateFlow()

    private val _requests = MutableStateFlow<List<GroupRequest>>(emptyList())
    val requests: StateFlow<List<GroupRequest>> = _requests.asStateFlow()

    private val _orders = MutableStateFlow<List<Order>>(emptyList())
    val orders: StateFlow<List<Order>> = _orders.asStateFlow()

    private val _liveEvents = MutableSharedFlow<LocalityEvent>(extraBufferCapacity
= 10)
    // ...
}
```

Listing 7.2: State Holders in GroupDealRepository (abridged)

Mutable flows are private and only read-only `StateFlow` views are exposed, so screens cannot modify state directly; all changes go through repository functions. The authentication state is a sealed interface with the cases Loading, Unauthenticated, OnboardingRequired and Authenticated, which lets `AppNavigation` handle every case exhaustively.

7.6 Repository Layer

`GroupDealRepository` is a Kotlin `object`, which makes it a process-wide singleton that every screen can access without dependency injection. Its public functions are grouped as follows: session (`loginWithEmail`, `completeOnboarding`, `updateProfile`,

`switchDemoRole`, `logout`), locality (`setLocality`), deals (`getDealById`, `joinGroup`, `leaveGroup`, `placeGroupOrder`) and demand (`createRequest`, `voteForRequest`). Private seeding functions create the initial deals, requests and orders. In the complete system the object is split into injected repositories (`AuthRepository`, `DealRepository`, `RequestRepository`, `OrderRepository`) backed by Firestore and Room.

7.7 Data Models

The data models are immutable Kotlin data classes (Appendix A.3). `GroupDeal` contains the product details, prices, tiers, member count, joined user identifiers, state, locality and optional winning bid, together with derived properties computed on demand. `PricingTier` defines a threshold, unit price, discount percentage and label. `GroupRequest` holds a demand request and the identifiers of users who voted. `Order` records the purchase and its tracking steps, and `UserModel` holds the profile, role, locality and `TrustProfile`.

```
val currentSavingsPercent: Int
    get() = if (originalMarketPrice > 0) {
        (((originalMarketPrice - currentUnitPrice) / originalMarketPrice) *
100).toInt()
    } else 0

val totalCommunitySavings: Double
    get() = (originalMarketPrice - currentUnitPrice) * currentMembers
```

Listing 7.3: Derived Savings Properties of GroupDeal

7.8 Locality Service

`LocalityService` provides the list of 15 popular localities, a lookup by pincode and a search function. `lookupPincode` first looks for an exact match; if none is found and the input is a valid six-digit number, it creates a locality whose city is inferred from the first two digits of the pincode (for example, 56 or 57 → Bengaluru, 60 or 61 → Chennai), with any other prefix mapped to a generic “Metro City”. Inputs that are not six digits return `null`. `searchLocalities` matches the query against locality name, city, pincode and state.

```
fun lookupPincode(pincode: String): Locality? {
    val trimmed = pincode.trim()
    val found = POPULAR_LOCALITIES.firstOrNull { it.pincode == trimmed }
    if (found != null) return found
    if (trimmed.length == 6 && trimmed.all { it.isDigit() }) {
        val (city, state) = when (trimmed.substring(0, 2)) {
            "11" -> "Delhi" to "Delhi"
            "56", "57" -> "Bengaluru" to "Karnataka"
            "60", "61" -> "Chennai" to "Tamil Nadu"
            // ... other prefixes
            else -> "Metro City" to "India"
        }
        return Locality(trimmed, "Sector / Ward $trimmed", city, state)
    }
    return null
}
```

Listing 7.4: Pincode Lookup in LocalityService (abridged)

In the complete system this service is replaced by Google Places autocomplete and a validated pincode directory, and deals are queried by locality so that the Discover feed shows only deals for the selected area. In the prototype, the seeded deals all belong to Koramangala and the feed is filtered by search text and category but not by the selected locality.

7.9 PricingEngine

`PricingEngine` is a stateless Kotlin `object` with four pure functions. Because it has no dependencies and no side effects, it can be unit-tested directly and can be reused unchanged inside a Cloud Function (by sharing the rule in TypeScript) in the complete system.

```
fun calculateUnitPrice(baseUnitPrice: Double, tiers: List<PricingTier>,
                       currentMembers: Int): Double {
    if (tiers.isEmpty() || currentMembers <= 0) return baseUnitPrice
    val activeTier = tiers
        .sortedByDescending { it.minMembers }
        .firstOrNull { currentMembers >= it.minMembers }
    return activeTier?.unitPrice ?: baseUnitPrice
}

fun calculateTierIndex(tiers: List<PricingTier>, currentMembers: Int): Int {
    val sorted = tiers.sortedBy { it.minMembers }
    var activeIndex = -1
    for (i in sorted.indices) {
        if (currentMembers >= sorted[i].minMembers) activeIndex = i
    }
    return activeIndex
}
```

Listing 7.5: Core Functions of PricingEngine

7.10 Group Management

`joinGroup` implements the central use case. It returns immediately if no user is signed in, the deal does not exist or the user has already joined. Otherwise it computes the new member

count, obtains the tier index and unit price from `PricingEngine`, derives the group state, creates an updated copy of the deal and publishes the new list. `leaveGroup` performs the inverse, never allowing the count to fall below zero.

```
val newMemberCount = deal.currentMembers + 1
val newTierIndex = PricingEngine.calculateTierIndex(deal.tiers, newMemberCount)
val newPrice = PricingEngine.calculateUnitPrice(deal.baseUnitPrice, deal.tiers,
newMemberCount)
val newState = when {
    newMemberCount >= deal.targetMembers -> GroupState.TARGET_REACHED
    newMemberCount >= (deal.targetMembers * 0.6) -> GroupState.FILLING
    else -> GroupState.OPEN
}
currentList[index] = deal.copy(currentMembers = newMemberCount,
    joinedUserIds = deal.joinedUserIds + currentUser.id, currentTierIndex =
newTierIndex,
    currentUnitPrice = newPrice, state = newState, unitsReserved =
deal.unitsReserved + 1)
_deals.value = currentList
```

Listing 7.6: Core of joinGroup() in GroupDealRepository (abridged)

Two behaviours are worth noting. First, only three of the ten `GroupState` values are computed automatically; the remaining states belong to the proposed auction, payment and delivery workflow. Second, `leaveGroup` recomputes price and tier but does not recompute the state, so a group that falls below 60% of its target keeps the FILLING label until the next join. Both are recorded in Chapter 9.

7.11 Demand Requests

`createRequest` publishes a request for the current locality with the creator's vote already counted (votes = 1) and a threshold of 15. `voteForRequest` adds a vote only if the user's identifier is not already in `votedUserIds`, which enforces one vote per user. The request lifecycle is shown in Figure 7.3; conversion of a request that reaches its threshold into a group deal is proposed.

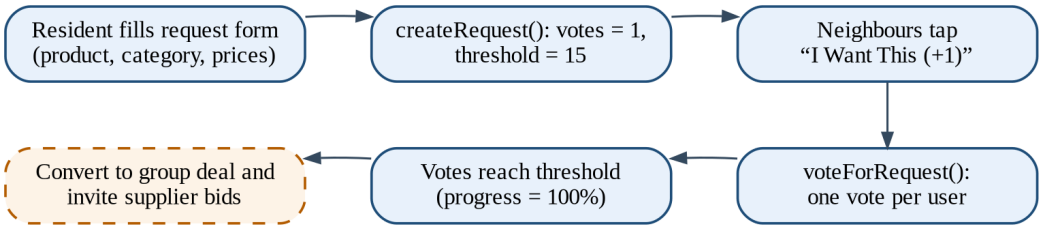


Figure 7.3: Demand Request Lifecycle

7.12 Supplier Bidding

The prototype contains the data structures for supplier bidding — `SupplierBid` with supplier name and score, bid amount per unit, promised delivery days, selection and

verification flags — and the corresponding group states. Two seeded deals carry a winning bid, which is displayed on the deal and used as the supplier name in orders. The logic for opening an auction, accepting and validating bids and selecting the winner is proposed and will run in a Cloud Function (Section 5.10) so that no client can manipulate the result.

7.13 Orders

`placeGroupOrder` creates an `Order` from the current deal: total = unit price × quantity, market total = MRP × quantity, and saved amount = the difference (never negative). The order receives a generated order number in the form GD-XXXXXX, the status CONFIRMED, the locality as delivery area, the winning supplier (or a default supplier) and four tracking steps: Group Target Reached, Order Confirmed, Supplier Dispatched and Locality Delivery. In the prototype the checkout button is available regardless of whether the group has reached its target, and the quantity is fixed at one unit in the UI; the proposed system allows checkout only after the auction and payment window open.

7.14 Payments

Payment processing is not implemented in the prototype: the “Confirm & Pay” action creates an order record directly. The proposed Razorpay integration (Section 5.11) separates responsibilities so that the client never computes the payable amount and never holds the secret key: a Cloud Function creates the Razorpay order from the server-side price, the app opens Razorpay Checkout, and a second Cloud Function verifies the HMAC signature before marking the order as paid.

7.15 Notifications

The prototype implements in-app notifications through the repository’s `SharedFlow<LocalityEvent>`. Events are emitted when a user joins a group (“You joined … Price is now ₹949”), votes for a request, publishes a request or changes locality, and the main shell displays them as a live banner. Push notifications through FCM are proposed (Section 5.12).

7.16 Validation

- **Membership validation:** a user cannot join the same group twice or leave a group they have not joined.
- **Vote validation:** one vote per user per request.
- **Pincode validation:** only six-digit numeric input is accepted by the fallback lookup.
- **Count bounds:** member counts and reserved units never fall below zero.

- **Tier validation (defined, not yet enforced):** `validateTiers` rejects schedules whose thresholds do not strictly increase or whose prices do not strictly decrease; it will run when suppliers create deals in the complete system.
- **Onboarding defaults:** blank names and phone numbers are replaced with safe defaults.

7.17 Error Handling

The prototype handles invalid operations defensively rather than by throwing exceptions: repository functions return early when there is no signed-in user or when a deal or request identifier is not found, and the Group Detail screen shows “Deal not found” if the selected deal no longer exists. Image loading failures are handled by Coil without affecting the rest of the screen. In the complete system, network and payment errors will be modelled as explicit error states in each screen’s `UiState`, with retry actions.

7.18 Security Considerations

- **No secrets in the client.** The Secrets Gradle plugin keeps API keys out of source control; the Razorpay secret key will exist only in Cloud Functions configuration.
- **Server-authoritative pricing.** In the complete system the price is computed by a Cloud Function inside a Firestore transaction, so a modified client cannot set its own price.
- **Firestore security rules.** Users may write only their own membership, vote and order documents; deal prices and auction results are writable only by Cloud Functions.
- **App Check.** Requests from non-genuine app instances are rejected by the backend [30].
- **Payment verification.** Orders are marked paid only after server-side signature verification.
- **Prototype limitation.** The current sign-in is simulated and performs no credential check; it must not be used with real user data.

CHAPTER 8

MOBILE APPLICATION IMPLEMENTATION

8.1 Application Setup and Configuration

The GroupDeal application is a single-activity Compose application. On launch, `MainActivity` enables edge-to-edge display, applies `GroupDealTheme` and hosts `AppNavigation`. The theme defines an emerald primary colour (#0F9D58) with darker emerald variants for headers, and orange and amber accents for deal and savings badges. Typography and shapes follow Material 3 defaults with custom weights for prices.

The figures in this chapter are **layout schematics reconstructed from the Compose source code**. Each schematic shows the components, labels, button texts and sample values exactly as they are defined in the code and seed data (for example, the Koramangala locality, trust score 780 and the ghee deal at ₹949). They are drawn to document the structure of each screen; they are not device screenshots.

8.2 Navigation Design

Navigation is driven by state rather than by a route graph (Figure 8.1). `AppNavigation` observes the authentication state and shows the Loading indicator, the Login screen, the Onboarding screen or the main shell. The main shell is a Material 3 `Scaffold` whose `NavigationBar` has five tabs: **Deals**, **Demands**, **My Groups**, **Orders** and **Profile**. Selecting a deal on the Discover or My Groups tab opens the Group Detail screen as an overlay, and the locality chip opens the Locality Picker as a modal bottom sheet. Signing out from Profile sets the state to Unauthenticated, which returns the user to Login.

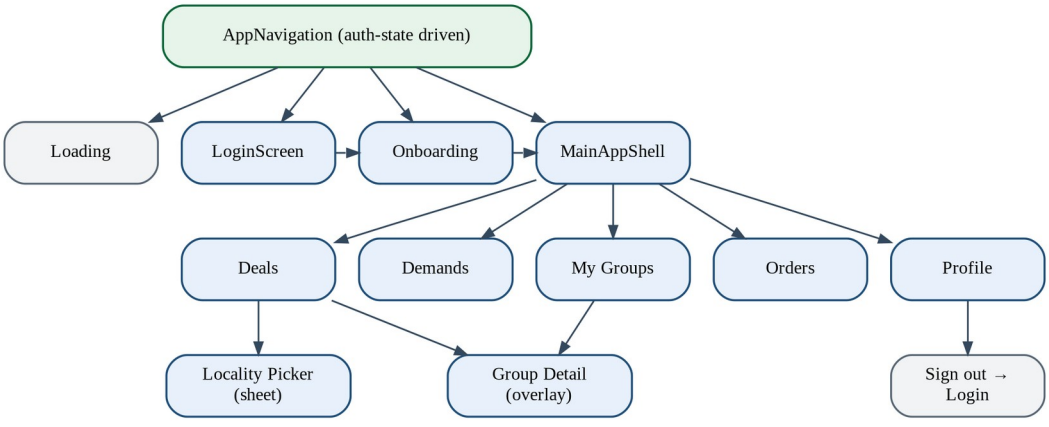


Figure 8.1: Navigation Map of the GroupDeal Application

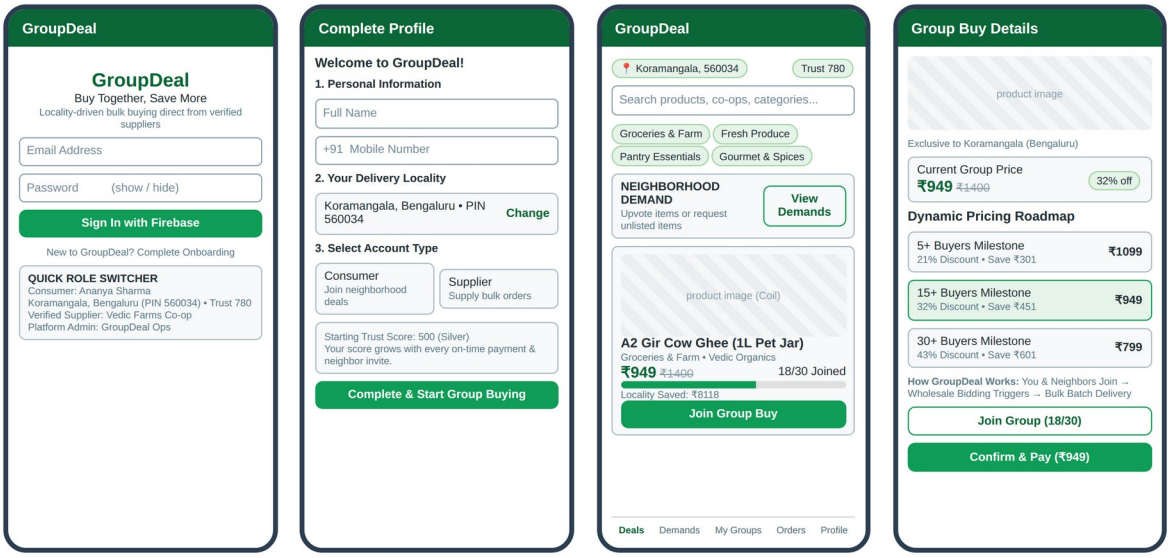
Table 8.1: Application Screens and Their Functionalities

Screen	Source file	Purpose	Main state used
Login	auth/LoginScreen.kt	Email/password sign-in (simulated), onboarding entry, demo role personas	authState
Onboarding	auth/OnboardingScreen.kt	Name, phone, delivery locality and account type for new users	authState, locality
Locality Picker	auth/LocalityPickerModal.kt	Search localities or enter a pincode; select delivery locality	currentLocality
Discover (Deals)	discovery/ DiscoverHomeScreen.kt	Deal feed, search, category chips, demand banner	deals, currentLocality
Group Detail	detail/GroupDetailScreen.kt	Pricing roadmap, join/leave, confirm and pay	deals
Demands	requests/ RequestGroupScreen.kt	Locality requests, upvotes, request form	requests
My Groups	mygroups/MyGroupsScreen.kt	Deals the user has joined, checkout	deals, authState
Orders	orders/OrdersScreen.kt	Order list, savings and tracking timeline	orders
Profile	profile/ProfileScreen.kt	Trust score, statistics, edit profile, role switch, sign out	authState

8.3 Authentication and Onboarding Screens

Login. The Login screen (Figure 8.2a) shows the GroupDeal title and tagline, email and password fields with a visibility toggle, and a primary button labelled “Sign In with Firebase”. In the prototype this button calls `loginWithEmail`, which waits 400 ms to simulate a network round trip and then signs in the demonstration user Ananya Sharma (Koramangala, trust score 780); no Firebase call is made. An email address containing “new” simulates a first-time user and opens onboarding. A **Quick Role Switcher** lists demonstration personas for a consumer, a verified supplier and a platform administrator.

Onboarding. The Onboarding screen (Figure 8.2b) collects the user’s full name and mobile number, the delivery locality (with a Change action that opens the Locality Picker) and the account type (Consumer or Supplier). It explains that the starting trust score is 500 (Silver) and grows with on-time payments and invitations. Completing onboarding creates the user profile and sets the current locality.



(a) Login (b) Onboarding (c) Discover (Deals) (d) Group Detail

Figure 8.2: Layout Schematics of Login, Onboarding, Discover and Group Detail Screens (reconstructed from source, not device screenshots)

8.4 Locality Selection

The Locality Picker is a Material 3 modal bottom sheet titled “Select Locality” with the hint “Group buying deals & suppliers are locality-specific”. A text field accepts a six-digit pincode or an area name. When the field is empty the sheet lists “Popular Hubs in India” — the 15 seeded localities with city, state, pincode and number of active deals; otherwise it shows search results from `LocalityService`. Selecting a locality calls `setLocality`, which updates the current locality and the user’s profile and emits a live event.

8.5 Home / Discovery Screen

The Discover screen (Figure 8.2c) is the home tab. Its header shows the current locality and the user’s trust score. Below it are a search field (“Search products, co-ops, categories...”), category filter chips (Groceries & Farm, Fresh Produce, Pantry Essentials and Gourmet & Spices) and a **Neighborhood Demand** banner linking to the Demands tab. Each deal is shown as a `GroupDealCard` with the product image, title, category and brand, current group price with the struck-through MRP, a progress bar with the member count (18/30 Joined), the total locality savings and a Join Group Buy button that changes to “Joined ✓ (Leave)” after joining.

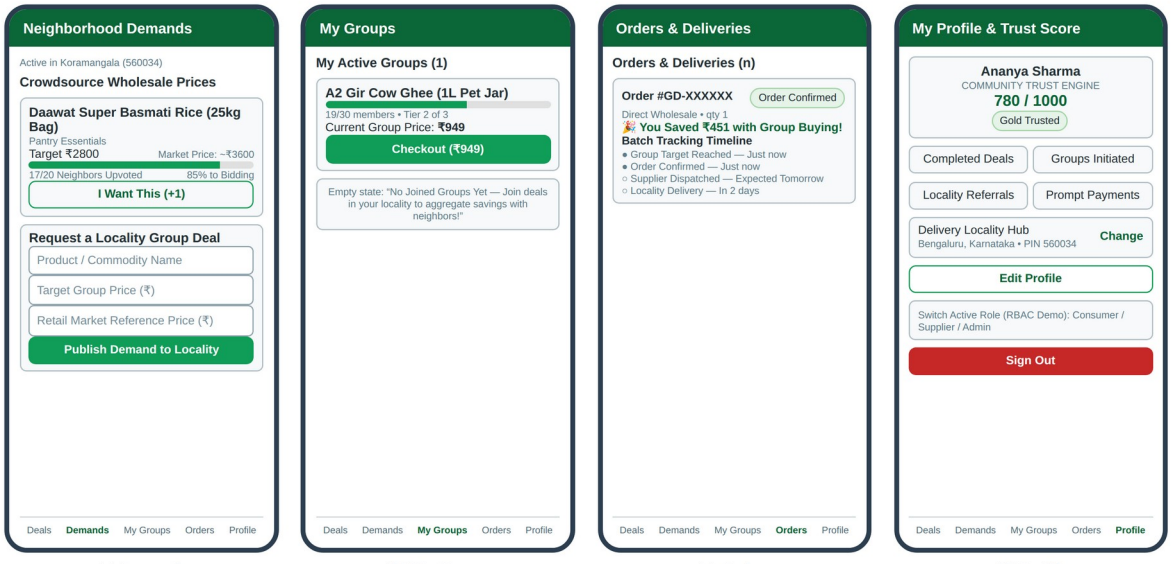
The deal list is filtered by the search text and the selected category. All seeded deals belong to Koramangala; filtering the feed by the selected locality is part of the proposed system (Section 7.8).

8.6 Group Detail and Pricing Screen

The Group Detail screen (Figure 8.2d) is the most important screen for the pricing concept. It shows the product image, the locality to which the deal is exclusive, the **Current Group Price** with MRP and discount, and a **Dynamic Pricing Roadmap** in which every tier is presented as a milestone — for the ghee deal: 5+ buyers at ₹1,099 (21% discount, save ₹301), 15+ buyers at ₹949 (32%, save ₹451) and 30+ buyers at ₹799 (43%, save ₹601), with the active tier highlighted. A "How GroupDeal Works" section explains the three steps (You & Neighbors Join → Wholesale Bidding Triggers → Bulk Batch Delivery). The Join Group button shows the live count, and the Confirm & Pay button shows the current total. When the user joins, the price, progress, highlighted tier and button labels update immediately because the screen observes the deals StateFlow.

8.7 Demand Requests and Upvotes

The Demands tab (Figure 8.3a) lists the requests of the current locality under the heading "Neighborhood Demands". Each request card shows the product, category, requester, target price, market reference price, a progress bar with "17/20 Neighbors Upvoted" and "85% to Bidding", and an "I Want This (+1)" button that changes to "Upvoted ✓" after voting. The Request Product form collects the product name, category, target group price and retail market reference price and publishes the request with "Publish Demand to Locality".



(a) Demands (b) My Groups (c) Orders (d) Profile

Figure 8.3: Layout Schematics of Demands, My Groups, Orders and Profile Screens (reconstructed from source, not device screenshots)

8.8 My Groups Screen

My Groups (Figure 8.3b) lists the deals that the signed-in user has joined, under "My Active Groups (n)". Each entry shows the live progress, the current group price and a Checkout

button. When the user has not joined any deal, the empty state “No Joined Groups Yet” invites the user to join deals in the locality.

8.9 Orders and Tracking Screen

The Orders tab (Figure 8.3c) lists orders under “Orders & Deliveries (n)”. Each order shows the order number, status badge, supplier, quantity, the amount saved (“You Saved ₹451 with Group Buying!” for the ghee deal at ₹949 against an MRP of ₹1,400) and a **Batch Tracking Timeline** with four steps. In the prototype the steps and their times are fixed when the order is created; live status updates are part of the proposed system.

8.10 Profile and Trust Screen

The Profile tab (Figure 8.3d) shows the user’s name, the **Community Trust Engine** score (for example, 780 / 1000) with its tier (Gold Trusted), statistics for completed deals, groups initiated, locality referrals and prompt payments, and the delivery locality hub with a Change action. The user can edit the name, phone and locality, switch the active role for demonstration purposes (“Switch Active Role (RBAC Demo)”) and sign out. The role switch changes only the displayed role; supplier and administrator functions are proposed.

8.11 Input Forms and Validation

Table 8.2: Input Forms and Validation Rules

Form	Fields	Validation / Behaviour
Login	Email, password	Simulated sign-in; email containing “new” routes to onboarding
Onboarding	Full name, mobile number, locality, account type	Blank name or phone replaced with defaults; locality required via picker
Locality Picker	Pincode or area text	Exact match, six-digit fallback or no result
Discover search	Search text, category chip	Case-insensitive match on deal fields; chip filters by category
Request form	Product name, category, target price, market price	Creates request with 1 vote and threshold 15
Edit profile	Full name, phone, locality	Updates profile and current locality

8.12 Module Integration

The modules are integrated through the shared repository rather than through direct calls between screens. The Discover, Group Detail and My Groups screens all read the same deals flow, so joining on one screen is reflected on the others without any additional code. The Demands tab reads the requests flow, Orders reads the orders flow, and every screen can read the authentication state for the current user. This integration style is the reason the prototype

behaves consistently across tabs, and it is also what allows the backend to be introduced later behind the same flows.

SIRAGU

CHAPTER 9

TESTING AND RESULTS

9.1 Testing Strategy

Testing was planned at four levels, following standard software-testing practice [1], [2]: unit tests of pure logic, functional tests of each user-visible feature, integration tests of the interaction between screens, repository and engine, and UI tests of the Compose screens. The workflow is shown in Figure 9.1.

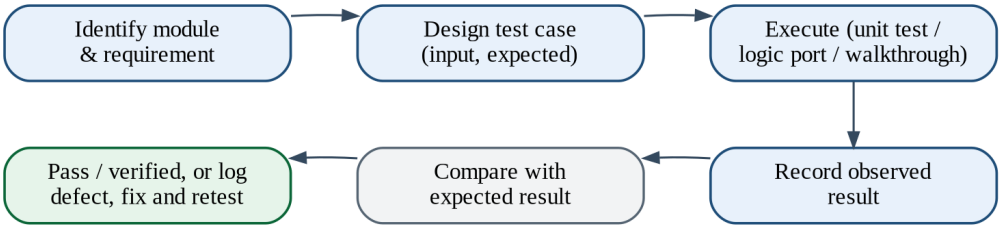


Figure 9.1: Application Testing Workflow

Three kinds of evidence are used in this chapter, and each test case states which one applies:

- **Unit test (source):** an assertion that exists in the project's JUnit/Robolectric test classes and is executed with `./gradlew testDebugUnitTest`.
- **Logic port run:** the pricing, trust-tier and pincode functions were ported line by line to Python and executed with the test inputs (Appendix A.7). This independently confirms the expected value of every assertion and exercises additional boundary cases.
- **Code walkthrough:** the behaviour was traced through the Kotlin source code, statement by statement, for the given input.

Features of the proposed system that are not implemented are covered by **planned** test cases, which define the tests to be executed once those features are built. No execution time, coverage percentage or performance figure is reported, because none was measured.

9.2 Unit Testing

The project contains the unit tests listed in Table 9.1. The pricing, locality and trust-tier tests directly exercise GroupDeal's domain logic.

Table 9.1: Unit Test Cases in the Project Source

Test ID	Test method	Assertions	Expected	Observed (evidence)	Status
UT01	read string from context	app_name resource	"GroupDeal"	strings.xml defines "GroupDeal" (walkthrough)	Pass

Test ID	Test method	Assertions	Expected	Observed (evidence)	Status
UT02	pricing engine calculates correct dynamic discount	base 1200; tiers 5/15/30 → 1000/800/600; n = 0, 3, 5, 18, 35	1200, 1200, 1000, 800, 600	1200, 1200, 1000, 800, 600 (logic port)	Pass
UT03	locality service resolves pincode and searches	560034 lookup; search “Mumbai”	Koramangala, Bengaluru; non-empty list	Koramangala; 3 Mumbai localities (port / walkthrough)	Pass
UT04	trust score tier mapping	scores 250, 500, 780, 950	Bronze, Silver, Gold, Platinum	Bronze, Silver, Gold, Platinum (logic port)	Pass
UT05	addition_isCorrect (template)	2 + 2	4	4	Pass

9.3 Functional Testing

Table 9.2: Functional Test Cases

Test ID	Module	Input / Action	Expected Result	Observed Result	Evidence	Status
FT01	Sign in	Email user@test.in, any password	Signed in as demo consumer	Authenticated after simulated 400 ms delay	Walkthrough	Pass
FT02	Onboarding	Email newuser@test.in	Onboarding screen shown	State = OnboardingRequired	Walkthrough	Pass
FT03	Locality	Pincode 560034	Koramangala selected	Exact match returned	Port	Pass
FT04	Locality	Pincode 600119	Chennai locality created	Sector / Ward 600119, Chennai	Port	Pass
FT05	Locality	Input 99999 (5 digits)	No locality	null returned	Port	Pass
FT06	Discover	Category chip “Fresh Produce”	Only mango deal listed	Filter by category applied	Walkthrough	Pass
FT07	Join group	Join ghee deal at 18/30	19/30, ₹949, FILLING	19 members, ₹949, FILLING	Port + walkthrough	Pass
FT08	Join group	Join same deal again	No change	Early return on duplicate id	Walkthrough	Pass
FT09	Leave group	Leave after joining	18/30, ₹949	Count and price recomputed	Walkthrough	Pass
FT10	Request	Publish “Coconut Oil” request	Request with 1 vote, threshold 15	votes = 1, threshold = 15	Walkthrough	Pass
FT11	Upvote	Vote on 17/20 request	18/20	Vote added; user id stored	Walkthrough	Pass
FT12	Upvote	Vote again on same request	No change	Early return on duplicate id	Walkthrough	Pass
FT13	Checkout	Confirm & Pay on ghee at ₹949	Order total ₹949, saved ₹451	Order CONFIRMED, saved 451	Walkthrough	Pass

Test ID	Module	Input / Action	Expected Result	Observed Result	Evidence	Status
FT14	Profile	Edit name and locality	Profile and locality updated	authState and currentLocality updated	Walkthrough	Pass
FT15	Sign out	Tap Sign Out	Login screen shown	State = Unauthenticated	Walkthrough	Pass

9.4 Integration Testing

Integration tests check that the screens, repository and engine work together. Because all screens observe the same StateFlow, a change made on one screen must appear on the others.

Table 9.3: Integration Test Cases

Test ID	Scenario	Expected Result	Evidence	Status
IT01	Join on Group Detail, then open Discover and My Groups	Same count, price and joined state on all three screens	Walkthrough (single deals flow)	Pass
IT02	Join a deal, then check out from My Groups	Order uses the current group price	Walkthrough	Pass
IT03	Change locality from Profile	Discover header and new requests use new locality	Walkthrough	Pass
IT04	Vote on a request	Live banner shows vote count	Walkthrough (SharedFlow)	Pass
IT05	Join with Firestore transaction and Cloud Function	All devices receive the same new price	Planned (proposed backend)	Planned

9.5 UI Testing

The Compose UI testing library is declared as a test dependency and the project contains a Roborazzi screenshot-test template. The following Compose UI tests are defined for the next iteration; they are listed as planned because they have not yet been written.

Table 9.4: Planned Compose UI Test Cases

Test ID	Screen	Action	Expected Result	Status
UI01	Discover	Type “Ghee” in search	Only matching card displayed	Planned
UI02	Group Detail	Tap Join Group	Button changes; count increments	Planned
UI03	Group Detail	Render roadmap	Three milestones; active tier highlighted	Planned
UI04	Demands	Tap I Want This (+1)	Label changes to Upvoted ✓	Planned
UI05	All tabs	Tap each navigation item	Correct screen displayed	Planned
UI06	Group Detail	Screenshot (Roborazzi)	Matches recorded baseline	Planned

9.6 Pricing Logic Testing

The pricing engine was tested at every tier boundary of the ghee deal (base ₹1,250; tiers 5 → ₹1,099, 15 → ₹949, 30 → ₹799) and for invalid or unusual inputs. Table 9.5 lists the observed results from the logic port run.

Table 9.5: Pricing Engine Boundary Tests (Ghee Deal)

Test ID	Members (n)	Unit price	Active tier	Members to next tier	Status
PT01	0	₹1,250	none (base)	5	Pass
PT02	1	₹1,250	none (base)	4	Pass
PT03	4	₹1,250	none (base)	1	Pass
PT04	5	₹1,099	tier 1	10	Pass
PT05	14	₹1,099	tier 1	1	Pass
PT06	15	₹949	tier 2	15	Pass
PT07	18	₹949	tier 2	12	Pass
PT08	29	₹949	tier 2	1	Pass
PT09	30	₹799	tier 3	— (highest tier)	Pass
PT10	35	₹799	tier 3	— (highest tier)	Pass
PT11	18, tiers given out of order	₹949	tier 2	12	Pass
PT12	18, empty tier list	₹1,250	none (base)	—	Pass
PT13	−3 (invalid)	₹1,250	none (base)	—	Pass

Table 9.6: Tier Validation Tests (validateTiers)

Test ID	Tier schedule	Expected	Observed	Status
VT01	5 → 1099, 15 → 949, 30 → 799	valid	valid	Pass
VT02	15 → 949, 5 → 1099 (unsorted)	invalid	invalid	Pass
VT03	5 → 900, 15 → 950 (price rises)	invalid	invalid	Pass
VT04	5 → 1000, 5 → 900 (duplicate threshold)	invalid	invalid	Pass
VT05	empty list	invalid	invalid	Pass

9.7 Group Membership Testing

Table 9.7: Group Membership and State Tests (Ghee Deal, Target 30)

Test ID	Action	Members	Price	State	Status
GM01	Seed value	18	₹949	FILLING	Pass
GM02	One member joins	19	₹949	FILLING	Pass

Test ID	Action	Members	Price	State	Status
GM03	Members join up to 29	29	₹949	FILLING	Pass
GM04	Thirtieth member joins	30	₹799	TARGET_REACHED	Pass
GM05	One member leaves at 18/30 (17 < 0.6 × 30)	17	₹949	FILLING (expected OPEN)	Defect D1
GM06	membersToNextTier with 35 of 30 members	35	₹799	value = -5	Defect D2

Two defects were identified. **D1:** `leaveGroup` recomputes the price and tier but not the state, so a group that drops to 17 of 30 members (below the 60% FILLING threshold of 18) still shows FILLING. **D2:** the derived property `GroupDeal.membersToNextTier` falls back to `targetMembers − currentMembers` at the highest tier, which becomes negative (−5) if more members than the target join; the `PricingEngine.membersToNextTier` function correctly returns no next tier in the same situation. Both defects have straightforward fixes — recomputing the state in `leaveGroup` and bounding the derived value at zero — and are scheduled for the next iteration.

9.8 Demand Request Testing

Table 9.8: Demand Request Test Cases

Test ID	Input / Action	Expected Result	Observed Result	Status
DR01	Create request (signed in)	votes = 1, threshold = 15, current locality	As expected	Pass
DR02	Create request (signed out)	No request created	Early return	Pass
DR03	Vote on Basmati rice (17/20)	18/20; progress 0.90	As expected	Pass
DR04	Second vote by same user	Ignored	Early return on duplicate id	Pass
DR05	Progress of 17/20	85% to bidding	0.85 (coerced to 0–1)	Pass
DR06	Request reaches threshold	Converted to deal	Not implemented	Planned

9.9 Supplier Bid Testing

Table 9.9: Supplier Bid Test Cases

Test ID	Scenario	Expected Result	Status
SB01	Display seeded winning bid (ghee)	Supplier Vedic Farms Co-op, ₹949 per unit shown	Pass (walkthrough)
SB02	Order for deal with winning bid	Order supplier = winning bid supplier	Pass (walkthrough)
SB03	Bid above ceiling (current tier price)	Bid rejected	Planned
SB04	Bid after deadline	Bid rejected	Planned

Test ID	Scenario	Expected Result	Status
SB05	Two bids with equal price	Higher supplier score wins	Planned
SB06	No valid bids by deadline	Fallback supplier or EXPIRED	Planned

9.10 Authentication Testing

Authentication in the prototype is simulated, so testing confirms the routing logic rather than credential verification (FT01, FT02, FT15). The planned tests for Firebase Authentication are: valid phone OTP signs in; wrong OTP is rejected; an expired session returns to Login; a supplier account cannot access customer-only writes (enforced by Firestore rules); and requests without a valid App Check token are rejected.

9.11 Payment Testing

The prototype's order calculation was verified (FT13): for the ghee deal at ₹949, quantity 1, the order total is ₹949, the market total is ₹1,400 and the saved amount is ₹451. Payment gateway tests are planned for Razorpay's test mode: successful payment with valid signature marks the order as paid; a tampered signature is rejected by the Cloud Function; a cancelled checkout leaves the order unpaid; and a duplicate callback does not create a second payment.

9.12 Notification Testing

In-app notifications were verified by walkthrough: `joinGroup`, `voteForRequest`, `createRequest` and `setLocality` each emit a `LocalityEvent`, and the SharedFlow buffer holds up to ten events. Planned FCM tests are: all members of a deal receive a price-drop notification when a tier is crossed; the auction winner notification reaches members; and notifications are received when the app is in the background.

9.13 Compatibility Testing

The build targets API 36 with a minimum of API 24, and the Robolectric tests are configured for SDK 36 (the screenshot template uses the Pixel 8 device profile). The planned compatibility matrix covers emulators at API 24 (Android 7.0), API 30 (Android 11), API 34 (Android 14) and API 36, in portrait orientation, on a compact phone and a large phone, checking layout, edge-to-edge insets, the bottom sheet and image loading.

9.14 Results

The main result of the project is a working Android prototype in which the locality-based group-buying flow runs end to end on seed data. Table 9.10 shows the output of the pricing engine for the four seeded deals, reproduced by the logic port; the values are identical to the prices stored in the seed data. These are demonstration values from seed data, not results of real transactions.

Table 9.10: Pricing Engine Output for the Four Seeded Deals

Deal	Members	Tier	Group price	MRP	Saving	Next tier	Locality saved
A2 Gir Cow Ghee 1L	18/30	2 of 3	₹949	₹1,400	32%	+12	₹8,118
Alphonso Mangoes dozen	34/50	2 of 3	₹1,599	₹2,500	36%	+16	₹30,634
Groundnut Oil 5L	40/40	3 of 3	₹319	₹600	46%	target reached	₹11,240
Kashmir Saffron 5g	14/20	2 of 3	₹2,750	₹4,000	31%	+6	₹17,500

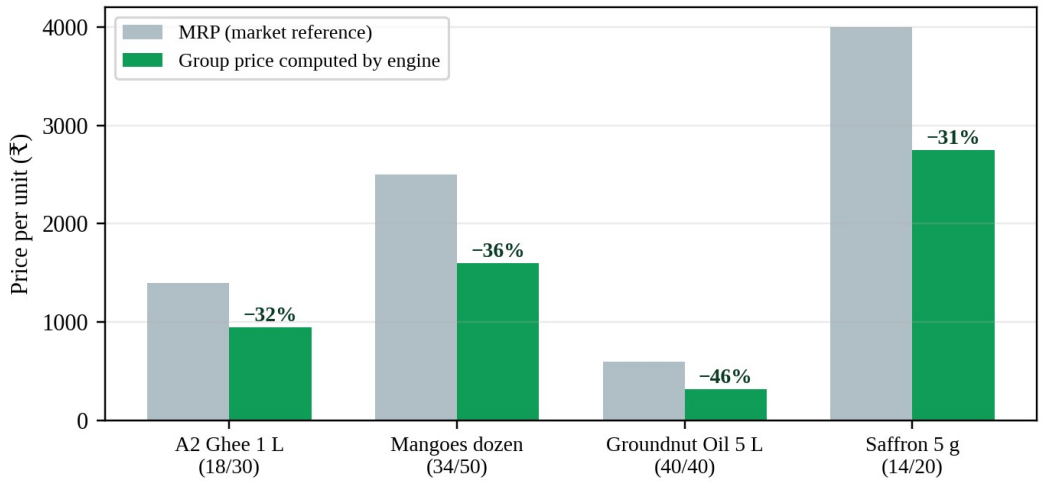


Figure 9.2: Market Price versus Engine-Computed Group Price for the Seeded Deals

Table 9.11: Summary of Verified Prototype Capabilities

Capability	Result
Locality selection	Exact pincode match, six-digit fallback and search verified
Tier-based dynamic pricing	All unit-test cases and 13 boundary/edge cases agree with expected values
Group membership	Join, duplicate prevention and leave verified; defects D1 and D2 identified
Demand requests	Creation, one-vote-per-user and progress verified
Orders	Totals, savings and tracking steps verified
Consistency across screens	Guaranteed by the single StateFlow source of truth
Backend, bidding, payment, push	Designed; planned test cases defined

9.15 Output Screenshots and Evidence

Listing 9.1 reproduces the console output of the logic port run described in Section 9.1. Layout schematics of every screen are given in Chapter 8 (Figures 8.2 and 8.3).

```
$ python3 port_verify.py
unit_test      n=0 -> 1200.0 | n=3 -> 1200.0 | n=5 -> 1000.0 | n=18 -> 800.0 | n=35 -> 600.0
ghee n=4      -> price 1250.0, tierIndex -1, toNext 1
ghee n=15     -> price 949.0,  tierIndex 1,  toNext 15
ghee n=18     -> price 949.0,  tierIndex 1,  toNext 12
ghee n=30     -> price 799.0,  tierIndex 2,  toNext None
unsorted_input(18) -> 949.0 | empty_tiers -> 1250.0 | negative_n -> 1250.0
validate: ghee=True unsorted=False price_up=False duplicate=False empty=False
trust: 250=BRONZE 500=SILVER 780=GOLD 950=PLATINUM
pincode: 560034=Koramangala 600119='Sector / Ward 600119 (Chennai)' 99999=None
seed A2 Gir Cow Ghee 1L       n=18/30 price=949.0  tier=2 save=32% state=FILLING
seed Alphonso Mangoes dozen   n=34/50 price=1599.0 tier=2 save=36% state=FILLING
seed Groundnut Oil 5L         n=40/40 price=319.0  tier=3 save=46% state=TARGET_REACHED
seed Kashmir Saffron 5g       n=14/20 price=2750.0 tier=2 save=31% state=FILLING
derived membersToNextTier(35 of 30) = -5   <- defect D2
```

Listing 9.1: Output of the Pricing Logic Port Run

SIRAGU

CHAPTER 10
APPLICATIONS

10.1 Application Areas

GroupDeal can be applied wherever several buyers in the same area repeatedly need the same product and a supplier can offer a lower price for a larger, single delivery. The main application areas are:

- **Neighbourhood grocery buying:** staples such as ghee, cooking oil, rice, pulses and spices bought in bulk by residents of one locality — the use case represented by the seed data.
- **Apartment communities:** a residents' association can use one locality group to buy for all flats and receive a single delivery at the gate.
- **Student communities and hostels:** students sharing accommodation can pool purchases of essentials and snacks.
- **Seasonal and fresh produce:** fruits such as Alphonso mangoes bought directly from grower collectives during the season.
- **Agricultural produce and farm cooperatives:** farmers' cooperatives can sell directly to urban localities, as in the seeded Vedic Farms Co-op example.
- **Household essentials:** cleaning supplies, water cans and cooking gas accessories with predictable repeat demand.
- **Cooperative purchasing:** workplaces and clubs that already organise collective purchases informally.

10.2 Real-World Applications

Consider a gated community in Perungudi, Chennai. A resident notices that cold-pressed groundnut oil is expensive at the local store and publishes a demand request with a target group price. Over a week, neighbours upvote the request until it reaches its threshold. In the complete system, the request becomes a group deal with a tier schedule; as residents join, the price moves down the tiers and every member sees the saving and the number of members still needed. When the target is reached, verified suppliers bid, the lowest valid bid wins, members pay through Razorpay and the order arrives as one batch at the community's collection point.

The same flow serves a supplier's perspective: instead of advertising to individual customers, a regional brand can respond to demand that has already been aggregated and priced, and deliver to one address per locality. For the community, the transparent tier roadmap replaces informal negotiation by one organiser.

10.3 Target User Groups

Table 10.1: Target User Groups and Benefits

User group	Typical need	Benefit from GroupDeal
Households	Regular purchase of staples	Bulk prices without buying bulk quantities alone
Residents' associations	Coordinating purchases for many flats	Automatic member tracking and transparent pricing
Students and young professionals	Budget-conscious buying	Lower unit prices through pooling
Farm cooperatives and wholesalers	Reaching urban buyers efficiently	Aggregated, locality-scoped orders (proposed bidding)
Local retailers	Predictable demand	Visibility of upvoted local demand

10.4 Potential Industry Applications

- **Quick-commerce and grocery platforms** could add locality group pricing to reduce last-mile cost per order.
- **Agricultural marketing** — farmer producer organisations could use aggregated demand to plan harvest and logistics.
- **Corporate and campus procurement** — recurring purchases for canteens, hostels and offices.
- **Community-driven brands** could launch products only when enough local demand has been demonstrated through requests.

10.5 Adapting GroupDeal to New Domains

Adapting GroupDeal to another domain requires only configuration of the data model: the product categories, the tier schedule of each deal and the locality definitions. The pricing engine is independent of the product type, and the demand-request mechanism works for any item. Domains with larger geographical units (for example, district-level agricultural inputs) can replace pincode localities with larger service areas without changing the pricing logic.

CHAPTER 11

ADVANTAGES AND LIMITATIONS

11.1 Advantages

- **Locality-aware demand:** buying groups are scoped to a locality, which matches shared delivery and local trust.
- **Group purchasing for individuals:** each household gets the bulk tier price while buying a single unit.
- **Transparent tier-based pricing:** the price is a published, deterministic function of the member count, explained on the roadmap.
- **Visible progress:** the number of members needed for the next tier encourages members to invite neighbours.
- **Community participation:** requests and upvotes let residents shape which deals are offered.
- **Supplier competition (proposed):** reverse auctions let suppliers compete only for proven demand.
- **Centralised order flow:** membership, checkout and tracking live in one application.
- **Trust mechanism:** trust scores and tiers reward reliable members.
- **Consistent state:** a single source of truth keeps every screen in agreement.
- **Extensible architecture:** the repository interface allows the Firebase backend to be introduced without rewriting the screens.

11.2 Limitations

11.2.1 Limitations of the Current Prototype

- **No persistent or shared data:** all data is held in memory and resets on restart; users on different devices do not see each other's actions.
- **Simulated authentication:** the sign-in button does not contact Firebase and performs no credential check.
- **Payment not processed:** checkout creates an order record without payment, and is allowed before the group target is reached.
- **Locality not used to filter the feed:** the selected locality is displayed and stored, but the deal feed is filtered only by search and category.
- **Partial state machine:** only OPEN, FILLING and TARGET_REACHED are computed; `leaveGroup` does not recompute the state (defect D1).
- **Derived next-tier value can be negative** when members exceed the target (defect D2).
- **Tier validation not enforced** when deals are created.

- **Static trust score:** the score does not change with behaviour.
- **Pincode fallback accepts any six-digit number,** including numbers that are not valid Indian pincodes.
- **Declared but unused dependencies** (Retrofit, Room, Navigation Compose, Firebase libraries) increase build size until they are used.

11.2.2 Limitations Inherent to Group Buying

- **User participation:** a deal succeeds only if enough neighbours join before the deadline.
- **Supplier participation:** reverse auctions require several verified suppliers per locality.
- **Locality density:** sparsely populated areas may never reach the thresholds.
- **Internet dependency:** the complete system requires connectivity for real-time updates and payment.
- **External service dependency:** payment, maps and notifications depend on third-party availability and pricing.
- **Logistics:** batch delivery requires a collection point and last-mile coordination outside the app.
- **Fraud and abuse:** fake accounts could inflate member counts; OTP sign-in, App Check and payment-before-dispatch are required.
- **Payment dependency:** members who join but do not pay can delay the whole group.
- **Data management and scalability:** many localities and deals require careful indexing and cost control in Firestore.

11.3 Security Considerations

The prototype is a single-device demonstration and must not be used with real personal or payment data. The complete system relies on four security principles: authenticate every user (Firebase Authentication with OTP); authorise every write with Firestore security rules, allowing users to modify only their own memberships, votes and orders; keep all trust-sensitive computation on the server (tier price, auction result, payment amount and signature verification in Cloud Functions); and reject non-genuine clients with App Check. API keys are managed through the Secrets Gradle plugin, and payment secrets never leave the server.

11.4 Performance Constraints

Performance was not measured in this PBL, so no figures are reported. Qualitatively, the pricing computation is constant time for the three-tier deals used, and the StateFlow model updates only the composables that read the changed state. In the prototype every change

replaces the whole deal list, which is inexpensive for four deals but should be replaced by per-deal documents and paged queries when the number of deals grows. In the complete system the main constraints are network latency for Firestore updates, Cloud Function cold starts, the cost of real-time listeners per user, and image sizes, which Coil mitigates by caching.

SIRAGU

CHAPTER 12

CONCLUSION AND FUTURE SCOPE

12.1 Conclusion

GroupDeal set out to make bulk pricing reachable for individual households by letting neighbours buy together. The project delivered a working native Android prototype, written in Kotlin with Jetpack Compose and Material 3, in which the complete locality-based group-buying flow runs end to end: a user selects a locality, discovers deals, joins a group, and immediately sees the tier price, the saving against MRP and the number of neighbours needed for the next price drop; unmet demand is captured through requests and upvotes; orders are recorded with a tracking timeline; and members carry a trust score.

At the core of the system is a transparent, rule-based, tier-based pricing engine. It was verified against the project's unit-test cases, at every tier boundary and for invalid inputs, and it reproduces the prices of all four seeded deals, which range from 31% to 46% below MRP. Testing also identified two defects in the membership logic, which are documented with their fixes.

Equally important, the project defined the complete system around the prototype: a Firebase backend with server-authoritative pricing, supplier reverse auctions, Razorpay payments and push notifications. The report consistently separates what is implemented from what is proposed, so the prototype provides an honest and tested foundation for the next phase.

12.2 Project Outcomes

- A native Android application with nine screens and sheets and a five-tab navigation shell.
- A tier-based dynamic pricing engine with validation and next-tier computation.
- Locality selection with pincode lookup across 15 localities in 8 cities.
- Group membership with duplicate prevention and automatic price and state updates.
- Demand requests with one-vote-per-user upvoting.
- Order records with savings and a batch-tracking timeline; trust score and tiers.
- A single-source-of-truth state layer using StateFlow and SharedFlow.
- A complete target architecture, data model, auction design, payment flow and notification flow.
- A documented test suite combining source unit tests, a logic port run, walkthroughs and planned tests.

12.3 Technical Contribution

The technical contribution of the project lies in three areas. First, a **clear formulation of group dynamic pricing** as a deterministic step function with explicit validation rules and edge-case behaviour, which can run identically on the client and on the server. Second, a **state architecture** in which all screens observe one immutable source of truth, making multi-screen consistency a structural property rather than a testing burden. Third, a **migration path** from an in-memory prototype to a serverless, real-time backend without changing the UI contract, together with an auction design that protects members through a price ceiling.

12.4 Future Enhancements

Table 12.1: Future Enhancements and Priority

Enhancement	Description	Priority
Firebase backend	Authentication, Firestore, Cloud Functions, Storage, FCM (Chapter 5)	High
Locality-filtered feed	Show only deals for the selected locality; Google Places search	High
Supplier reverse auction	Bid submission, validation, ranking and automatic closing	High
Razorpay payments	Server-created orders and signature verification	High
Request-to-deal conversion	Convert requests that reach threshold into deals	Medium
Defect fixes D1 and D2	State recomputation on leave; bound next-tier value	High
Dynamic trust score	Update score from payments, cancellations and referrals	Medium
Multi-language UI	Tamil and Hindi localisation	Medium
Referral rewards	Rewards for inviting neighbours who complete purchases	Low

12.5 AI/ML Opportunities

GroupDeal does not use artificial intelligence or machine learning; its pricing is entirely rule-based. Once real transaction data is available, the following **future** AI/ML features could be explored, each as an assistant to the transparent rules rather than a replacement for them:

- **Demand forecasting** — predicting, from request and purchase history, which products a locality is likely to need and when.
- **Personalised recommendations** — suggesting deals based on a member's past purchases and locality trends.
- **Demand trend detection** — identifying rising requests early so suppliers can prepare bids.
- **Fraud detection** — flagging suspicious joins, fake accounts or payment patterns.
- **Supplier selection assistance** — combining price, delivery performance and ratings into a recommended ranking for administrators.

- **Tier schedule suggestions** — helping suppliers choose thresholds and prices that are likely to be reached, while the published schedule remains fixed for each deal.

12.6 Cloud Scalability

The serverless Firebase design scales with usage: Firestore distributes documents automatically, and Cloud Functions scale with the number of events. To keep costs and contention low, each deal is a separate document with members in a sub-collection, member counts are updated in transactions (or distributed counters for very popular deals), listeners are scoped to the user's locality, and images are served from Cloud Storage with caching.

12.7 Supplier/Admin Web Dashboard

Suppliers and administrators work better on larger screens. A web dashboard backed by the same Firestore project would let suppliers create deals with validated tier schedules, monitor auctions and manage fulfilment, and let administrators verify suppliers, moderate requests and deals, and resolve disputes. Role-based access control would be enforced through custom claims in Firebase Authentication and security rules.

12.8 Logistics Integration

Batch delivery to a locality collection point is central to the cost advantage. Future work includes integration with delivery partners for pickup scheduling and live tracking, collection-point management within apartment communities, and delivery-slot selection, so that the tracking timeline reflects real status updates.

12.9 Advanced Analytics

Analytics dashboards could show, for each locality, the number of active deals, completion rate, average time to reach target, total community savings and the most requested products. For suppliers, analytics on won auctions and delivery performance would support planning. All analytics would be built on aggregated data with appropriate privacy safeguards.

CHAPTER 13

REFERENCES

13.1 Books

[1] R. S. Pressman and B. R. Maxim, *Software Engineering: A Practitioner's Approach*, 9th ed. New York, NY, USA: McGraw-Hill Education, 2019.

[2] I. Sommerville, *Software Engineering*, 10th ed. Harlow, U.K.: Pearson, 2016.

[3] M. Fowler, *UML Distilled: A Brief Guide to the Standard Object Modeling Language*, 3rd ed. Boston, MA, USA: Addison-Wesley, 2003.

[4] K. T. Talluri and G. J. van Ryzin, *The Theory and Practice of Revenue Management*. New York, NY, USA: Springer, 2004.

13.2 Research Papers / Journals

[5] K. S. Anand and R. Aron, "Group buying on the web: A comparison of price-discovery mechanisms," *Management Science*, vol. 49, no. 11, pp. 1546–1562, 2003.

[6] R. J. Kauffman and B. Wang, "New buyers' arrival under dynamic pricing market microstructure: The case of group-buying discounts on the Internet," *Journal of Management Information Systems*, vol. 18, no. 2, pp. 157–188, 2001.

[7] R. J. Dolan, "Quantity discounts: Managerial issues and research opportunities," *Marketing Science*, vol. 6, no. 1, pp. 1–22, 1987.

[8] W. Elmaghraby and P. Keskinocak, "Dynamic pricing in the presence of inventory considerations: Research overview, current practices, and future directions," *Management Science*, vol. 49, no. 10, pp. 1287–1309, 2003.

[9] S. D. Jap, "An exploratory study of the introduction of online reverse auctions," *Journal of Marketing*, vol. 67, no. 3, pp. 96–107, 2003.

[10] J. Chen, X. Chen, and X. Song, "Comparison of the group-buying auction and the fixed pricing mechanism," *Decision Support Systems*, vol. 43, no. 2, pp. 445–459, 2007.

[11] G. Gallego and G. van Ryzin, "Optimal dynamic pricing of inventories with stochastic demand over finite horizons," *Management Science*, vol. 40, no. 8, pp. 999–1020, 1994.

[12] B. Jing and J. Xie, "Group buying: A new mechanism for selling through social interactions," *Management Science*, vol. 57, no. 8, pp. 1354–1372, 2011.

[13] T.-P. Liang and E. Turban, "Introduction to the special issue: Social commerce: A research framework for social commerce," *International Journal of Electronic Commerce*, vol. 16, no. 2, pp. 5–14, 2011.

[14] D. J. Kim, D. L. Ferrin, and H. R. Rao, "A trust-based consumer decision-making model in electronic commerce: The role of trust, perceived risk, and their antecedents," *Decision Support Systems*, vol. 44, no. 2, pp. 544–564, 2008.

[15] P. Resnick, K. Kuwabara, R. Zeckhauser, and E. Friedman, “Reputation systems,” *Communications of the ACM*, vol. 43, no. 12, pp. 45–48, 2000.

13.3 Official Technical Documentation

[16] Android Developers, “Guide to app architecture.” [Online]. Available: https://developer.android.com/topic/architecture

[17] Android Developers, “Jetpack Compose.” [Online]. Available: https://developer.android.com/compose

[18] JetBrains, “Coroutines guide,” Kotlin Documentation. [Online]. Available: https://kotlinlang.org/docs/coroutines-guide.html

[19] Android Developers, “StateFlow and SharedFlow.” [Online]. Available: https://developer.android.com/kotlin/flow/stateflow-and-sharedflow

[20] Google, “Firebase Authentication,” Firebase Documentation. [Online]. Available: https://firebase.google.com/docs/auth

[21] Google, “Transactions and batched writes,” Cloud Firestore Documentation. [Online]. Available: https://firebase.google.com/docs/firestore/manage-data/transactions

[22] Google, “Firebase Cloud Messaging,” Firebase Documentation. [Online]. Available: https://firebase.google.com/docs/cloud-messaging

[23] Razorpay, “Payment Gateway Documentation.” [Online]. Available: https://razorpay.com/docs/payments/

[24] Google, “Places SDK for Android,” Google Maps Platform Documentation. [Online]. Available: https://developers.google.com/maps/documentation/places/android-sdk/overview

[25] Google, “Material Design 3.” [Online]. Available: https://m3.material.io

[26] Coil Contributors, “Coil: Image loading for Android and Compose Multiplatform.” [Online]. Available: https://coil-kt.github.io/coil/

[27] Robolectric, “Robolectric: Fast and reliable Android unit tests.” [Online]. Available: https://robolectric.org

[28] Android Developers, “Dependency injection with Hilt.” [Online]. Available: https://developer.android.com/training/dependency-injection/hilt-android

[29] Google, “Cloud Functions for Firebase,” Firebase Documentation. [Online]. Available: https://firebase.google.com/docs/functions

[30] Google, “Firebase App Check,” Firebase Documentation. [Online]. Available: https://firebase.google.com/docs/app-check

[31] Android Developers, “Save data in a local database using Room.” [Online]. Available: https://developer.android.com/training/data-storage/room

[32] Square, “Retrofit: A type-safe HTTP client for Android and Java.” [Online]. Available: https://square.github.io/retrofit/

CHAPTER 14

APPENDIX

This appendix contains selected source code of the GroupDeal prototype, the project structure, the proposed data structures, the seed data, the test harness, the weekly PBL log and the team assessment. Source listings are reproduced from the project files; import statements are omitted for brevity. No API keys, passwords, tokens or service-account credentials are included.

A.1 Important Source Code

Listing A.1 shows the root navigation composable, which selects the top-level screen from the authentication state.

```
@Composable
fun AppNavigation() {
    val authState by GroupDealRepository.authState.collectAsState()
    var isManualOnboardingFlow by remember { mutableStateOf(false) }

    when (val state = authState) {
        is AuthUiState.Loading -> {
            Box(
                modifier = Modifier
                    .fillMaxSize()
                    .background(Color.White),
                contentAlignment = Alignment.Center
            ) {
                CircularProgressIndicator(
                    color = Emerald40,
                    modifier = Modifier.size(48.dp)
                )
            }
        }

        is AuthUiState.Unauthenticated -> {
            if (isManualOnboardingFlow) {
                OnboardingScreen(onNavigateBack = { isManualOnboardingFlow = false })
            } else {
                LoginScreen(onNavigateToOnboarding = { isManualOnboardingFlow = true })
            }
        }

        is AuthUiState.OnboardingRequired -> {
            OnboardingScreen(onNavigateBack = { GroupDealRepository.logout() })
        }

        is AuthUiState.Authenticated -> {
            MainAppShell()
        }
    }
}
```

Listing A.1: AppNavigation.kt — Auth-State Driven Navigation (lines 68–104)

A.2 PricingEngine Code

```
package com.example.core.engine

// imports omitted

object PricingEngine {

    /**
     * Determines current unit price strictly based on active tier matching current member
count.
     * If member count is below first tier, falls back to baseUnitPrice.
     */
    fun calculateUnitPrice(
        baseUnitPrice: Double,
        tiers: List<PricingTier>,
        currentMembers: Int
    ): Double {
        if (tiers.isEmpty() || currentMembers <= 0) return baseUnitPrice

        val activeTier = tiers
            .sortedByDescending { it.minMembers }
            .firstOrNull { currentMembers >= it.minMembers }

        return activeTier?.unitPrice ?: baseUnitPrice
    }

    /**
     * Finds current tier index in the ordered tiers list.
     */
    fun calculateTierIndex(tiers: List<PricingTier>, currentMembers: Int): Int {
        val sorted = tiers.sortedBy { it.minMembers }
        var activeIndex = -1
        for (i in sorted.indices) {
            if (currentMembers >= sorted[i].minMembers) {
                activeIndex = i
            }
        }
        return activeIndex
    }

    /**
     * Validates that pricing tiers are strictly monotonic:
     * - Thresholds strictly increase (minMembers)
     * - Unit prices strictly decrease (unitPrice)
     */
    fun validateTiers(tiers: List<PricingTier>): Boolean {
        if (tiers.isEmpty()) return false
        for (i in 0 until tiers.size - 1) {
            if (tiers[i].minMembers >= tiers[i + 1].minMembers) return false
            if (tiers[i].unitPrice <= tiers[i + 1].unitPrice) return false
        }
        return true
    }

    /**
     * Computes the members remaining to unlock the next cheaper tier.
     */
    fun membersToNextTier(tiers: List<PricingTier>, currentMembers: Int): Int? {
```

```
        val sorted = tiers.sortedBy { it.minMembers }
        val nextTier = sorted.firstOrNull { it.minMembers > currentMembers }
        return nextTier?.let { it.minMembers - currentMembers }
    }
}
```

Listing A.2: PricingEngine.kt (complete)

A.3 Data Models

```
package com.example.core.model

enum class GroupState(val label: String, val badgeColorHex: Long) {
    OPEN("Open for Joining", 0xFF0F9D58),
    FILLING("Filling Fast 🔥", 0xFFE65100),
    TARGET_REACHED("Target Reached 🎉", 0xFF2E7D32),
    BIDDING("Supplier Bidding Active ⚡", 0xFF6D28D9),
    SUPPLIER_SELECTED("Supplier Selected", 0xFF0284C7),
    PAYMENT_PENDING("Payment Window Open", 0xFFD97706),
    PROCESSING("Order In Batching", 0xFF475569),
    DELIVERED("Delivered to Locality", 0xFF15803D),
    EXPIRED("Expired", 0xFF64748B),
    CANCELLED("Cancelled", 0xFFDC2626)
}

data class PricingTier(
    val minMembers: Int,
    val unitPrice: Double,
    val discountPercent: Int,
    val label: String
)

data class SupplierBid(
    val id: String,
    val supplierId: String,
    val supplierName: String,
    val supplierScore: Double,
    val bidAmountPerUnit: Double,
    val promisedDeliveryDays: Int,
    val isSelected: Boolean = false,
    val verifiedBadge: Boolean = true
)

data class GroupDeal(
    val id: String,
    val title: String,
    val description: String,
    val category: String,
    val imageUrl: String,
    val originalMarketPrice: Double,
    val baseUnitPrice: Double,
    val currentUnitPrice: Double,
    val targetMembers: Int,
    val currentMembers: Int,
    val currentTierIndex: Int,
    val tiers: List<PricingTier>,
    val state: GroupState,
    val locality: Locality,
    val expiresAtMillis: Long,
```

```
    val joinedUserIds: List<String> = emptyList(),
    val winningBid: SupplierBid? = null,
    val unitsReserved: Int = 0,
    val brand: String = "",
    val specifications: Map<String, String> = emptyMap()
) {
    val activeTier: PricingTier?
        get() = if (currentTierIndex >= 0 && currentTierIndex < tiers.size)
tiers[currentTierIndex] else null

    val nextTier: PricingTier?
        get() = if (currentTierIndex + 1 < tiers.size) tiers[currentTierIndex + 1] else null

    val membersToNextTier: Int
        get() = (nextTier?.minMembers ?: targetMembers) - currentMembers

    val progressToTarget: Float
        get() = (currentMembers.toFloat() / targetMembers.toFloat()).coerceIn(0f, 1f)

    val currentSavingsPercent: Int
        get() = if (originalMarketPrice > 0) {
            (((originalMarketPrice - currentUnitPrice) / originalMarketPrice) * 100).toInt()
        } else 0

    val totalCommunitySavings: Double
        get() = (originalMarketPrice - currentUnitPrice) * currentMembers
}

data class GroupRequest(
    val id: String,
    val productName: String,
    val category: String,
    val targetPrice: Double,
    val referenceMarketPrice: Double,
    val currentVotes: Int,
    val thresholdVotes: Int = 20,
    val requesterName: String,
    val locality: Locality,
    val votedUserIds: List<String> = emptyList(),
    val createdAtMillis: Long = System.currentTimeMillis()
) {
    val progress: Float
        get() = (currentVotes.toFloat() / thresholdVotes.toFloat()).coerceIn(0f, 1f)
}
```

Listing A.3: GroupDeal.kt — GroupState, PricingTier, SupplierBid, GroupDeal, GroupRequest

```
package com.example.core.model

enum class OrderStatus(val displayName: String, val badgeColorHex: Long) {
    CONFIRMED("Order Confirmed", 0xFF0F9D58),
    SUPPLIER_ASSIGNED("Supplier Sourced", 0xFF0284C7),
    DISPATCHED("Dispatched in Bulk", 0xFF6D28D9),
    OUT_FOR_DELIVERY("Out for Locality Delivery", 0xFFD97706),
    DELIVERED("Delivered", 0xFF15803D),
    CANCELLED("Refunded / Cancelled", 0xFFDC2626)
}

data class TrackingStep(
    val title: String,
    val description: String,
    val timestampText: String,
    val isCompleted: Boolean
)

data class Order(
    val id: String,
    val orderNumber: String,
    val groupDealId: String,
    val productTitle: String,
    val productImageUrl: String,
    val quantity: Int,
    val unitPrice: Double,
    val totalAmount: Double,
    val originalMarketTotal: Double,
    val savedAmount: Double,
    val status: OrderStatus,
    val orderDateMillis: Long,
    val deliveryLocality: Locality,
    val deliveryAddressLine: String,
    val winningSupplierName: String = "Apex Direct Wholesale",
    val trackingSteps: List<TrackingStep> = emptyList()
)
```

Listing A.4: Order.kt — OrderStatus, TrackingStep, Order

```
package com.example.core.model

enum class UserRole(val displayName: String) {
    CONSUMER("Consumer / Buyer"),
    SUPPLIER("Verified Supplier"),
    ADMIN("Platform Admin")
}

enum class TrustTier(val label: String, val minScore: Int, val badgeColorHex: Long) {
    BRONZE("Bronze Member", 0, 0xFFCD7F32),
    SILVER("Silver Verified", 400, 0xFF94A3B8),
    GOLD("Gold Trusted", 700, 0xFFF59E0B),
    PLATINUM("Platinum VIP", 900, 0xFF06B6D4);

    companion object {
        fun fromScore(score: Int): TrustTier {
            return when {
                score >= 900 -> PLATINUM
                score >= 700 -> GOLD
                score >= 400 -> SILVER
```

```
                else -> BRONZE
            }
        }
    }
}

data class Locality(
    val pincode: String,
    val localityName: String,
    val city: String,
    val state: String,
    val activeGroupsCount: Int = 0,
    val activeDemandsCount: Int = 0
) {
    val fullDisplayName: String
        get() = "$localityName, $city ($pincode)"

    val shortDisplayName: String
        get() = "$localityName, $pincode"
}

data class TrustProfile(
    val score: Int = 650,
    val completedPurchases: Int = 8,
    val successfulGroupsInitiated: Int = 3,
    val neighborhoodReferrals: Int = 5,
    val cancellationRatePercent: Double = 0.0,
    val promptPaymentRatePercent: Double = 100.0,
    val trustBadges: List<String> = listOf("Verified Phone", "Prompt Payer", "Locality Anchor")
) {
    val tier: TrustTier
        get() = TrustTier.fromScore(score)
}

data class UserModel(
    val id: String,
    val firebaseUid: String,
    val fullName: String,
    val email: String,
    val phone: String,
    val role: UserRole = UserRole.CONSUMER,
    val locality: Locality? = null,
    val trustProfile: TrustProfile = TrustProfile(),
    val isOnboarded: Boolean = true,
    val joinedAtMillis: Long = System.currentTimeMillis()
)
```

Listing A.5: User.kt — UserRole, TrustTier, Locality, TrustProfile, UserModel

Listing A.6 reproduces the repository functions for group membership and voting. The pincode lookup of LocalityService is reproduced in abridged form in Listing 7.4.

```
    fun joinGroup(dealId: String) {
        val currentUser = (_authState.value as? AuthUiState.Authenticated)?.user ?: return
        val currentList = _deals.value.toMutableList()
        val index = currentList.indexOfFirst { it.id == dealId }
        if (index == -1) return
```

```
        val deal = currentList[index]
        if (deal.joinedUserIds.contains(currentUser.id)) return

        val newMemberCount = deal.currentMembers + 1
        val newJoinedList = deal.joinedUserIds + currentUser.id
        val newTierIndex = PricingEngine.calculateTierIndex(deal.tiers, newMemberCount)
        val newPrice = PricingEngine.calculateUnitPrice(deal.baseUnitPrice, deal.tiers,
newMemberCount)

        val newState = when {
            newMemberCount >= deal.targetMembers -> GroupState.TARGET_REACHED
            newMemberCount >= (deal.targetMembers * 0.6) -> GroupState.FILLING
            else -> GroupState.OPEN
        }

        val updatedDeal = deal.copy(
            currentMembers = newMemberCount,
            joinedUserIds = newJoinedList,
            currentTierIndex = newTierIndex,
            currentUnitPrice = newPrice,
            state = newState,
            unitsReserved = deal.unitsReserved + 1
        )
        currentList[index] = updatedDeal
        _deals.value = currentList

        // Emit live activity
        scope.launch {
            _liveEvents.emit(
                LocalityEvent(
                    message = "🎉 You joined '${deal.title}' in ${deal.locality.localityName}!
Price is now ₹${newPrice.toInt()}",
                    localityName = deal.locality.localityName
                )
            )
        }
    }

    fun leaveGroup(dealId: String) {
        val currentUser = (_authState.value as? AuthUiState.Authenticated)?.user ?: return
        val currentList = _deals.value.toMutableList()
        val index = currentList.indexOfFirst { it.id == dealId }
        if (index == -1) return

        val deal = currentList[index]
        if (!deal.joinedUserIds.contains(currentUser.id)) return

        val newMemberCount = (deal.currentMembers - 1).coerceAtLeast(0)
        val newJoinedList = deal.joinedUserIds - currentUser.id
        val newTierIndex = PricingEngine.calculateTierIndex(deal.tiers, newMemberCount)
        val newPrice = PricingEngine.calculateUnitPrice(deal.baseUnitPrice, deal.tiers,
newMemberCount)

        currentList[index] = deal.copy(
            currentMembers = newMemberCount,
            joinedUserIds = newJoinedList,
            currentTierIndex = newTierIndex,
            currentUnitPrice = newPrice,
            unitsReserved = (deal.unitsReserved - 1).coerceAtLeast(0)
```

```
        )
        _deals.value = currentList
    }

    fun voteForRequest(requestId: String) {
        val currentUser = (_authState.value as? AuthUiState.Authenticated)?.user ?: return
        val list = _requests.value.toMutableList()
        val index = list.indexOfFirst { it.id == requestId }
        if (index == -1) return

        val req = list[index]
        if (req.votedUserIds.contains(currentUser.id)) return

        val updated = req.copy(
            currentVotes = req.currentVotes + 1,
            votedUserIds = req.votedUserIds + currentUser.id
        )
        list[index] = updated
        _requests.value = list

        scope.launch {
            _liveEvents.emit(
                LocalityEvent(
                    message = "👍 Voted for '${req.productName}'! (${updated.currentVotes}/$
{req.thresholdVotes} neighbors interested)",
                    localityName = req.locality.localityName
                )
            )
        }
    }
```

Listing A.6: GroupDealRepository.kt — joinGroup, leaveGroup, voteForRequest

A.4 Repository Structure

```
app/
├── build.gradle.kts                (module build script, Kotlin DSL)
└── src/
    ├── main/
    │   ├── AndroidManifest.xml
    │   ├── res/ (values/strings.xml, themes, launcher icons)
    │   └── java/com/example/
    │       ├── MainActivity.kt
    │       ├── core/
    │       │   ├── data/    GroupDealRepository.kt, LocalityService.kt
    │       │   ├── engine/  PricingEngine.kt
    │       │   └── model/   GroupDeal.kt, Order.kt, User.kt
    │       └── ui/
    │           ├── auth/        LoginScreen, OnboardingScreen, LocalityPickerModal
    │           ├── components/  GDBadges
    │           ├── detail/      GroupDetailScreen
    │           ├── discovery/   DiscoverHomeScreen, GroupDealCard
    │           ├── mygroups/    MyGroupsScreen
    │           ├── navigation/  AppNavigation
    │           ├── orders/      OrdersScreen
    │           ├── profile/     ProfileScreen
    │           ├── requests/    RequestGroupScreen
    │           └── theme/       Color, Theme, Type
    ├── test/java/com/example/          ExampleRobolectricTest, ExampleUnitTest,
    │                                   GreetingScreenshotTest
    └── androidTest/java/com/example/   ExampleInstrumentedTest
gradle/libs.versions.toml             (version catalog)
```

Listing A.7: Repository Structure of the GroupDeal Project

A.5 Database / Data Structure

The prototype has no database. Listing A.8 shows how a deal would be stored as a Cloud Firestore document in the proposed system, using the seeded ghee deal as the example. Members are stored in a sub-collection so that a join is a single transactional write.

```
// Proposed Firestore document: /deals/{dealId}
{
  "title": "Royal Organic Royal A2 Gir Cow Ghee (1L Pet Jar)",
  "category": "Groceries & Farm",
  "brand": "Vedic Organics",
  "localityId": "560034",
  "originalMarketPrice": 1400,
  "baseUnitPrice": 1250,
  "tiers": [ { "minMembers": 5,  "unitPrice": 1099 },
             { "minMembers": 15, "unitPrice": 949 },
             { "minMembers": 30, "unitPrice": 799 } ],
  "targetMembers": 30,
  "memberCount": 18,            // written only by Cloud Functions
  "currentUnitPrice": 949,      // written only by Cloud Functions
  "currentTierIndex": 1,
  "state": "FILLING",
  "expiresAt": <timestamp>
}
// /deals/{dealId}/members/{uid}   -> { joinedAt, quantity }
// /deals/{dealId}/bids/{bidId}    -> { supplierId, pricePerUnit, deliveryDays, status }
```

Listing A.8: Proposed Firestore Document for a Group Deal

A.6 Seed Data and Localities

Table 14.1: Seeded Group Deals

Deal	Category	MRP	Base	Tiers (members → price)	Members	Seeded state
A2 Gir Cow Ghee (1 L)	Groceries & Farm	₹1,400	₹1,250	5 → ₹1,099; 15 → ₹949; 30 → ₹799	18/30	FILLING
Alphonso Mangoes (dozen)	Fresh Produce	₹2,500	₹2,100	10 → ₹1,850; 25 → ₹1,599; 50 → ₹1,349	34/50	FILLING
Groundnut Oil (5 L)	Pantry Essentials	₹600	₹520	8 → ₹449; 20 → ₹379; 40 → ₹319	40/40	TARGET_REACHED
Kashmir Saffron (5 g)	Gourmet & Spices	₹4,000	₹3,600	5 → ₹3,200; 12 → ₹2,750; 20 → ₹2,299	14/20	BIDDING

Table 14.2: Seeded Demand Requests

Product	Category	Target price	Market price	Votes
Daawat Super Basmati Rice (25 kg)	Pantry Essentials	₹2,800	₹3,600	17/20
Organic Kashmiri Walnuts (1 kg)	Dry Fruits	₹850	₹1,250	12/15
Cold Pressed Virgin Coconut Oil (5 L)	Pantry Essentials	₹1,100	₹1,600	9/15

Table 14.3: Seeded Localities in LocalityService

City	Localities (pincode)
Bengaluru	Koramangala (560034), HSR Layout (560102), Indiranagar (560038), Electronic City (560100), Whitefield (560066)
New Delhi	Connaught Place (110001)
Gurugram	DLF Cyber City (122002)
Noida	Sector 18 (201301)
Mumbai	Bandra West (400050), Powai (400076), Andheri West (400053)
Hyderabad	HITEC City (500081), Gachibowli (500032)
Pune	Hinjewadi (411057)
Chennai	Perungudi OMR (600096)

The Saffron deal is seeded in the BIDDING state to demonstrate the auction label; in the prototype this state is set in the seed data and is not computed by `joinGroup`.

A.7 Detailed Test Cases

Listing A.9 shows the core of the logic port used in Chapter 9. Each function mirrors the corresponding Kotlin function statement by statement.

```
import json, math
def calculateUnitPrice(base, tiers, n):
    if not tiers or n <= 0: return base
    for t in sorted(tiers, key=lambda t: -t['minMembers']):
        if n >= t['minMembers']: return t['unitPrice']
    return base
def calculateTierIndex(tiers, n):
    s = sorted(tiers, key=lambda t: t['minMembers']); idx = -1
    for i, t in enumerate(s):
        if n >= t['minMembers']: idx = i
    return idx
def validateTiers(tiers):
    if not tiers: return False
    for a, b in zip(tiers, tiers[1:]):
        if a['minMembers'] >= b['minMembers']: return False
        if a['unitPrice'] <= b['unitPrice']: return False
    return True
def membersToNextTier(tiers, n):
    nxt = next((t for t in sorted(tiers, key=lambda t: t['minMembers']) if t['minMembers'] >
n), None)
    return None if nxt is None else nxt['minMembers'] - n
def trust(score): return 'PLATINUM' if score >= 900 else 'GOLD' if score >= 700 else 'SILVER'
if score >= 400 else 'BRONZE'
POP = {'560034': 'Koramangala'}
def lookup(p):
    p = p.strip()
    if p in POP: return POP[p]
    if len(p) == 6 and p.isdigit():
        pre = p[:2]; m =
{'11':'Delhi','12':'Gurugram','13':'Gurugram','20':'Noida','21':'Noida','40':'Mumbai','41':'Pun
e','50':'Hyderabad','56':'Bengaluru','57':'Bengaluru','60':'Chennai','61':'Chennai','70':'Kolka
ta'}
        return f"Sector / Ward {p} ({m.get(pre,'Metro City')})"
    return None
```

Listing A.9: Logic Port of PricingEngine, TrustTier and LocalityService (excerpt)

```
package com.example

import android.content.Context
import androidx.test.core.app.ApplicationProvider
import com.example.core.data.GroupDealRepository
import com.example.core.data.LocalityService
import com.example.core.engine.PricingEngine
import com.example.core.model.PricingTier
import com.example.core.model.TrustTier
import org.junit.Assert.assertEquals
import org.junit.Assert.assertNotNull
import org.junit.Assert.assertTrue
import org.junit.Test
import org.junit.runner.RunWith
import org.robolectric.RobolectricTestRunner
import org.robolectric.annotation.Config
```

```
@RunWith(RobolectricTestRunner::class)
@Config(sdk = [36])
class ExampleRobolectricTest {

  @Test
  fun `read string from context`() {
    val context = ApplicationProvider.getApplicationContext<Context>()
    val appName = context.getString(R.string.app_name)
    assertEquals("GroupDeal", appName)
  }

  @Test
  fun `pricing engine calculates correct dynamic discount based on active tier`() {
    val tiers = listOf(
      PricingTier(minMembers = 5, unitPrice = 1000.0, discountPercent = 20, label = "Tier 1"),
      PricingTier(minMembers = 15, unitPrice = 800.0, discountPercent = 35, label = "Tier 2"),
      PricingTier(minMembers = 30, unitPrice = 600.0, discountPercent = 50, label = "Tier 3")
    )
    val basePrice = 1200.0

    // 0 members -> base price
    assertEquals(1200.0, PricingEngine.calculateUnitPrice(basePrice, tiers, 0), 0.01)
    // 3 members -> base price (below min tier)
    assertEquals(1200.0, PricingEngine.calculateUnitPrice(basePrice, tiers, 3), 0.01)
    // 5 members -> Tier 1 (1000.0)
    assertEquals(1000.0, PricingEngine.calculateUnitPrice(basePrice, tiers, 5), 0.01)
    // 18 members -> Tier 2 (800.0)
    assertEquals(800.0, PricingEngine.calculateUnitPrice(basePrice, tiers, 18), 0.01)
    // 35 members -> Tier 3 (600.0)
    assertEquals(600.0, PricingEngine.calculateUnitPrice(basePrice, tiers, 35), 0.01)
  }

  @Test
  fun `locality service correctly resolves pincode and searches areas`() {
    val koramangala = LocalityService.lookupPincode("560034")
    assertNotNull(koramangala)
    assertEquals("Koramangala", koramangala?.localityName)
    assertEquals("Bengaluru", koramangala?.city)

    val searchResults = LocalityService.searchLocalities("Mumbai")
    assertTrue(searchResults.isNotEmpty())
  }

  @Test
  fun `trust score tier calculation properly maps scores`() {
    assertEquals(TrustTier.BRONZE, TrustTier.fromScore(250))
    assertEquals(TrustTier.SILVER, TrustTier.fromScore(500))
    assertEquals(TrustTier.GOLD, TrustTier.fromScore(780))
    assertEquals(TrustTier.PLATINUM, TrustTier.fromScore(950))
  }
}
```

Listing A.10: ExampleRobolectricTest.kt — Unit Tests in the Project

A.8 Weekly PBL Progress Log

Table 14.4: Weekly PBL Progress Log

Week	Activities / Work Completed	Mentor Sign-Off
Week 1	Identified the problem of individual retail pricing and fragmented local demand; proposed locality-based group buying as the project idea.	
Week 2	Studied existing purchasing channels and group-buying literature; defined objectives, scope, target users and the feature list.	
Week 3	Prepared and presented the zeroth review (23-07-2026): problem statement, scope, planned technology stack and project plan.	
Week 4	Designed the data model (deals, tiers, requests, orders, users) and the navigation flow; created Material 3 screen layouts.	
Week 5	Set up the Kotlin/Compose project; implemented the models, PricingEngine and LocalityService with seed data.	
Week 6	Implemented GroupDealRepository with StateFlows; built Discover, Group Detail and the locality picker; join/leave with dynamic pricing.	
Week 7	Implemented demand requests and upvotes, My Groups, Orders with tracking, and Profile with trust score; login and onboarding flows.	
Week 8	Wrote unit tests for pricing, locality and trust tiers; walkthrough testing; identified defects D1 and D2.	
Week 9	Designed the complete system: Firebase backend, reverse auction, Razorpay payment flow and notifications; prepared the poster.	
Week 10	Prepared diagrams, test documentation and this report; final review of implemented versus proposed features.	

A.9 Team Responsibilities and Self / Peer Assessment

Table 14.5: Self and Peer Assessment

Team Member	Self-Rated Contribution (%)	Peer-Rated Contribution (%)	Remarks
Arul Murugan M (24CS0084)	50%	50%	Project planning, UI/UX design, Compose screens, navigation, UI validation, poster and documentation.
Ritvik Doddapaneni (24CS0755)	50%	50%	Data models, repository and state layer, pricing engine, locality service, unit tests, backend and payment design, documentation.

A.10 Additional Diagram — Proposed Firestore Security Rules

Listing A.11 sketches the proposed Firestore security rules that enforce the server-authoritative design described in Sections 7.18 and 11.3. It is a design artefact for the complete system and is not part of the prototype.

```
rules_version = '2';
service cloud.firestore {
  match /databases/{db}/documents {
    match /deals/{dealId} {
      allow read: if request.auth != null;
      allow write: if false;                      // prices and counts: Cloud Functions only
      match /members/{uid} {
        allow read: if request.auth != null;
        allow create, delete: if request.auth.uid == uid;
      }
      match /bids/{bidId} {
        allow create: if request.auth.token.role == 'supplier';
        allow read: if request.auth.token.role in ['supplier', 'admin'];
      }
    }
    match /requests/{reqId}/votes/{uid} {
      allow create: if request.auth.uid == uid;    // one vote per user
    }
    match /orders/{orderId} {
      allow read: if request.auth.uid == resource.data.uid;
      allow write: if false;                      // created/paid via Cloud Functions
    }
  }
}
```

Listing A.11: Proposed Firestore Security Rules (Design Sketch)

A.11 Project Repository Information

Table 14.6: Project Build and Run Information

Item	Detail
Project name	GroupDeal
Module	app (single Android application module)
Package	com.example
Build	./gradlew assembleDebug
Unit tests	./gradlew testDebugUnitTest
Run	Open in Android Studio, select an emulator or device (API 24+), Run ‘app’
Demo sign-in	Any email and password; an email containing “new” opens onboarding
Secrets	Managed with the Secrets Gradle plugin; no keys are stored in the repository